BỘ KHOA HỌC VÀ CÔNG NGHỆ

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

KHOA CÔNG NGHỆ THÔNG TIN 1

ĐỒ ÁN KẾT THÚC HỌC PHẦN

HỌC PHẦN: CƠ SỞ DỮ LIỆU PHÂN TÁN

Đề tài: Hệ Thống Quản Lý Ngân Hàng Phân Tán

Giảng viên hướng dẫn	**: Ts. Kim Ngọc Bách**
Nhóm môn học	**: 10**
Nhóm bài tập lớn	**: 06**
Danh sách thành viên	**MSV**
Mai Anh Đức	**B23DCCN172 (Nhóm trưởng)**
Trịnh Anh Tú	**B23DCCN876**
Trần Đăng Dương	**B23DCCN227**

HÀ NỘI, 2026

LỜI CẢM ƠN

Nhóm chúng em xin bày tỏ lòng biết ơn sâu sắc đến thầy **TS. Kim Ngọc Bách** đã tận tình hướng dẫn, hỗ trợ và tạo mọi điều kiện thuận lợi trong suốt quá trình thực hiện đồ án.

Những kiến thức chuyên môn, sự chỉ bảo tận tâm cùng những góp ý quý báu của thầy đã giúp chúng em có định hướng đúng đắn trong quá trình nghiên cứu, thiết kế và triển khai đề tài. Thông qua đồ án này, chúng em không chỉ củng cố được kiến thức đã học trên giảng đường mà còn có cơ hội vận dụng vào thực tiễn, từ đó nâng cao kỹ năng nghiên cứu, làm việc nhóm và giải quyết vấn đề.

Mặc dù đã nỗ lực hoàn thành đồ án với tinh thần nghiêm túc và trách nhiệm cao, nhưng do kiến thức và kinh nghiệm thực tế còn hạn chế nên báo cáo khó tránh khỏi những thiếu sót. Chúng em rất mong nhận được những ý kiến nhận xét và đóng góp từ thầy để có thể tiếp tục hoàn thiện và phát triển hơn trong tương lai.

Một lần nữa, chúng em xin chân thành cảm ơn thầy **TS. Kim Ngọc Bách** đã luôn đồng hành, hướng dẫn và hỗ trợ chúng em trong suốt thời gian thực hiện đồ án.

Kính chúc thầy luôn dồi dào sức khỏe, hạnh phúc và gặt hái nhiều thành công trong sự nghiệp giảng dạy và nghiên cứu khoa học.

Nhóm thực hiện

- **Mai Anh Đức**
- **Trần Đăng Dương**
- **Trịnh Anh Tú**

MỤC LỤC

LỜI CẢM ƠN *2*

DANH MỤC HÌNH ẢNH *5*

DANH MỤC BẢNG BIỂU *8*

PHÂN CÔNG CÔNG VIỆC VÀ MỨC ĐỘ HOÀN THÀNH *9*

CHƯƠNG 1. PHÂN TÍCH BÀI TOÁN *13*

1.1 Bối cảnh đề tài **13**

1.2 Mục tiêu của hệ thống **14**

1.3 Các chức năng chính **14**

1.4 Công nghệ sử dụng **15**

1.5 Kiến trúc hệ thống **16**

1.6 Kết chương **17**

CHƯƠNG 2. THIẾT KẾ CƠ SỞ DỮ LIỆU PHÂN TÁN *19*

2.1 Lược đồ toàn cục **19**

2.2 Kiến trúc phân tán **25**

2.3 Phân mảnh dữ liệu **26**

2.4. Cấp phát dữ liệu (Data Allocation) **27**

2.5 Chiến lược nhân bản dữ liệu (Data Replication) **27**

2.6 Cơ sở lựa chọn phân mảnh ngang **28**

2.7 Ma trận phân phối dữ liệu **29**

2.8 Đánh giá ưu điểm của mô hình kiến trúc **29**

2.9 Kết chương **30**

CHƯƠNG 3. THIẾT KẾ GIAO DỊCH PHÂN TÁN *32*

3.1 Tổng quan về giao dịch phân tán **32**

3.2 Đặc tả giao dịch chuyển tiền liên chi nhánh **32**

3.3 Kiến trúc điều phối giao dịch **32**

3.4 Quy trình thực hiện và Mô phỏng giao thức Two-Phase Commit (2PC) **33**

3.5 Đảm bảo tính toàn vẹn dữ liệu (Tính chất ACID) **34**

3.6 Cơ chế mô phỏng lỗi, Rollback và Phục hồi (Crash Recovery) **35**

3.7 Đánh giá giải pháp thiết kế **36**

3.8 Kết chương **37**

CHƯƠNG 4. CÀI ĐẶT VÀ THỬ NGHIỆM HỆ THỐNG *38*

4.1 Môi trường phát triển **38**

4.1.1 Yêu cầu phần cứng 38

4.1.2 Yêu cầu phần mềm 38

4.1.3 Công nghệ sử dụng 39

4.2 Cài đặt cơ sở dữ liệu phân tán **40**

4.2.1 Kiến trúc cơ sở dữ liệu 40
4.2.2 Cấu hình Docker Compose 44
4.2.3 Khởi chạy cơ sở dữ liệu 51
4.2.4 Khởi tạo cấu trúc bảng 52

4.3 Cài đặt Backend **56**
4.3.1 Cấu trúc thư mục Backend 56
4.3.2 Cấu hình Multi-DataSource 62
4.3.3 Cấu hình CORS 63
4.3.4 Cài đặt REST API 65
4.3.5 Cài đặt xử lý giao dịch phân tán 69

4.4 Cài đặt Frontend **75**
4.4.1 Cấu trúc thư mục Frontend 75
4.4.2 Cấu hình Axios và Proxy 76
4.4.3 Điều hướng ứng dụng 78
4.4.4 Các chức năng chính 78
4.4.5 Chạy Frontend 80

4.5 Triển khai hệ thống **80**
4.5.1 Quy trình triển khai 81
4.5.2 Kiến trúc triển khai 82
4.5.3 Cổng mạng sử dụng 83

4.6 Kiểm thử hệ thống **84**
4.6.1 Kiểm thử kết nối 84
4.6.2 Kiểm thử chức năng gửi tiền(deposit) 87
4.6.3 Kiểm thử chức năng rút tiền và Xử lý tương tranh (Concurrency) 95
4.6.4 Kiểm thử chức năng chuyển tiền cùng chi nhánh (Internal Transfer) 107
4.6.5 Kiểm thử chức năng chuyển liên chi nhánh (Internal Transfer) 114
4.7 Các truy vấn phân tán (Distributed Queries) 132

4.8. Tổng kết chương **139**

CHƯƠNG 5: ĐÁNH GIÁ VÀ ĐỀ XUẤT CẢI TIẾN ***140***

5.1 Đánh giá hiệu năng hệ thống **140**
5.1.1 Hiệu năng giao dịch cục bộ (Local Transaction) 140
5.1.2 Hiệu năng giao dịch phân tán (Distributed Transaction — 2PC) 141
5.1.3 Hiệu năng truy vấn phân tán (Distributed Query) 142

5.2 So sánh phương án tập trung và phân tán **143**
5.2.1 Bảng so sánh tổng quát 143
5.2.2 Phân tích ưu nhược điểm của hệ thống phân tán đã triển khai 145

5.3 Đề xuất cải tiến hệ thống **146**
5.3.1 Cải tiến giao thức giao dịch phân tán 146
5.3.2 Cải tiến hiệu năng xử lý dữ liệu 146
5.3.3 Cải tiến tính sẵn sàng cao (High Availability) 147
5.3.4 Cải tiến an toàn bảo mật (Security) 147

5.4 Kết luận chương **148**

DANH MỤC HÌNH ẢNH

Hình 1.1 Kiến trúc tổng thể hệ thống ngân hàng phân tán....17
Hình 4.1 Cấu hình docker cho hệ thống....45
Hình 4.2 Khởi chạy docker....52
Hình 4.3 Tạo bảng chi nhánh....52
Hình 4.4 Tạo bảng khách hàng....53
Hình 4.5 Tạo bảng account....54
Hình 4.6 Tạo bảng lịch sử giao dịch....54
Hình 4.7 Tạo bảng theo dõi giao dịch phân tán....55
Hình 4.8 Tạo bảng thành phần tham gia....56
Hình 4.9 Cấu hình Multi-DataSource....62
Hình 4.10 Cấu hình datasource cho site Hà Nội....63
Hình 4.11 Cấu hình CORS....64
Hình 4.12 Pessimistic Locking....70
Hình 4.13 Ordered Locking....71
Hình 4.14 Cấu hình Proxy....77
Hình 4.15 Cấu hình Axios....77
Hình 4.16 Kiểm thử kết nối....85
Hình 4.17 Kiểm tra kết nối tới từng site....86
Hình 4.18 Payload gửi lên API POST /api/transactions/deposit....87
Hình 4.19 Lệnh curl gọi API Backend - deposit....88
Hình 4.20 Kết quả trả về - deposit....88
Hình 4.21 Bảng account trước khi thực hiện deposit....89
Hình 4.22 Bảng transaction_history trước khi thực hiện deposit....89
Hình 4.23 Thao tác chức năng gửi tiền - deposit....89
Hình 4.24 Bảng account sau khi thực hiện deposit....90
Hình 4.25 Bảng transaciton_history sau khi thực hiện deposit....90
Hình 4.26 Đoạn code Mô phỏng sự cố Server Crash trước khi Commit....90
Hình 4.27 Kết quả trả về khi server bị crash....91
Hình 4.28 Bảng account trước khi deposit (server crassh)....92
Hình 4.29 Bảng transaction_history trước khi deposit (server crash)....92
Hình 4.30 Thao tác deposit - server crash....92
Hình 4.31 Bảng account sau khi deposit - server crash....93
Hình 4.32 Bảng transaction_history sau khi deposit - server crash....93
Hình 4.33 Kết quả thực thi deposit từng bước khi server crash....94
Hình 4.34 Bảng account trước khi thực hiện withdraw....95
Hình 4.35 Bảng transaction_history sau trước khi thực hiện withdraw....96
Hình 4.36 Thực hiện withdraw trên giao diện....96
Hình 4.37 Bảng account sau khi thực hiện withdraw....96
Hình 4.38 Bảng transaction_history sau khi thực hiện withdraw....97
Hình 4.39 Quá trình thực hiện rút tiền....97
Hình 4.40 Kết quả Json trả về rút tiền đồng thời không xử lý tương tranh....99
Hình 4.41 Bảng account trước khi rút tiền - không xử lý tương tranh....100
Hình 4.42 Bảng transaction_history trước khi thực hiện rút tiền đồng thời - không xử lý tương tranh....100

Hình 4.43 Thực hiện rút tiền đồng thời - không xử lý tương tranh 100
Hình 4.44 Bảng account sau khi thực hiện rút tiền đồng thời - không xử lý trương tranh 101
Hình 4.45 Bảng transaction_history sau khi thực hiện rút tiền đồng thời - không xử lý tương tranh 101
Hình 4.46 Mô phỏng quá trình rút tiền đồng thời - không xử lý tương tranh 102
Hình 4.47 Kết quả trả về khi thực hiện rút tiền đồng thời - SỬ DỤNG Khóa (Pessimistic Locking) dưới dạng JSON 103
Hình 4.48 Bảng account trước khi thực hiện rút tiền đồng thời - sử dụng Pessimistic Lock 104
Hình 4.49 Bảng transaction_history trước khi thực hiện rút tiền đồng thời - sử dụng Pessimistic Lock 104
Hình 4.50 Thực hiện rút tiền đồng thời - sử dụng Pessimistic Lock trên giao diện 104
Hình 4.51 Bảng account sau khi thực hiện rút tiền đồng thời - sử dụng Pessimistic LOCK 105
Hình 4.52 Bảng transaction_history sau khi thực hiện rút tiền đồng thời - sử dụng Pessimistic Lock 105
Hình 4.53 Mô phỏng các bước rút tiền đồng thời - sử dụng Pessimistic Lock 106
Hình 4.54 Payload gửi lên API POST /api/transfers/internal 108
Hình 4.55 Kết quả trả về dạng JSON sau khi thực hiện chuyển tiền cùng chi nhánh 109
Hình 4.56 Bảng account trước khi thực hiện chuyển tiền cùng chi nhánh 109
Hình 4.57 Bảng transaction_history trước khi thực hiện chuyển tiền cùng chi nhánh 110
Hình 4.58 Thực hiện rút tiền cùng chi nhánh trên giao diện 111
Hình 4.59 Bảng account sau khi thực hiện rút tiền cùng chi nhánh 111
Hình 4.60 Bảng transaction_history sau khi thực hiện rút tiền cùng chi nhánh 112
Hình 4.61 Mô phỏng quá trình thực hiện rút tiền cùng chi nhánh 112
Hình 4.62 Kết quả đưới dạng JSON sau khi thực hiện chuyển tiền liên chi nhánh - không xảy ra sự cố 115
Hình 4.63 Bảng account tại site HNt trước khi thực hiện chuyển tiền liên chi nhánh - không xảy ra sự cố 116
Hình 4.64 Bảng account tại site HCM trước khi thực hiện chuyển tiền liên chi nhánh - không xảy ra sự cố 116
Hình 4.65 Bảng transaction_history tại site HN trước khi thực hiện chuyển tiền liên chi nhánh - không xảy ra sự cố 117
Hình 4.66 Bảng transaction_history tại site HCM trước khi thực hiện chuyển tiền liên chi nhánh - không xảy ra sự cố 117
Hình 4.67 Bảng distributed_transaction_log tại site HN trước khi thực hiện chuyển tiền liên chi nhánh - không xảy ra sự cố 117
Hình 4.68 Bảng distributed_transaction_log tại site HCM trước khi thực hiện chuyển tiền liên chi nhánh - không xảy ra sự cố 118
Hình 4.69 Bảng transaction_participant tại site HN trước khi chuyển tiền liên chi nhánh - không xảy ra sự cố 118
Hình 4.70 Bảng transaction_participant tại site HCM trước khi chuyển tiền liên chi nhánh - không xảy ra sự cố 118
Hình 4.71 Thực hiện chuyển tiền liên chi nhánh trên giao diện - không xảy ra sự cố 119
Hình 4.72 Bảng account tại site HNt sau khi thực hiện chuyển tiền liên chi nhánh - không xảy ra sự cố 120
Hình 4.73 Bảng account tại site HCM sau khi thực hiện chuyển tiền liên chi nhánh - không xảy ra sự cố 120

Hình 4.74 Bảng transaction_history tại site HN sau khi thực hiện chuyển tiền liên chi nhánh - không xảy ra sự cố 121
Hình 4.75 Bảng transaction_history tại site HCM sau khi thực hiện chuyển tiền liên chi nhánh - không xảy ra sự cố 121
Hình 4.76 Bảng distributed_transaction_log tại site HN sau khi thực hiện chuyển tiền liên chi nhánh - không xảy ra sự cố 121
Hình 4.77 Bảng distributed_transaction_log tại site HCM sau khi thực hiện chuyển tiền liên chi nhánh - không xảy ra sự cố 122
Hình 4.78 Bảng transaction_participant tại site HN sau khi chuyển tiền liên chi nhánh - không xảy ra sự cố 122
Hình 4.79 Bảng transaction_participant tại site HCM sau khi chuyển tiền liên chi nhánh - không xảy ra sự cố 122
Hình 4.80 Kết quả mô phỏng quá trình chuyển tiền liên chi nhánh - không xảy ra sự cố 123
Hình 4.81 Kết quả trả về dưới dạng JSON thực hiện chuyển tiền liên chi nhánh - xảy ra lỗi ... 124
Hình 4.82 Bảng account tại site HN trước khi thực hiện chuyển tiền liên chi nhánh - xảy ra lỗi 125
Hình 4.83 Bảng account tại site HCM trước khi thực hiện chuyển tiền liên chi nhánh - xảy ra lỗi 125
Hình 4.84 Bảng transaction_history tại site HN trước khi thực hiện chuyển tiền liên chi nhánh - xảy ra lỗi 126
Hình 4.85 Bảng transaction_history tại site HCM trước khi thực hiện chuyển tiền liên chi nhánh - xảy ra lỗi 126
Hình 4.86 Bảng distributed_transaction_log tại site HN trước khi thực hiện chuyển tiền liên chi nhánh - xảy ra lỗi 126
Hình 4.87 Bảng distributed_transaction_log tại site HCM trước khi thực hiện chuyển tiền liên chi nhánh - xảy ra lỗi 127
Hình 4.88 Bảng transaction_participant tại site HN trước khi thực hiện chuyển tiền liên chi nhánh - xảy ra lỗi 127
Hình 4.89 Bảng transaction_participant tại site HCM trước khi thực hiện chuyển tiền liên chi nhánh - xảy ra lỗi 127
Hình 4.90 Thực hiện chuyển tiền liên chi nhánh trên giao diện - xảy ra lỗi 128
Hình 4.91 Bảng account tại site HN sau khi thực hiện chuyển tiền liên chi nhánh - xảy ra lỗi .. 129
Hình 4.92 Bảng account tại site HCM sau khi thực hiện chuyển tiền liên chi nhánh - xảy ra lỗi 129
Hình 4.93 Bảng transaction_history tại site HN sau khi thực hiện chuyển tiền liên chi nhánh - xảy ra lỗi 130
Hình 4.94 Bảng transaction_history tại site HCM sau khi thực hiện chuyển tiền liên chi nhánh - xảy ra lỗi 130
Hình 4.95 Bảng distributed_transaction_log tại site HN sau khi thực hiện chuyển tiền liên chi nhánh - xảy ra lỗi 130
Hình 4.96 Bảng distributed_transaction_log tại site HCM sau khi thực hiện chuyển tiền liên chi nhánh - xảy ra lỗi 131
Hình 4.97 Bảng transaction_participant tại site HN sau khi thực hiện chuyển tiền liên chi nhánh - xảy ra lỗi 131
Hình 4.98 Bảng transaction_participant tại site HCM sau khi thực hiện chuyển tiền liên chi nhánh - xảy ra lỗi 131
Hình 4.99 Mô phỏng kết quả chuyển tiền liên chi nhánh - xảy ra lỗi 132
Hình 4.100 Truy vấn phân tán - Tổng số dư của hệ thống 133

Hình 4.101 Truy vấn phân tán - Top khách hàng giàu nhất 134
Hình 4.102 Truy vấn phân tán - Giao dịch chuyển tiền liên chi nhánh 135
Hình 4.103 Truy vấn phân tán - Giao dịch cùng chi nhánh 135
Hình 4.104 Truy vấn phân tán - Khách hàng có nhiều chi nhánh 136
Hình 4.105 Truy vấn phân tán - Thống kê giao dịch theo từng chi nhánh 137
Hình 4.106 Truy vấn phân tán - Lịch sử gửi tiền 137
Hình 4.107 Truy vấn phân tán - Lịch sử rút tiền 138
Hình 4.108 Truy vấn phân tán - Top KH gửi tiền 139

DANH MỤC BẢNG BIỂU

Bảng 2.1 Bảng Chi Nhánh - Branch 19
Bảng 2.2 Bảng Khách hàng - Customer 20
Bảng 2.3 Bảng Tài khoản - Account 20
Bảng 2.4 Bảng lịch sử giao dịch - transaction_history 22
Bảng 1.5 Bảng chi tiết các chi nhánh tham gia giao dịch phân tán - transaction_participant 23
Bảng 2.6 Bảng theo dõi giao dịch phân tán-distributed_transaction_log: 25
Bảng 2.7 Bảng kiến trúc phân tán 26
Bảng 2.8 Bảng tổng hợp chiến lược thiết kế cơ sở dữ liệu phân tán 29
Bảng 3.1 Minh họa tính nhất quán dữ liệu trước và sau giao dịch liên chi nhánh 34
Bảng 3.2 Kết quả mô phỏng Rollback và phục hồi dữ liệu sau sự cố 36
Bảng 4.1 Bảng yêu cầu phần cứng 38
Bảng 4.2 Bảng yêu cầu phần mềm 39
Bảng 4.3 Bảng công nghệ sử dụng - Backend 40
Bảng 4.4 Bảng công nghệ sử dụng - Frontend 40
Bảng 4.5 Bảng chia site 41
Bảng 4.6 Bảng dữ liệu tại mỗi site 43
Bảng 4.7 Bảng chi tiết cấu hình của docker 49
Bảng 4.8 Bảng ánh xạ port của mỗi site 50
Bảng 4.9 Bảng cấu trúc package 57
Bảng 4.10 Giải thích cấu hình CORS 64
Bảng 4.11 Phân hệ Giao dịch & Tài khoản 66
Bảng 4.12 Phân hệ Quản lý Danh mục - Master Data 68
Bảng 4.13 Phân hệ Truy vấn và Tổng hợp dữ liệu phân tán 68
Bảng 4.14 Cổng mạng sử dụng 84
Bảng 5.1 Hiệu năng giao dịch cục bộ 140
Bảng 5.2 Hiệu năng giao dịch phân tán 142
Bảng 5.3 Hiệu năng truy vấn phân tán 143
Bảng 5.4 So sánh tổng quát 145

PHÂN CÔNG CÔNG VIỆC VÀ MỨC ĐỘ HOÀN THÀNH

Thành viên	**Vai trò & Chỉ số**	**Danh sách các hạng mục thực hiện**
1. Mai Anh Đức	Vai trò: **Nhóm trưởng (Phụ trách Backend, Cơ sở dữ liệu phân tán và Kiến trúc hệ thống)** • Khối lượng: **Khoảng 60%** • Hoàn thành: **100%**	1. Đề xuất ý tưởng và thiết kế kiến trúc tổng thể hệ thống ngân hàng phân tán. 2. Thiết kế mô hình phân mảnh dữ liệu theo chi nhánh ngân hàng. 3. Thiết kế cơ sở dữ liệu và xây dựng kịch bản khởi tạo dữ liệu cho các site 4. Triển khai môi trường phân tán bằng Docker và Docker Compose. 5. Xây dựng Backend bằng Spring Boot. 6. Cấu hình Dynamic Routing DataSource và Transaction Manager cho từng chi nhánh. 7. Xây dựng các chức năng quản lý khách hàng, tài khoản, nạp tiền, rút tiền và chuyển tiền. 8. Triển khai cơ chế giao dịch cục bộ (Local Transaction).

		9. Xây dựng cơ chế phòng tránh Deadlock bằng Ordered Locking. 10. Thiết kế và hiện thực giao thức Two-Phase Commit (2PC) cho giao dịch liên chi nhánh. 11. Xây dựng cơ chế ghi log giao dịch phân tán (distributed_transaction_log, transaction_participant). 12. Triển khai cơ chế Rollback, Compensation và Crash Recovery. 13. Xây dựng các truy vấn phân tán và chức năng tổng hợp dữ liệu từ nhiều site. 14. Thiết kế và phát triển toàn bộ RESTful API. 15. Xây dựng các kịch bản kiểm thử giao dịch đồng thời, deadlock và lỗi hệ thống. 16. Viết báo cáo đồ án, tài liệu kỹ thuật và slide thuyết trình.
2. Trần Đăng Dương	• Vai trò: **Thành viên (Phụ trách Frontend)**	1. Thiết kế giao diện người dùng của hệ thống.

	• Khối lượng: **Khoảng 20%** • Hoàn thành: **100%**	2. Thiết kế mô hình phân mảnh dữ liệu theo chi nhánh ngân hàng. 3. Thiết kế cơ sở dữ liệu và xây dựng kịch bản khởi tạo dữ liệu cho các site. 4. Xây dựng giao diện bằng ReactJS. 5. Thiết kế các màn hình quản lý khách hàng, tài khoản và giao dịch. 6. Xây dựng giao diện chuyển tiền nội bộ và liên chi nhánh. 7. Tích hợp API từ Backend vào giao diện. 8. Thiết kế giao diện hiển thị lịch sử giao dịch và thống kê dữ liệu. 9. Tham gia kiểm thử giao diện và hoàn thiện trải nghiệm người dùng.
3. Trịnh Anh Tú	Vai trò: **Thành viên (Phụ trách Frontend)** • Khối lượng: **Khoảng 20%**	1. Thiết kế mô hình phân mảnh dữ liệu theo chi nhánh ngân hàng. 2. Thiết kế cơ sở dữ liệu và xây dựng kịch bản khởi tạo dữ liệu cho các site.

	• Hoàn thành: **100%**	3. Phối hợp phát triển giao diện người dùng bằng ReactJS. 4. Xây dựng các component và layout giao diện. 5. Thiết kế giao diện mô phỏng giao dịch phân tán, rollback và crash recovery. 6. Xây dựng màn hình hiển thị log giao dịch và trạng thái hệ thống. 7. Hỗ trợ tích hợp API Backend với Frontend. 8. Kiểm thử giao diện trên các chức năng chính. 9. Hỗ trợ hoàn thiện báo cáo và chuẩn bị nội dung trình bày sản phẩm.

Kết quả: Tất cả thành viên đều hoàn thành đầy đủ nhiệm vụ được phân công, trong đó phần Backend, cơ sở dữ liệu phân tán và nghiệp vụ giao dịch do nhóm trưởng phụ trách chính; phần giao diện người dùng được hai thành viên còn lại phối hợp thực hiện.

Source code: https://github.com/anhduc-developer/Distributed-Banking-System

CHƯƠNG 1. PHÂN TÍCH BÀI TOÁN

1.1 Bối cảnh đề tài

Trong bối cảnh nền kinh tế số và sự phát triển mạnh mẽ của ngành tài chính, các ngân hàng quy mô lớn luôn có xu hướng mở rộng mạng lưới hoạt động với hàng loạt chi nhánh và phòng giao dịch trải dài trên nhiều tỉnh thành khác nhau. Đặc thù của mô hình này là mỗi chi nhánh sẽ trực tiếp lưu trữ và quản lý khối lượng lớn dữ liệu nội bộ phát sinh tại địa phương, bao gồm: thông tin khách hàng, tài khoản, và lịch sử giao dịch.

Tuy nhiên, mặc dù dữ liệu có tính chất phân mảnh về mặt địa lý, hệ thống ngân hàng vẫn phải vận hành như một thực thể thống nhất. Toàn hệ thống bắt buộc phải hỗ trợ trơn tru các nghiệp vụ liên chi nhánh phức tạp như chuyển tiền chéo giữa các tỉnh thành, tra cứu thông tin tài khoản từ xa, và tổng hợp báo cáo, thống kê dữ liệu trên quy mô toàn ngân hàng.

Để đáp ứng các yêu cầu khắt khe đó, hệ thống cơ sở dữ liệu phân tán (Distributed Database) được ứng dụng như một giải pháp công nghệ tất yếu nhằm:

- Phân tán dữ liệu: Phân bổ và lưu trữ dữ liệu hợp lý về các site (chi nhánh) khác nhau tương ứng với nơi phát sinh.
- Tăng khả năng mở rộng (Scalability): Dễ dàng bổ sung các chi nhánh mới vào hệ thống mà không làm gián đoạn vận hành.
- Giảm tải cho hệ thống trung tâm: Tránh tình trạng thắt nút cổ chai (bottleneck) bằng cách chia sẻ tài nguyên xử lý về các node địa phương.
- Tăng tính sẵn sàng và hiệu năng xử lý: Hạn chế rủi ro lỗi điểm đơn (Single Point of Failure) và tối ưu hóa thời gian đáp ứng nhờ cơ chế truy xuất dữ liệu gần nhất.

Xuất phát từ thực tiễn đó, đồ án "Xây dựng hệ thống quản lý ngân hàng phân tán" được thực hiện. Đồ án tập trung mô phỏng một mạng lưới ngân hàng gồm nhiều chi nhánh, nơi dữ liệu được lưu trữ độc lập tại từng site nhưng vẫn đảm bảo tính trong suốt, hỗ trợ thực thi an toàn các truy vấn và giao dịch trên toàn hệ thống.

1.2 Mục tiêu của hệ thống

Hệ thống được xây dựng hướng tới hai nhóm mục tiêu chính: mục tiêu nghiệp vụ thực tiễn và mục tiêu học thuật.

1. Mục tiêu nghiệp vụ (Mô phỏng hoạt động ngân hàng):

- Quản lý thông tin khách hàng và tài khoản một cách độc lập theo từng chi nhánh.
- Thực hiện các giao dịch cơ bản: Nạp tiền, rút tiền, và chuyển khoản nội bộ (cùng chi nhánh).
- Hỗ trợ các giao dịch phức tạp: Chuyển tiền liên chi nhánh một cách an toàn và chính xác.
- Cung cấp công cụ tra cứu lịch sử giao dịch và thống kê, tổng hợp dữ liệu trên toàn mạng lưới ngân hàng.

2. Mục tiêu học thuật (Minh họa lý thuyết Hệ phân tán):

- Hiện thực hóa các kỹ thuật phân mảnh dữ liệu (Data Fragmentation) và cấp phát dữ liệu (Data Allocation).
- Xây dựng cơ chế xử lý truy vấn phân tán (Distributed Query) và quản lý giao dịch phân tán (Distributed Transaction).
- Mô phỏng trực quan hoạt động của giao thức commit hai pha (2-Phase Commit - 2PC) nhằm đảm bảo tính nhất quán của dữ liệu.
- Xây dựng cơ chế phục hồi (Crash Recovery/Rollback) tự động khi xảy ra lỗi mạng hoặc lỗi hệ thống tại một site bất kỳ trong quá trình giao dịch.

1.3 Các chức năng chính

Hệ thống được thiết kế theo dạng module hóa, bao gồm các nhóm chức năng trọng tâm sau:

- Module Quản lý chi nhánh (Branch Management):
    - Xem thông tin chi tiết của từng chi nhánh.
    - Thống kê cục bộ: số lượng tài khoản, tổng số dư, và lưu lượng giao dịch tại chi nhánh.

- Module Quản lý khách hàng (Customer Management):
    - Thêm mới, cập nhật thông tin và xóa khách hàng (CRUD).
    - Tra cứu thông tin khách hàng theo từng phân mảnh (chi nhánh).
- Module Quản lý tài khoản (Account Management):
    - Tạo mới tài khoản ngân hàng liên kết với khách hàng.
    - Tra cứu số dư tài khoản hiện tại.
    - Quản lý danh sách tài khoản theo từng site độc lập.
- Module Giao dịch ngân hàng (Banking Transactions):
    - Giao dịch cục bộ: Nạp tiền, Rút tiền, Chuyển khoản giữa các tài khoản thuộc cùng một site.
    - Giao dịch phân tán: Chuyển khoản liên chi nhánh (yêu cầu phối hợp nhiều site).
- Module Truy vấn và Thống kê phân tán (Distributed Queries & Analytics):
    - Tính toán tổng số dư và tổng số lượng tài khoản trên toàn hệ thống.
    - Liệt kê chi tiết các giao dịch liên chi nhánh đã thực hiện.
    - Tìm kiếm khách hàng có thông tin lưu trữ đa chi nhánh.
    - Xác định các tài khoản có số dư lớn nhất trên toàn mạng lưới.

1.4 Công nghệ sử dụng

Để đảm bảo hiệu năng và tính hiện đại, hệ thống được phát triển dựa trên các công nghệ tiêu chuẩn của ngành công nghiệp phần mềm:

- Phần Backend (Máy chủ điều phối & Xử lý nghiệp vụ):
    - Java Spring Boot: Xây dựng kiến trúc hệ thống, phát triển các RESTful APIs và đóng vai trò là Coordinator (Nút điều phối).
    - JDBC Template: Thao tác và giao tiếp trực tiếp với cơ sở dữ liệu, tối ưu hóa các truy vấn SQL thô trong hệ phân tán.
- Phần Frontend (Giao diện người dùng):
    - ReactJS: Xây dựng ứng dụng trang đơn (SPA) giúp trải nghiệm người dùng mượt mà.

    - Ant Design: Sử dụng thư viện UI component để thiết kế giao diện quản trị chuyên nghiệp, nhất quán.
- Hệ quản trị Cơ sở dữ liệu:
    - MySQL: Lưu trữ dữ liệu quan hệ tại từng phân mảnh.
- Môi trường triển khai & Mô phỏng hệ phân tán:
    - Docker: Container hóa các dịch vụ và cơ sở dữ liệu để dễ dàng triển khai.
    - Hệ thống thiết lập 03 site cơ sở dữ liệu độc lập mô phỏng 3 khu vực địa lý: Hà Nội (HN), Đà Nẵng (DN), và TP. Hồ Chí Minh (HCM).

1.5 Kiến trúc hệ thống

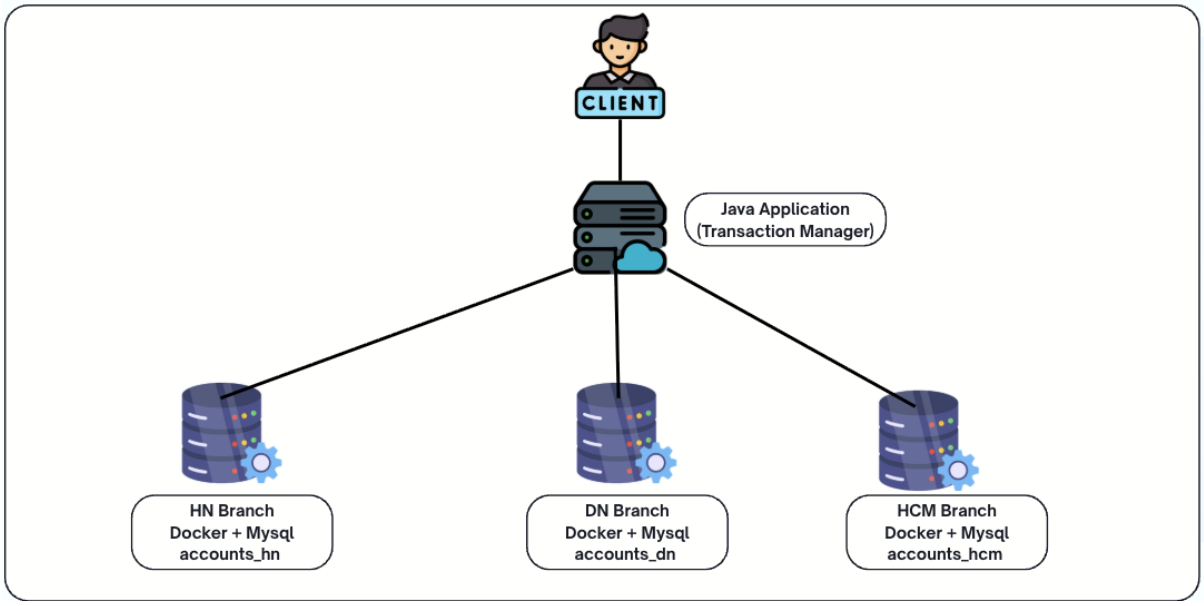
Hệ thống được thiết kế theo mô hình client-server kết hợp cơ sở dữ liệu phân tán, bao gồm các thành phần chính:

1. Client (Frontend): Ứng dụng web ReactJS, đóng vai trò giao tiếp với người dùng và gửi các HTTP Request tới máy chủ trung tâm.
2. Coordinator (Nút điều phối trung tâm): Một ứng dụng Spring Boot đóng vai trò làm trung gian tiếp nhận yêu cầu. Nó chịu trách nhiệm phân tích ngữ nghĩa của request để định tuyến truy vấn (routing) tới đúng site lưu trữ, hoặc đứng ra điều phối (coordinate) các giao dịch đa site.
3. Data Nodes (Các site cơ sở dữ liệu): Gồm 3 máy chủ MySQL độc lập (Site Hà Nội, Site Đà Nẵng, Site TP.HCM). Mỗi site chỉ chứa một phần dữ liệu của toàn hệ thống (dữ liệu được phân mảnh theo chi nhánh).

Cơ chế hoạt động cốt lõi của kiến trúc:

- Truy vấn cục bộ (Local Query): Khi request chỉ chạm đến dữ liệu của một chi nhánh (ví dụ: rút tiền tại Hà Nội), Coordinator sẽ định tuyến lệnh thẳng tới site đó để thực thi độc lập.
- Truy vấn phân tán (Distributed Query): Khi người dùng yêu cầu xem "Tổng số dư toàn hệ thống", Coordinator sẽ gửi truy vấn đồng thời tới cả 3 site, thu thập kết quả trả về, tổng hợp (aggregate) và gửi lại cho Frontend.

- Giao dịch phân tán liên site (Distributed Transaction): Khi xảy ra nghiệp vụ chuyển tiền từ site Hà Nội sang site TP.HCM, Coordinator sẽ áp dụng giao thức commit hai pha. Hệ thống đảm bảo tính chất ACID bằng cách mô phỏng quá trình rollback hoàn toàn (hủy bỏ giao dịch trên tất cả các site liên quan) nếu có bất kỳ site nào báo lỗi hoặc mất kết nối trong quá trình thực thi.



Hình 1.1 Kiến trúc tổng thể hệ thống ngân hàng phân tán

1.6 Kết chương

Trong chương này, đề tài đã trình bày tổng quan về hệ thống quản lý ngân hàng phân tán, từ bối cảnh thực tiễn của mô hình ngân hàng đa chi nhánh đến các yêu cầu đặt ra đối với một hệ cơ sở dữ liệu phân tán hiện đại. Trên cơ sở đó, hệ thống được định hướng xây dựng nhằm đáp ứng cả nhu cầu nghiệp vụ thực tế và mục tiêu học thuật của môn học Hệ cơ sở dữ liệu phân tán.

Chương cũng đã mô tả các chức năng chính của hệ thống như quản lý khách hàng, tài khoản, giao dịch ngân hàng và các truy vấn thống kê phân tán. Đồng thời, kiến trúc tổng thể của hệ thống gồm Client, Coordinator và các Data Node cũng được trình bày nhằm làm rõ cách thức tổ chức và phối hợp giữa các site trong môi trường phân tán.

Bên cạnh đó, các công nghệ sử dụng như Spring Boot, ReactJS, MySQL và Docker đã được lựa chọn để hỗ trợ việc xây dựng và mô phỏng hệ thống một cách trực quan và hiệu quả. Đặc biệt, cơ chế xử lý giao dịch phân tán thông qua giao thức Two-

Phase Commit (2PC) và cơ chế rollback khi xảy ra lỗi được xem là trọng tâm nhằm đảm bảo tính nhất quán dữ liệu trên toàn hệ thống.

Những nội dung được trình bày trong chương này sẽ là nền tảng để thực hiện phân tích thiết kế cơ sở dữ liệu, phân mảnh dữ liệu và triển khai các cơ chế xử lý phân tán trong các chương tiếp theo.

CHƯƠNG 2. THIẾT KẾ CƠ SỞ DỮ LIỆU PHÂN TÁN

2.1 Lược đồ toàn cục

Lược đồ toàn cục biểu diễn cấu trúc logic của toàn bộ dữ liệu trong hệ thống trước khi tiến hành phân mảnh. Trong mô hình quản lý ngân hàng, cơ sở dữ liệu được thiết kế với 4 thực thể (bảng) trọng tâm, có mối quan hệ chặt chẽ với nhau:

- **Bảng 1: Chi nhánh (Branch) Lưu trữ thông tin danh mục về các chi nhánh trực thuộc ngân hàng.**

Thuộc tính	**Kiểu dữ liệu**	**Ý nghĩa**
branch_id	VARCHAR	Mã chi nhánh
branch_name	VARCHAR	Tên chi nhánh
city	VARCHAR	Thành phố
created_at	DATE	Thời gian tạo

Bảng 2.1 Bảng Chi Nhánh - Branch

- **Bảng 2: Khách hàng (Customer) Lưu trữ thông tin định danh và liên lạc của khách hàng. Mỗi khách hàng sẽ được quản lý bởi một chi nhánh cụ thể (nơi mở hồ sơ).**

Thuộc tính	**Kiểu dữ liệu**	**Ý nghĩa**

customer_id	BIGINT	Mã khách hàng
full_name	VARCHAR	Họ tên
phone	VARCHAR	Số điện thoại
address	VARCHAR	Địa chỉ
branch_id	VARCHAR	Chi nhánh quản lý
email	VARCHAR	Email
created_at	DATE	Thời gian tạo

Bảng 2.2 Bảng Khách hàng - Customer

- **Bảng 3: Tài khoản (Account): Quản lý thông tin số dư của các tài khoản ngân hàng. Một khách hàng có thể sở hữu nhiều tài khoản.**

Thuộc tính	**Kiểu dữ liệu**	**Ý nghĩa**
account_id	BIGINT	Mã tài khoản
customer_id	BIGINT	Chủ tài khoản
balance	DOUBLE	Số dư
branch_id	VARCHAR	Chi nhánh quản lý
status	VARCHAR	Trạng thái tài khoản
created_at	DATE	Thời gian tạo

Bảng 2.3 Bảng Tài khoản - Account

- **Bảng 4: Lịch sử giao dịch (Transaction_History): Ghi nhận toàn bộ biến động số dư, bao gồm cả các giao dịch diễn ra trong nội bộ một site và các giao dịch chuyển tiền liên site.**

Thuộc tính	**Kiểu dữ liệu**	**Ý nghĩa**
transaction_id	BIGINT	Mã giao dịch (Khóa chính)
transaction_type	VARCHAR	Loại giao dịch (Ví dụ: DEPOSIT, WITHDRAW, TRANSFER,...)
amount	DECIMAL(15,2)	Số tiền giao dịch
account_id	BIGINT	Mã tài khoản thực hiện giao dịch
related_account_id	BIGINT	Mã tài khoản đối ứng (nếu có)
related_branch_id	VARCHAR	Mã chi nhánh đối ứng
balance_after	DECIMAL(15,2)	Số dư tài khoản sau khi thực hiện giao dịch

status	VARCHAR	Trạng thái giao dịch (Ví dụ: SUCCESS,FAILED, PENDING,...)
distributed_txn_id	VARCHAR	Mã giao dịch phân tán (Distributed Transaction ID)
description	VARCHAR	Nội dung/Lời nhắn của giao dịch
created_at	TIMESTAMP	Thời gian khởi tạo giao dịch

Bảng 2.4 Bảng lịch sử giao dịch - transaction_history

- **Bảng 5: Chi tiết các chi nhánh tham gia giao dịch phân tán(transaction_participant): Mỗi chi nhánh nào tham gia vào distributed transaction và trạng thái hiện tại của từng chi nhánh trong 2PC.**

Thuộc tính	**Kiểu dữ liệu**	**Ý nghĩa**
id	BIGINT	ID của thành phần tham gia (Participant ID - Khóa chính)
txn_id	VARCHAR	Mã giao dịch phân tán (Distributed Transaction ID - Dùng để liên kết với bảng cha)
branch_id	VARCHAR	Mã chi nhánh tham gia vào giao dịch

role	VARCHAR	Vai trò của site trong giao dịch (Ví dụ: SOURCE - Nguồn, DESTINATION - Đích)
status	VARCHAR	Trạng thái của thành phần tham gia (Ví dụ: PREPARED, COMMITTED, ABORTED,...)
action	VARCHAR	Hành động thực hiện tài chính (Ví dụ: DEBIT - Ghi nợ, CREDIT - Ghi có)
created_at	TIMESTAMP	Thời gian thành phần này tham gia vào giao dịch

Bảng 1.5 Bảng chi tiết các chi nhánh tham gia giao dịch phân tán - transaction_participant

- **Bảng 6: Bảng theo dõi giao dịch phân tán(distributed_transaction_log): Lưu thông tin và trạng thái của các giao dịch phân tán giữa nhiều chi nhánh trong hệ thống ngân hàng, phục vụ cơ chế Two Phase Commit (2PC) và rollback khi có lỗi xảy ra.**

Thuộc tính	**Kiểu dữ liệu**	**Ý nghĩa**
txn_id	VARCHAR	Mã định danh duy nhất của giao dịch phân tán (Global Transaction ID - Khóa chính)

txn_type	VARCHAR	Loại giao dịch phân tán (Ví dụ: INTER_BRANCH_TRANSFER, SYSTEM_SYNC,...)
status	VARCHAR	Trạng thái hiện tại của giao dịch (Ví dụ: INIT, PROCESSING, SUCCESS, FAILED)
source_branch	VARCHAR	Mã chi nhánh nguồn nơi phát động/thực hiện giao dịch
dest_branch	VARCHAR	Mã chi nhánh đích nhận kết quả giao dịch
amount	DECIMAL(15,2)	Tổng số tiền thực hiện trong giao dịch
source_account_id	BIGINT	Mã tài khoản nguồn (Tài khoản chuyển/trừ tiền)
dest_account_id	BIGINT	Mã tài khoản đích (Tài khoản nhận/cộng tiền)

error_message	VARCHAR	Nội dung chi tiết lỗi nếu giao dịch rơi vào trạng thái thất bại (FAILED)
created_at	TIMESTAMP	Thời gian khởi tạo giao dịch phân tán
updated_at	TIMESTAMP	Thời gian cập nhật trạng thái gần nhất của giao dịch

Bảng 2.6 Bảng theo dõi giao dịch phân tán-distributed_transaction_log:

2.2 Kiến trúc phân tán

Hệ thống được triển khai theo cấu trúc cơ sở dữ liệu phân tán nhiều node (Multi-site Architecture). Cấu trúc này bám sát mô hình triển khai thực tế của các ngân hàng thương mại, trong đó mỗi chi nhánh sẽ sở hữu một hệ thống lưu trữ dữ liệu nội bộ riêng để đảm bảo tốc độ và giảm sự phụ thuộc vào đường truyền mạng diện rộng.

- Hệ thống vật lý được mô phỏng bao gồm 3 site độc lập:

Site (Chi nhánh)	**Tên Database Instance**	**Vai trò**
Site Hà Nội (HN)	mysql_hanoi	Quản lý dữ liệu phát sinh tại khu vực phía Bắc.
Site Đà Nẵng (DN)	mysql_danang	Quản lý dữ liệu phát sinh tại khu vực miền Trung.

Site TP. HCM (HCM)	mysql_hcm	Quản lý dữ liệu phát sinh tại khu vực phía Nam.

Bảng 2.7 Bảng kiến trúc phân tán

2.3 Phân mảnh dữ liệu

Để tối ưu hóa hiệu năng, hệ thống áp dụng kỹ thuật Phân mảnh ngang nguyên thủy (Primary Horizontal Fragmentation). Cơ sở của việc phân mảnh dựa trên đặc tính cục bộ của dữ liệu: phần lớn các truy vấn tra cứu thông tin và giao dịch cơ bản của khách hàng đều diễn ra tại chính chi nhánh mà họ đã mở tài khoản.

Tiêu chuẩn phân mảnh (Selection Predicate) được sử dụng thống nhất trên toàn hệ thống là thuộc tính branch_id. Theo đó, các bảng Customer, Account, và Transaction_History sẽ được chia cắt theo từng khu vực:

1. Phân mảnh dữ liệu Khách hàng (Customer):

SQL:

1. Fragment Customer_HN (Dữ liệu khách hàng tại Hà Nội)

SQL

```
-- Fragment Customer_HN (Dữ liệu khách hàng tại Hà Nội)
SELECT * FROM Customer WHERE branch_id = 'HN';
```

2. Fragment Customer_DN (Dữ liệu khách hàng tại Đà Nẵng)

SQL

```
-- Fragment Customer_DN (Dữ liệu khách hàng tại Đà Nẵng)
SELECT * FROM Customer WHERE branch_id = 'DN';
```

3. Fragment Customer_HCM (Dữ liệu khách hàng tại TP.HCM)

SQL

```
-- Fragment Customer_HCM (Dữ liệu khách hàng tại TP.HCM)
SELECT * FROM Customer WHERE branch_id = 'HCM';
```

2. Phân mảnh dữ liệu Tài khoản và Giao dịch

Nguyên tắc tương tự được áp dụng đồng bộ. Dữ liệu tài khoản (Account) và các bản ghi lịch sử giao dịch (Transaction_History) phát sinh tại chi nhánh nào sẽ được gom thành một phân mảnh (Fragment) tương ứng với mã branch_id của chi nhánh đó.

Ghi chú thiết kế: Riêng bảng Branch (Danh mục chi nhánh) có kích thước rất nhỏ và dữ liệu tĩnh (ít thay đổi), hệ thống có thể áp dụng chiến lược nhân bản toàn phần (Fully Replicated) – lưu trữ một bản sao đầy đủ tại tất cả các site để phục vụ việc tra cứu nhanh mã chi nhánh mà không cần truy vấn xuyên mạng.

2.4. Cấp phát dữ liệu (Data Allocation)

Sau khi định nghĩa các phân mảnh logic, hệ thống tiến hành ánh xạ và cấp phát (allocate) các phân mảnh này xuống các node vật lý. Mô hình phân bổ sử dụng chiến lược cấp phát không dư thừa (Non-redundant Allocation) đối với dữ liệu nghiệp vụ, nghĩa là dữ liệu của chi nhánh nào sẽ được lưu trữ độc quyền tại máy chủ của chi nhánh đó:

- Tại Site HN (mysql_hanoi): Lưu trữ các phân mảnh Customer_HN, Account_HN và Transaction_History_HN.
- Tại Site DN (mysql_danang): Lưu trữ các phân mảnh Customer_DN, Account_DN và Transaction_History_DN.
- Tại Site HCM (mysql_hcm): Lưu trữ các phân mảnh Customer_HCM, Account_HCM và Transaction_History_HCM.

Đánh giá mô hình: Kiến trúc cấp phát này tái hiện tính chân thực của mạng lưới ngân hàng lõi (Core Banking). Việc phân tách dữ liệu triệt để giúp từng site có khả năng tự trị cao (Local Autonomy). Đồng thời, thông qua máy chủ điều phối trung tâm (Coordinator), hệ thống vẫn duy trì sức mạnh tổng thể để thực thi các nghiệp vụ đa site phức tạp như chuyển tiền liên ngân hàng/liên chi nhánh hoặc truy vấn báo cáo thống kê toàn hệ thống.

2.5 Chiến lược nhân bản dữ liệu (Data Replication)

Bên cạnh kỹ thuật phân mảnh ngang đối với các dữ liệu giao dịch, hệ thống áp dụng chiến lược nhân bản toàn phần (Fully Replicated) đối với bảng danh mục Branch. Cụ thể, một bản sao hoàn chỉnh của bảng này sẽ được lưu trữ tại tất cả các site (HN, DN, HCM). Việc nhân bản bảng Branch mang lại nhiều lợi ích thiết thực:

- Tối ưu hóa hiệu năng truy vấn: Quá trình tra cứu thông tin chi nhánh (thường đi kèm với các thao tác join bảng khi truy vấn khách hàng, tài khoản) được thực hiện hoàn toàn tại local site, triệt tiêu độ trễ mạng do không cần truy cập chéo sang các site khác.
- Tăng tính sẵn sàng (High Availability): Nếu một site bị mất kết nối, các site còn lại vẫn có đầy đủ danh mục hệ thống để duy trì hoạt động độc lập.
- Phù hợp với đặc thù dữ liệu: Bảng danh mục chi nhánh có dung lượng rất nhỏ và chứa dữ liệu tĩnh (tần suất cập nhật, thêm mới cực kỳ thấp). Do đó, chi phí đồng bộ hóa (synchronization cost) khi có sự thay đổi là không đáng kể so với hiệu năng đạt được.

2.6 Cơ sở lựa chọn phân mảnh ngang

Kỹ thuật phân mảnh ngang (Horizontal Fragmentation) được lựa chọn làm chiến lược chủ đạo cho các bảng dữ liệu nghiệp vụ (Customer, Account, Transaction_History) dựa trên những cơ sở cốt lõi sau:

- Tính cục bộ của dữ liệu (Data Locality): Trong môi trường ngân hàng thực tế, phần lớn các nghiệp vụ thường nhật (gửi tiền, rút tiền, tra cứu số dư, cập nhật thông tin) của một khách hàng thường diễn ra tại chính chi nhánh quản lý tài khoản đó. Việc đưa dữ liệu về đúng nơi phát sinh giúp hệ thống giải quyết hầu hết các request ngay tại cơ sở dữ liệu địa phương.
- Giảm thiểu lưu lượng mạng (Reduce Network Traffic): Bằng cách xử lý cục bộ các giao dịch cơ bản, hệ thống hạn chế tối đa các luồng truyền tải dữ liệu qua lại trên mạng diện rộng (WAN), từ đó ngăn ngừa tình trạng nghẽn cổ chai mạng.
- Tối ưu hóa hiệu suất cơ sở dữ liệu: Khối lượng dữ liệu khổng lồ của toàn ngân hàng được chia nhỏ và phân tán. Các máy chủ tại từng site chỉ phải quét và xử

lý một tập dữ liệu nhỏ hơn rất nhiều, giúp rút ngắn đáng kể thời gian thực thi các câu lệnh SQL.

- Khả năng mở rộng linh hoạt (Scalability): Khi ngân hàng mở thêm chi nhánh mới, hệ thống chỉ cần thiết lập một node cơ sở dữ liệu mới và định nghĩa một phân mảnh ngang tương ứng, hoàn toàn không làm xáo trộn hay ảnh hưởng đến hiệu năng của các site hiện tại.

2.7 Ma trận phân phối dữ liệu

Bảng dưới đây tổng hợp chiến lược thiết kế cơ sở dữ liệu phân tán của toàn bộ hệ thống, thể hiện rõ phương pháp phân mảnh và vị trí cấp phát đối với từng thực thể:

Bảng (Thực thể)	**Kiểu phân tán**	**Vị trí cấp phát (Sites)**
Customer	Phân mảnh ngang (Horizontal)	HN, DN, HCM
Account	Phân mảnh ngang (Horizontal)	HN, DN, HCM
Transaction_History	Phân mảnh ngang (Horizontal)	HN, DN, HCM
Branch	Nhân bản toàn phần (Replication)	HN, DN, HCM

Bảng 2.8 Bảng tổng hợp chiến lược thiết kế cơ sở dữ liệu phân tán

2.8 Đánh giá ưu điểm của mô hình kiến trúc

Mô hình kiến trúc phân tán được thiết kế mang lại những giá trị vượt trội, giải quyết triệt để các hạn chế của kiến trúc cơ sở dữ liệu tập trung (Centralized Database) truyền thống:

- Tối ưu hiệu năng tổng thể: Cơ chế xử lý phân tán giúp dàn đều tải trọng lên nhiều máy chủ vật lý khác nhau. Các truy vấn cục bộ được thực thi ngay tại chỗ với tốc độ cao, trong khi các truy vấn phân tán (như thống kê toàn ngân hàng) tận dụng được sức mạnh xử lý song song của nhiều site cùng lúc.
- Đảm bảo tính sẵn sàng và khả năng chịu lỗi (Fault Tolerance): Hệ thống không tồn tại điểm lỗi duy nhất (Single Point of Failure). Sự cố sập nguồn hoặc đứt mạng tại một chi nhánh chỉ làm gián đoạn cục bộ dữ liệu tại khu vực đó, các chi nhánh khác vẫn duy trì khả năng phục vụ khách hàng bình thường.
- Đáp ứng trọn vẹn nghiệp vụ liên vùng: Bất chấp việc dữ liệu bị chia cắt về mặt vật lý, thông qua máy chủ điều phối, hệ thống vẫn duy trì tính trong suốt (Data Transparency) đối với người dùng. Các nghiệp vụ phức tạp như chuyển tiền liên site được xử lý an toàn, nhất quán.
- Tính thực tiễn cao: Sơ đồ phân mảnh và cấp phát dữ liệu phản ánh chính xác cấu trúc tổ chức và phương thức vận hành của các hệ thống Core Banking thực tế tại các ngân hàng thương mại hiện nay.

2.9 Kết chương

Trong chương này, hệ thống cơ sở dữ liệu phân tán cho bài toán quản lý ngân hàng đã được thiết kế ở mức logic và vật lý. Trước hết, lược đồ toàn cục của hệ thống được xây dựng với bốn thực thể chính gồm Branch, Customer, Account và Transaction_History, phản ánh đầy đủ các nghiệp vụ cốt lõi của ngân hàng.

Dựa trên đặc thù hoạt động của mô hình ngân hàng đa chi nhánh, đề tài đã lựa chọn kiến trúc phân tán nhiều site với ba node dữ liệu độc lập tại Hà Nội, Đà Nẵng và TP.HCM. Trên cơ sở đó, kỹ thuật phân mảnh ngang nguyên thủy được áp dụng cho các bảng nghiệp vụ nhằm tối ưu tính cục bộ dữ liệu, giảm lưu lượng truyền tải qua mạng và nâng cao hiệu năng xử lý tại từng site. Đồng thời, chiến lược cấp phát không dư thừa giúp mỗi chi nhánh có khả năng tự trị cao trong việc quản lý dữ liệu của mình.

Bên cạnh đó, hệ thống cũng áp dụng cơ chế nhân bản toàn phần đối với bảng Branch để tăng tốc độ truy vấn và đảm bảo tính sẵn sàng khi xảy ra sự cố mạng hoặc lỗi tại một site bất kỳ. Các nội dung về cơ sở lựa chọn phân mảnh, ma trận phân phối

dữ liệu và đánh giá ưu điểm của mô hình kiến trúc đã cho thấy giải pháp thiết kế phù hợp cả về mặt lý thuyết lẫn thực tiễn triển khai.

Những thiết kế được trình bày trong chương này sẽ là nền tảng quan trọng để triển khai các cơ chế xử lý truy vấn phân tán, giao dịch phân tán, kiểm soát đồng thời và phục hồi dữ liệu trong các chương tiếp theo của đồ án.

CHƯƠNG 3. THIẾT KẾ GIAO DỊCH PHÂN TÁN

3.1 Tổng quan về giao dịch phân tán

Trong hệ thống ngân hàng phân tán, bên cạnh các nghiệp vụ xử lý dữ liệu tại chỗ, tồn tại những nghiệp vụ phức tạp đòi hỏi phải thao tác đồng thời trên tập dữ liệu thuộc quyền quản lý của nhiều site vật lý khác nhau. Những nghiệp vụ này được gọi là giao dịch phân tán (Distributed Transactions).

Nghiệp vụ tiêu biểu và quan trọng nhất đối với một hệ thống Core Banking phân tán là chuyển tiền liên chi nhánh. Trong phạm vi đồ án này, hệ thống tập trung mô phỏng và giải quyết bài toán giao dịch chuyển khoản giữa các tài khoản thuộc 3 site độc lập: Hà Nội (HN), Đà Nẵng (DN) và TP. Hồ Chí Minh (HCM).

3.2 Đặc tả giao dịch chuyển tiền liên chi nhánh

Để làm rõ bản chất của giao dịch phân tán, đồ án đặt ra một kịch bản giao dịch thực tế: Khách hàng yêu cầu chuyển 1.000.000 VNĐ từ tài khoản thuộc chi nhánh Hà Nội sang tài khoản thuộc chi nhánh TP.HCM.

Quá trình này bắt buộc hệ thống phải thực hiện hai luồng cập nhật dữ liệu xuyên biên giới mạng:

1. Cập nhật (trừ tiền) tại cơ sở dữ liệu mysql_hanoi.
2. Cập nhật (cộng tiền) tại cơ sở dữ liệu mysql_hcm.

=> Do dữ liệu phân tán ở hai node khác nhau, một truy vấn SQL cục bộ thông thường không thể giải quyết trọn vẹn nghiệp vụ. Giao dịch này phải được điều phối chặt chẽ để tránh rủi ro sai lệch tài chính.

3.3 Kiến trúc điều phối giao dịch

Hệ thống giải quyết bài toán trên bằng mô hình kiến trúc có một nút điều phối trung tâm:

- Transaction Coordinator (Nút điều phối): Ứng dụng Backend Spring Boot đảm nhận vai trò tiếp nhận request, phân tích ngữ cảnh, điều phối các pha của giao dịch và ra quyết định commit hoặc rollback.
- Participant Sites (Nút tham gia): Các cơ sở dữ liệu MySQL tại các site (HN, DN, HCM) đóng vai trò thực thi các thao tác dữ liệu cục bộ theo lệnh của Coordinator.

3.4 Quy trình thực hiện và Mô phỏng giao thức Two-Phase Commit (2PC)

Thay vì sử dụng các giao dịch phân tán XA nguyên thủy (có thể gây lock tài nguyên kéo dài), hệ thống mô phỏng cơ chế đồng thuận Hai pha (2PC) ở cấp độ ứng dụng để đảm bảo tính nhất quán.

Bước 1: Client gửi yêu cầu giao dịch

Ứng dụng Frontend (ReactJS) khởi tạo một RESTful Request dưới định dạng JSON gửi tới máy chủ Spring Boot:

JSON:

```
{
  "fromBranch": "HN",
  "fromAccountId": 1,
  "toBranch": "HCM",
  "toAccountId": 2,
  "amount": 1000000,
  "simulateCrash": false
}
```

Bước 2: Pha Chuẩn bị (Prepare Phase)

Coordinator không tiến hành trừ tiền ngay mà gửi yêu cầu kiểm tra tính hợp lệ tới các site liên quan:

- Tại Site nguồn (HN): Kiểm tra tài khoản gửi có tồn tại không và trạng thái số dư có lớn hơn hoặc bằng 1.000.000 VNĐ không.
- Tại Site đích (HCM): Kiểm tra định danh tài khoản nhận có thực sự tồn tại và đang hoạt động hay không.
  Nếu *tất cả* các site đều xác nhận đủ điều kiện (READY), hệ thống mới bước sang

pha tiếp theo. Nếu bất kỳ site nào báo lỗi (không đủ tiền, sai tài khoản), Coordinator lập tức hủy giao dịch.

Bước 3: Pha Quyết định (Commit Phase)

Sau khi Phase 1 thành công, Coordinator phát lệnh thực thi đồng thời:

- Site nguồn: Thực thi truy vấn trừ tiền khỏi tài khoản gửi.
- Site đích: Thực thi truy vấn cộng tiền vào tài khoản nhận.
- Ghi nhận lịch sử (Logging): Bản ghi chi tiết về giao dịch (Transaction_History) được insert vào cơ sở dữ liệu tại cả hai site để phục vụ quá trình đối soát và sao kê sau này.

3.5 Đảm bảo tính toàn vẹn dữ liệu (Tính chất ACID)

Hệ thống được thiết kế để duy trì 4 đặc tính kinh điển của một giao dịch, áp dụng vào bối cảnh phân tán:

- Atomicity (Tính nguyên thủy): Giao dịch là một khối thống nhất vô hướng. Hoặc tiền được chuyển thành công ở cả 2 site, hoặc toàn bộ giao dịch bị hủy, hoàn toàn không có trạng thái trung gian (chỉ trừ tiền mà không cộng, hoặc ngược lại).
- Consistency (Tính nhất quán): Tổng tài sản của toàn hệ thống trước và sau giao dịch luôn được bảo toàn.

Trạng thái	**Số dư Site HN**	**Số dư Site HCM**	**Tổng toàn hệ thống**
Trước giao dịch	5.000.000	2.000.000	7.000.000
Sau giao dịch	4.000.000	3.000.000	7.000.000

Bảng 3.1 Minh họa tính nhất quán dữ liệu trước và sau giao dịch liên chi nhánh

- Isolation (Tính độc lập): Các giao dịch diễn ra đồng thời không nhìn thấy các dữ liệu trung gian của nhau cho đến khi quá trình commit hoàn tất.

- Durability (Tính bền vững): Một khi Coordinator thông báo giao dịch thành công, dữ liệu số dư mới và lịch sử giao dịch sẽ được ghi cứng xuống ổ đĩa vật lý của các site MySQL, không bị mất ngay cả khi hệ thống khởi động lại.

3.6 Cơ chế mô phỏng lỗi, Rollback và Phục hồi (Crash Recovery)

Để kiểm chứng tính bền bỉ của kiến trúc, đồ án chủ động tích hợp cơ chế mô phỏng các sự cố nghiêm trọng như sập máy chủ (Site Down) hoặc đứt cáp mạng (Network Failure) ngay giữa quá trình Commit.

Kịch bản lỗi:

Sau khi Coordinator ra lệnh cho Site HN trừ tiền thành công, nhưng trước khi lệnh cộng tiền kịp bay tới Site HCM, hệ thống kích hoạt một ngoại lệ (Exception):

Java throw new RuntimeException("HCM SITE DOWN: Connection Refused");

Cơ chế Rollback (Compensation):

Lúc này, tiền đã bị trừ ở HN nhưng chưa tới HCM. Nhận diện được ngoại lệ, khối xử lý giao dịch của Coordinator lập tức kích hoạt cơ chế bồi thường (Compensation Rollback). Coordinator tính toán và gửi một truy vấn bù trừ về lại Site HN để hoàn trả số tiền:

SQL:

```
UPDATE account
SET balance = balance + 1000000
WHERE account_id = 1;
```

Kết quả sau xử lý sự cố:

Trạng thái mô phỏng	Số dư Site HN	Số dư Site HCM

1. Khởi điểm	5.000.000	2.000.000
2. Khi lỗi xảy ra (Treo)	4.000.000 *(đã trừ)*	2.000.000 *(chưa cộng)*
3. Sau khi Rollback (Phục hồi)	5.000.000	2.000.000

Bảng 3.2 Kết quả mô phỏng Rollback và phục hồi dữ liệu sau sự cố

Như vậy, bất chấp sự cố hệ thống, dữ liệu tài chính không bị thất thoát và tính nhất quán được giữ vững.

3.7 Đánh giá giải pháp thiết kế

Ưu điểm:

- Mô phỏng thành công và trực quan hóa được luồng thực thi phức tạp của một giao dịch phân tán.
- Tách bạch rõ ràng quá trình kiểm tra (Prepare) và thực thi (Commit), tuân thủ tư tưởng của giao thức 2PC.
- Xử lý an toàn các rủi ro hệ thống thông qua cơ chế Rollback chủ động, đảm bảo tuyệt đối tính toàn vẹn dữ liệu tài chính.

Hạn chế:

- Hệ thống hiện tại xử lý Rollback theo hướng bồi thường (Compensation) bằng thủ công ở tầng ứng dụng, chưa ứng dụng chuẩn giao dịch XA (eXtended Architecture) chuyên sâu của hệ quản trị CSDL ở tầng Driver.
- Chưa phát triển hệ thống Transaction Recovery Log lưu trên đĩa cứng của Coordinator để tự động phục hồi trong trường hợp chính bản thân Coordinator bị crash giữa chừng.
- *Mặc dù còn những điểm hạn chế về mặt công nghệ lõi, giải pháp thiết kế này hoàn toàn đáp ứng xuất sắc các tiêu chí đánh giá mô phỏng và minh họa thuật toán của môn học Cơ sở dữ liệu phân tán.*

3.8 Kết chương

Trong chương này, đề tài đã tập trung phân tích và thiết kế cơ chế giao dịch phân tán cho hệ thống ngân hàng đa chi nhánh. Thông qua bài toán chuyển tiền liên chi nhánh giữa các site Hà Nội, Đà Nẵng và TP.HCM, chương đã làm rõ bản chất của giao dịch phân tán và những thách thức trong việc đảm bảo tính nhất quán dữ liệu khi thao tác đồng thời trên nhiều cơ sở dữ liệu vật lý khác nhau.

Hệ thống được xây dựng theo mô hình Coordinator – Participant, trong đó Spring Boot đóng vai trò nút điều phối trung tâm và các site MySQL thực hiện xử lý dữ liệu cục bộ. Trên nền tảng đó, đề tài đã mô phỏng thành công giao thức Two-Phase Commit (2PC) thông qua hai pha Prepare và Commit nhằm đảm bảo các giao dịch liên site được thực hiện một cách đồng bộ và an toàn.

Bên cạnh đó, chương cũng trình bày cơ chế đảm bảo các tính chất ACID trong môi trường phân tán, đặc biệt là tính nguyên thủy (Atomicity) và tính nhất quán (Consistency) đối với các giao dịch tài chính. Đồng thời, hệ thống còn tích hợp cơ chế mô phỏng lỗi và rollback nhằm kiểm chứng khả năng phục hồi khi xảy ra sự cố mất kết nối hoặc lỗi tại một site trong quá trình giao dịch.

Mặc dù giải pháp hiện tại mới dừng ở mức mô phỏng cơ chế rollback ở tầng ứng dụng và chưa triển khai Transaction Recovery Log chuyên sâu, kiến trúc thiết kế đã đáp ứng tốt yêu cầu minh họa các khái niệm cốt lõi của hệ cơ sở dữ liệu phân tán. Những nội dung trong chương này sẽ là nền tảng quan trọng để triển khai phần cài đặt, kiểm thử và đánh giá hệ thống trong các chương tiếp theo.

CHƯƠNG 4. CÀI ĐẶT VÀ THỬ NGHIỆM HỆ THỐNG

4.1 Môi trường phát triển

4.1.1 Yêu cầu phần cứng

Thành Phần	**Yêu cầu tối thiểu**
CPU	Intel Core i5 hoặc tương đương
RAM	8 GB trở lên
Ổ cứng	20 GB dung lượng trống
Mạng	Kết nối Internet để tải dependencies

Bảng 4.1 Bảng yêu cầu phần cứng

4.1.2 Yêu cầu phần mềm

Phần mềm	**Phiên bản**	**Mục đích**
Java Development Kit (JDK)	17	Phát triển Backend
Node.js	18+	Chạy Frontend
npm	9+	Quản lý thư viện Frontend

Docker	20+	Container hóa hệ thống
Docker Compose	2.0+	Quản lý multi-container
MySQL	8.0	Hệ quản trị cơ sở dữ liệu
Git	2.0+	Quản lý mã nguồn

Bảng 4.2 Bảng yêu cầu phần mềm

4.1.3 Công nghệ sử dụng

- **Backend**

Công nghệ	**Vai trò**
Spring Boot	Framework Backend
Spring JDBC	Truy cập và thao tác dữ liệu
Spring Transaction	Quản lý transaction
Lombok	Giảm mã boilerplate
MySQL Connector/J	Driver kết nối MySQL

Gradle	Build project

Bảng 4.3 Bảng công nghệ sử dụng - Backend

- **Frontend**

Công nghệ	**Vai trò**
ReactJS	Xây dựng giao diện SPA
Vite	Build tool
Ant Design	UI Component Library
Axios	Gọi REST API
React Router DOM	Điều hướng ứng dụng

Bảng 4.4 Bảng công nghệ sử dụng - Frontend

4.2 Cài đặt cơ sở dữ liệu phân tán

4.2.1 Kiến trúc cơ sở dữ liệu

Hệ thống ngân hàng phân tán được triển khai theo mô hình Phân mảnh ngang (*Horizontal Fragmentation*), trong đó dữ liệu được chia thành nhiều phân mảnh độc lập và lưu trữ tại các site khác nhau tương ứng với từng chi nhánh ngân hàng.

Mỗi site hoạt động như một cơ sở dữ liệu độc lập, chịu trách nhiệm quản lý dữ liệu cục bộ của chi nhánh mình. Việc phân tách dữ liệu theo khu vực giúp:

- Giảm tải truy vấn giữa các site
- Tăng tốc độ xử lý giao dịch nội bộ
- Tăng tính mở rộng của hệ thống
- Hạn chế xung đột dữ liệu trong môi trường phân tán

Hệ thống hiện bao gồm ba site cơ sở dữ liệu:

Site	**Chi nhánh**	**Database**	**Port**	**Container**
Site 1	Hà Nội (HN)	bank_hanoi	3307	mysql-hanoi
Site 2	Đà Nẵng (DN)	bank_danang	3308	mysql-danang
Site 3	TP.HCM (HCM)	bank_hcm	3309	mysql-hcm

Bảng 4.5 Bảng chia site

Mỗi site được triển khai dưới dạng một MySQL container độc lập thông qua Docker Compose. Các site có thể hoạt động riêng biệt nhưng vẫn phối hợp với nhau thông qua cơ chế giao dịch phân tán.

Cấu Trúc Dữ Liệu Tại Mỗi Site (Database Schema)

Để đáp ứng đồng thời các nghiệp vụ ngân hàng cốt lõi (Core Banking) và cơ chế điều phối giao dịch phân tán theo giao thức Two-Phase Commit (2PC), cơ sở dữ liệu tại mỗi site được thiết kế đồng nhất và bao gồm 6 bảng chức năng chính sau đây:

- Bảng Tổng Hợp Danh Mục Các Bảng Dữ Liệu

Tên bảng	**Phân nhóm chức năng**	**Ý nghĩa và vai trò trong hệ thống**
branch	Core Banking	Lưu trữ thông tin định danh và cấu hình của chi nhánh ngân hàng.
customer	Core Banking	Lưu trữ hồ sơ thông tin cá nhân/doanh nghiệp của khách hàng.
account	Core Banking	Quản lý thông tin tài khoản ngân hàng, loại tiền và số dư hiện tại.
transaction_history	Core Banking	Ghi vết chi tiết lịch sử giao dịch và biến động số dư của tài khoản.

distributed_transaction_log	2PC Coordinator	Lưu trữ trạng thái tổng quan (Global State) của các giao dịch phân tán.
transaction_participant	2PC Participant	Lưu trữ thông tin, vai trò và trạng thái cục bộ của các site tham gia.

Bảng 4.6 Bảng dữ liệu tại mỗi site

- Chi Tiết Phân Nhóm Chức Năng Của Các Bảng

A. Nhóm Nghiệp Vụ Ngân Hàng Cơ Bản (Core Banking Tables)

Nhóm này chịu trách nhiệm vận hành các tính năng tại chỗ (Local) của từng chi nhánh, đảm bảo tính tự trị dữ liệu cao:

- branch & customer: Giúp xác định khách hàng đó thuộc quyền quản lý vật lý của chi nhánh nào để điều hướng truy vấn chính xác đến site tương ứng.
- account & transaction_history: Xử lý các tác vụ kiểm tra số dư, gửi tiền (DEPOSIT), rút tiền (WITHDRAW) hoặc chuyển khoản nội bộ chi nhánh một cách nhanh chóng mà không cần gọi ra ngoài site.

B. Nhóm Điều Phối Giao Dịch Phân Tán (2PC Protocol Tables)

Nhóm này chỉ kích hoạt khi phát sinh các giao dịch liên chi nhánh (ví dụ: Chuyển tiền từ Site Hà Nội vào Site TP.HCM) nhằm đảm bảo tính toàn vẹn dữ liệu (ACID):

- distributed_transaction_log: Được ghi tại site khởi tạo giao dịch (Coordinator) để theo dõi vòng đời của transaction từ khi bắt đầu (INIT), đang xử lý (PROCESSING) cho đến khi đóng gói thành công (SUCCESS) hoặc hủy bỏ (FAILED).

- transaction_participant: Ghi lại phiếu bầu (Vote Prepare) và trạng thái thực thi (PREPARED, COMMITTED, ABORTED) của từng site thành phần, đóng vai trò là cơ sở dữ liệu để hệ thống có thể tự động phục hồi (Recovery) hoặc thực hiện giao dịch bù khi có sự cố mạng xảy ra giữa các chi nhánh.

4.2.2 Cấu hình Docker Compose

```yaml
▷Run All Services
services:
  ▷Run Service
  mysql-hanoi:
    image: mysql:8
    container_name: mysql-hanoi
    restart: always

    command: --character-set-server=utf8mb4 --collation-server=utf8mb4_unicode_ci

    environment:
      MYSQL_ROOT_PASSWORD: root
      MYSQL_DATABASE: bank_hanoi
      MYSQL_ROOT_HOST: "%"

    ports:
      - "3307:3306"
    volumes:
      - ./docker/init-hanoi.sql:/docker-entrypoint-initdb.d/init.sql
      - mysql-hanoi-data:/var/lib/mysql

    healthcheck:
      test: [ "CMD", "mysqladmin", "ping", "-h", "localhost", "-uroot", "-proot" ]
      interval: 10s
      timeout: 5s
      retries: 5
      start_period: 30s
  ▷Run Service
  mysql-danang:
    image: mysql:8
    container_name: mysql-danang
    restart: always

    command: --character-set-server=utf8mb4 --collation-server=utf8mb4_unicode_ci

    environment:
      MYSQL_ROOT_PASSWORD: root
      MYSQL_DATABASE: bank_danang
      MYSQL_ROOT_HOST: "%"

    ports:
      - "3308:3306"

    volumes:
      - ./docker/init-danang.sql:/docker-entrypoint-initdb.d/init.sql
      - mysql-danang-data:/var/lib/mysql

    healthcheck:
      test: [ "CMD", "mysqladmin", "ping", "-h", "localhost", "-uroot", "-proot" ]
      interval: 10s
      timeout: 5s
      retries: 5
      start_period: 30s
  ▷Run Service
  mysql-hcm:
    image: mysql:8
    container_name: mysql-hcm
    restart: always

    command: --character-set-server=utf8mb4 --collation-server=utf8mb4_unicode_ci

    environment:
      MYSQL_ROOT_PASSWORD: root
      MYSQL_DATABASE: bank_hcm
      MYSQL_ROOT_HOST: "%"

    ports:
      - "3309:3306"

    volumes:
      - ./docker/init-hcm.sql:/docker-entrypoint-initdb.d/init.sql
      - mysql-hcm-data:/var/lib/mysql

    healthcheck:
      test: [ "CMD", "mysqladmin", "ping", "-h", "localhost", "-uroot", "-proot" ]
      interval: 10s
      timeout: 5s
      retries: 5
      start_period: 30s
volumes:
  mysql-hanoi-data:
  mysql-danang-data:
  mysql-hcm-data:
```

Hình 4.1 Cấu hình docker cho hệ thống

Toàn bộ các database của hệ thống phân tán được triển khai và quản lý tập trung thông qua Docker Compose. Dưới đây là cấu hình chi tiết cho cả 3 site (Hà Nội, Đà Nẵng, TP.HCM):

Giải thích các tham số cấu hình:

- Bảng Giải Thích Tham Số Cấu Hình Tổng Quan

Tham số cấu hình	**Ý nghĩa & Vai trò kỹ thuật**
services	Khai báo gốc, định nghĩa danh sách tất cả các container/dịch vụ sẽ được khởi chạy trong hệ thống.
mysql-hanoi / mysql-danang / mysql-hcm	Tên của từng service trong file cấu hình, đại diện cho điều phối logic của từng cụm CSDL.
image: mysql:8	Chỉ định Docker Image chính thức của MySQL phiên bản 8.0 làm nền tảng cho container.
container_name	Tên định danh vật lý của container khi hiển thị trong môi trường Docker (ví dụ: mysql-hanoi).

restart: always	Chính sách tự khởi động: Docker sẽ tự động bật lại container nếu nó bị crash, lỗi, hoặc khi dịch vụ Docker Engine được khởi động lại.
command	Ghi đè lệnh khởi động mặc định của MySQL để truyền thêm các tham số thiết lập hệ thống (Charset, Collation).
environment	Phân vùng khai báo các biến môi trường cấu hình cho tiến trình chạy bên trong container.
MYSQL_ROOT_PASSWORD	Biến môi trường thiết lập mật khẩu tối cao cho tài khoản quản trị root của MySQL.
MYSQL_DATABASE	Tên database cục bộ được hệ thống tự động khởi tạo ngay trong lần đầu tiên container chạy.
MYSQL_ROOT_HOST	Định hình bảo mật: Cho phép tài khoản root kết nối từ xa từ bất kỳ host nào (thường đặt là %).

ports	Cơ chế ánh xạ (Mapping) cổng kết nối giữa máy vật lý (Host) và môi trường ảo bên trong container.
volumes	Cơ chế gắn kết dữ liệu (Mounting) giữa thư mục máy Host hoặc ổ đĩa Docker với bộ nhớ trong container.
./docker/init-hanoi.sql:/.../init.sql	Cơ chế tự động chạy script SQL để khởi tạo cấu trúc 6 bảng ngay khi database được dựng lên lần đầu.
mysql-hanoi-data:/var/lib/mysql	Ánh xạ vùng lưu trữ dữ liệu nghiệp vụ của MySQL ra ổ đĩa của Docker để đảm bảo tính toàn vẹn dữ liệu.
healthcheck	Phân đoạn cấu hình giúp Docker giám sát xem ứng dụng bên trong container có thực sự "sống" và sẵn sàng nhận kết nối hay không.
test	Câu lệnh hoặc script được thực thi định kỳ để kiểm tra trạng thái hoạt động của MySQL (ví dụ: mysqladmin ping).

interval	Tần suất thực hiện việc kiểm tra sức khỏe (Healthcheck) của container (Ví dụ: mỗi 10 giây).
timeout	Thời gian tối đa chờ đợi câu lệnh test trả về kết quả. Quá thời gian này sẽ tính là 1 lần lỗi.
retries	Số lần thử lại liên tiếp tối đa cho phép. Nếu vượt quá, Docker sẽ đánh dấu container là unhealthy.
start_period	Thời gian ân hạn ban đầu. Docker sẽ chờ container khởi động xong trong khoảng này rồi mới bắt đầu tính healthcheck.
volumes (ở cuối file)	Khai báo danh sách các Named Volumes độc lập toàn cục để Docker cấp phát không gian lưu trữ cứng trên máy Host.

Bảng 4.7 Bảng chi tiết cấu hình của docker

- Giải Thích Chi Tiết Các Cơ Chế Cấu Hình Đặc Thù

A. Cơ Chế Ánh Xạ Port (Cấu hình cổng kết nối)

Do 3 CSDL độc lập cùng chạy trên một máy Host (để test/triển khai), việc đổi port bên ngoài là bắt buộc để tránh xung đột hệ thống:

Site	Host Port (Cổng máy vật lý)	Container Port (Mặc định)	Ý nghĩa truy cập
Hà Nội	3307	3306	Kết nối CSDL Hà Nội qua địa chỉ localhost:3307
Đà Nẵng	3308	3306	Kết nối CSDL Đà Nẵng qua địa chỉ localhost:3308
TP.HCM	3309	3306	Kết nối CSDL TP.HCM qua địa chỉ localhost:3309

Bảng 4.8 Bảng ánh xạ port của mỗi site

Nguyên lý: Bên trong nội bộ mỗi container, MySQL luôn chạy ở port mặc định là 3306. Tuy nhiên, Docker Router sẽ làm nhiệm vụ dịch chuyển các request từ các port 3307, 3308, 3309 từ máy Host vào đúng port 3306 của từng container tương ứng.

B. Cấu Hình Bộ Mã Hóa Tiếng Việt (UTF8MB4)

Tham số cấu hình thông qua command:

YAML

```
command:
  --character-set-server=utf8mb4
  --collation-server=utf8mb4_unicode_ci
```

- utf8mb4: Là bảng mã tối ưu nhất của MySQL, hỗ trợ đầy đủ 4 bytes cho mỗi ký tự Unicode.
- Tác dụng: Giúp hệ thống ghi nhận chính xác tên khách hàng bằng tiếng Việt có dấu, nội dung giao dịch (trường description) không bị lỗi font/lỗi mã hóa dữ liệu, đồng thời hỗ trợ lưu trữ các ký tự đặc biệt hoặc biểu tượng (Emoji) nếu phát sinh từ ứng dụng Mobile Banking.

C. Cơ Chế Lưu Trữ Bền Vững (Named Volume)

Trong Docker, container có đặc tính "vòng đời ngắn" (Ephemerality) – nghĩa là nếu container bị xóa hoặc tạo mới, toàn bộ dữ liệu sinh ra trong quá trình chạy sẽ biến mất.

Để giải quyết vấn đề này trong hệ thống Ngân hàng, cấu hình sử dụng Named Volume:

YAML

```
mysql-hanoi-data:/var/lib/mysql
```

- mysql-hanoi-data: Tên vùng lưu trữ được quản lý độc lập bởi Docker Engine trên ổ cứng máy vật lý.
- /var/lib/mysql: Thư mục mặc định mà MySQL dùng để chứa các file dữ liệu (.ibd, tablespace, log...).
- Ý nghĩa: Khi container khởi chạy, thư mục /var/lib/mysql sẽ được "gắn liên kết" trực tiếp vào vùng mysql-hanoi-data trên máy host. Mọi thao tác ghi log giao dịch, số dư tài khoản thực chất đều được ghi thẳng vào ổ cứng máy Host. Nhờ vậy, ngay cả khi bạn chạy lệnh docker compose down để xóa bỏ container, dữ liệu giao dịch tài chính vẫn được bảo toàn nguyên vẹn 100% cho lần khởi động kế tiếp.

4.2.3 Khởi chạy cơ sở dữ liệu

- Khởi động toàn bộ hệ thống cơ sở dữ liệu: docker compose up -d
- Kiểm tra trạng thái container: docker compose ps

- Kết quả mong đợi:

```
NAME            IMAGE     COMMAND                  SERVICE        CREATED       STATUS        PORTS
mysql-danang    mysql:8   "docker-entrypoint.s…"   mysql-danang   4 days ago    Up 43 hours   0.0.0.0:3308->3306/tcp, [::]:3308->3306/tcp
mysql-hanoi     mysql:8   "docker-entrypoint.s…"   mysql-hanoi    4 days ago    Up 43 hours   0.0.0.0:3307->3306/tcp, [::]:3307->3306/tcp
mysql-hcm       mysql:8   "docker-entrypoint.s…"   mysql-hcm      4 days ago    Up 43 hours   0.0.0.0:3309->3306/tcp, [::]:3309->3306/tcp
```

Hình 4.2 Khởi chạy docker

4.2.4 Khởi tạo cấu trúc bảng

Sau khi các container MySQL được khởi chạy thành công, hệ thống tiến hành tạo cấu trúc dữ liệu cho từng site cơ sở dữ liệu. Mỗi database tại các site Hà Nội, Đà Nẵng và TP.HCM đều được khởi tạo cùng một tập bảng nhằm đảm bảo tính đồng nhất trong mô hình phân tán.

Hệ thống bao gồm 6 bảng dữ liệu chính như sau:

1. Bảng Chi nhánh (branch)

Bảng branch dùng để lưu thông tin định danh của các chi nhánh ngân hàng trong hệ thống.

Do hệ thống áp dụng kỹ thuật phân mảnh ngang, mỗi site chỉ lưu đúng một bản ghi đại diện cho chính chi nhánh của mình.

Ví dụ:

- Site Hà Nội chỉ lưu thông tin chi nhánh HN
- Site Đà Nẵng chỉ lưu thông tin chi nhánh DN
- Site TP.HCM chỉ lưu thông tin chi nhánh HCM

```sql
CREATE TABLE IF NOT EXISTS branch (
    branch_id   VARCHAR(10)  PRIMARY KEY COMMENT 'Mã chi nhánh: HN, DN, HCM',
    branch_name VARCHAR(100) NOT NULL    COMMENT 'Tên đầy đủ chi nhánh',
    city        VARCHAR(50)  NOT NULL    COMMENT 'Thành phố',
    created_at  TIMESTAMP    DEFAULT CURRENT_TIMESTAMP
);
```

Hình 4.3 Tạo bảng chi nhánh

2. Bảng Khách hàng (customer)

Bảng customer dùng để quản lý thông tin khách hàng thuộc từng chi nhánh.

Mỗi khách hàng sẽ được gắn với một branch_id nhằm xác định site quản lý dữ liệu tương ứng.

```sql
CREATE TABLE IF NOT EXISTS customer (
    customer_id BIGINT       AUTO_INCREMENT PRIMARY KEY COMMENT 'ID tự tăng',
    full_name   VARCHAR(100) NOT NULL     COMMENT 'Họ và tên',
    phone       VARCHAR(20)  UNIQUE       COMMENT 'SĐT – unique toàn site',
    email       VARCHAR(100)              COMMENT 'Email (tuỳ chọn)',
    address     VARCHAR(200)              COMMENT 'Địa chỉ',
    branch_id   VARCHAR(10)  NOT NULL     COMMENT 'FK → branch',
    created_at  TIMESTAMP    DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (branch_id) REFERENCES branch(branch_id)
);
```

Hình 4.4 Tạo bảng khách hàng

3. Bảng Tài khoản (account)

Bảng account dùng để quản lý thông tin tài khoản ngân hàng và số dư tiền tệ của khách hàng.

Trường balance được thiết kế với kiểu DECIMAL(15,2) nhằm tránh sai số dấu phẩy động trong các phép tính tài chính.

Ngoài ra, hệ thống sử dụng ràng buộc:

CHECK (balance >= 0)

để ngăn không cho số dư âm ở cấp độ cơ sở dữ liệu.

```sql
CREATE TABLE IF NOT EXISTS account (
    account_id  BIGINT        AUTO_INCREMENT PRIMARY KEY COMMENT 'ID tài khoản',
    customer_id BIGINT        NOT NULL     COMMENT 'FK → customer (chủ TK)',
    branch_id   VARCHAR(10)   NOT NULL     COMMENT 'FK → branch',
    balance     DECIMAL(15,2) NOT NULL DEFAULT 0.00 COMMENT 'Số dư (VND)',
    status      VARCHAR(20)   NOT NULL DEFAULT 'ACTIVE' COMMENT 'ACTIVE/INACTIVE/FROZEN',
    created_at  TIMESTAMP     DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (customer_id) REFERENCES customer(customer_id),
    FOREIGN KEY (branch_id)   REFERENCES branch(branch_id),
    CHECK (balance >= 0)
);
```

Hình 4.5 Tạo bảng account

4. Bảng Lịch sử giao dịch (transaction_history)

Bảng transaction_history dùng để ghi nhận toàn bộ lịch sử biến động số dư như:

- Nạp tiền
- Rút tiền
- Chuyển khoản nội bộ
- Chuyển khoản liên chi nhánh

Bảng này hỗ trợ đối soát giao dịch giữa các site thông qua trường related_branch_id.

```sql
CREATE TABLE IF NOT EXISTS transaction_history (
    transaction_id     BIGINT        AUTO_INCREMENT PRIMARY KEY,
    transaction_type   VARCHAR(30)   NOT NULL COMMENT 'DEPOSIT/WITHDRAW/TRANSFER_IN/TRANSFER_OUT/INTER_BRANCH_IN/INTER_BRANCH_OUT',
    amount             DECIMAL(15,2) NOT NULL COMMENT 'Số tiền giao dịch',
    account_id         BIGINT        NOT NULL COMMENT 'Tài khoản thực hiện',
    related_account_id BIGINT                  COMMENT 'TK đối ứng (nếu chuyển tiền)',
    related_branch_id  VARCHAR(10)             COMMENT 'Chi nhánh đối ứng (nếu liên CN)',
    balance_after      DECIMAL(15,2)           COMMENT 'Số dư sau giao dịch',
    status             VARCHAR(20)   NOT NULL DEFAULT 'SUCCESS' COMMENT 'SUCCESS/FAILED/ROLLED_BACK',
    distributed_txn_id VARCHAR(50)             COMMENT 'Link đến dist_txn_log',
    description        VARCHAR(200)            COMMENT 'Ghi chú',
    created_at         TIMESTAMP     DEFAULT CURRENT_TIMESTAMP
);
```

Hình 4.6 Tạo bảng lịch sử giao dịch

5. Bảng Theo dõi giao dịch phân tán (distributed_transaction_log)

Đây là bảng quan trọng nhất trong cơ chế giao dịch phân tán.

Bảng distributed_transaction_log được sử dụng bởi Coordinator để quản lý trạng thái tổng thể của một distributed transaction trong cơ chế Two Phase Commit (2PC).

Các trạng thái chính gồm:

- STARTED
- PREPARING
- PREPARED
- COMMITTED
- ABORTED

Bảng này giúp hệ thống:

- Theo dõi transaction liên chi nhánh
- Hỗ trợ rollback khi xảy ra lỗi

- Phục vụ recovery khi hệ thống gặp sự cố

```sql
CREATE TABLE IF NOT EXISTS distributed_transaction_log (
    txn_id            VARCHAR(50)   PRIMARY KEY COMMENT 'UUID giao dịch phân tán',
    txn_type          VARCHAR(30)   NOT NULL COMMENT 'INTER_BRANCH_TRANSFER',
    status            VARCHAR(20)   NOT NULL DEFAULT 'STARTED',
    source_branch     VARCHAR(10)   NOT NULL COMMENT 'Chi nhánh nguồn',
    dest_branch       VARCHAR(10)   NOT NULL COMMENT 'Chi nhánh đích',
    amount            DECIMAL(15,2) NOT NULL COMMENT 'Số tiền',
    source_account_id BIGINT        NOT NULL COMMENT 'TK nguồn',
    dest_account_id   BIGINT        NOT NULL COMMENT 'TK đích',
    error_message     VARCHAR(500)           COMMENT 'Lý do lỗi (nếu có)',
    created_at        TIMESTAMP     DEFAULT CURRENT_TIMESTAMP,
    updated_at        TIMESTAMP     DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
);
```

Hình 4.7 Tạo bảng theo dõi giao dịch phân tán

6. Bảng Thành phần tham gia transaction (transaction_participant)

Bảng transaction_participant dùng để lưu thông tin các participant (site tham gia) trong một distributed transaction.

Mỗi participant sẽ lưu:

- Site tham gia

- Vai trò (SOURCE/DESTINATION)
- Trạng thái prepare/commit/rollback
- Hành động thực hiện (DEBIT/CREDIT)

Bảng này giúp Coordinator theo dõi trạng thái của từng site trong quá trình Two Phase Commit.

```sql
CREATE TABLE IF NOT EXISTS transaction_participant (
    id         BIGINT      AUTO_INCREMENT PRIMARY KEY,
    txn_id     VARCHAR(50) NOT NULL COMMENT 'FK → distributed_transaction_log',
    branch_id  VARCHAR(10) NOT NULL COMMENT 'Site tham gia',
    role       VARCHAR(20) NOT NULL COMMENT 'SOURCE/DESTINATION',
    status     VARCHAR(20) NOT NULL DEFAULT 'PREPARING',
    action     VARCHAR(30)          COMMENT 'DEBIT/CREDIT',
    created_at TIMESTAMP   DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (txn_id) REFERENCES distributed_transaction_log(txn_id)
);
```

Hình 4.8 Tạo bảng thành phần tham gia

4.3 Cài đặt Backend

4.3.1 Cấu trúc thư mục Backend

Hệ thống Backend được tổ chức thành các package chính như sau:Phần Backend của hệ thống được phát triển bằng Spring Boot theo mô hình Layered Architecture (Kiến trúc phân tầng). Kiến trúc này giúp tách biệt rõ ràng giữa các tầng xử lý trong hệ thống, từ đó tăng khả năng bảo trì, mở rộng và tái sử dụng mã nguồn.

Hệ thống Backend chịu trách nhiệm:

- Xử lý logic nghiệp vụ ngân hàng
- Điều phối transaction phân tán
- Kết nối tới nhiều cơ sở dữ liệu độc lập
- Cung cấp REST API cho Frontend ReactJS

Luồng xử lý Request

Quá trình xử lý request trong hệ thống diễn ra theo nhiều tầng như sau:

1. Frontend ReactJS gửi HTTP Request tới Backend.
2. Controller tiếp nhận request và ánh xạ endpoint REST API.

3. Service xử lý logic nghiệp vụ.
4. JdbcTemplate thực hiện truy vấn SQL trực tiếp tới MySQL.
5. Entity và DTO được sử dụng để ánh xạ và truyền dữ liệu giữa các tầng.

Luồng xử lý này giúp hệ thống:

- Tách biệt rõ giữa tầng API và tầng nghiệp vụ
- Giảm sự phụ thuộc giữa các module
- Dễ dàng mở rộng chức năng mới
- Hỗ trợ triển khai các cơ chế phân tán như 2PC

Cấu trúc package Backend

Package	**Chức năng**
config	Chứa các lớp cấu hình hệ thống và cấu hình Multi-DataSource
controller	Tiếp nhận HTTP Request và trả JSON Response
service	Xử lý logic nghiệp vụ chính của hệ thống
dto	Chứa các Data Transfer Object dùng để trao đổi dữ liệu
entity	Chứa các lớp ánh xạ dữ liệu từ cơ sở dữ liệu
exception	Xử lý ngoại lệ toàn cục của hệ thống

Bảng 4.9 Bảng cấu trúc package

a) Package Config

Package config chứa toàn bộ các lớp cấu hình của hệ thống, đặc biệt là cấu hình cơ sở dữ liệu phân tán nhiều site.

Một số file tiêu biểu:

- CorsConfig.java
- DatabaseConfig.java
- TransactionConfig.java
- HanoiDataSourceConfig.java
- DanangDataSourceConfig.java
- HcmDataSourceConfig.java

Các lớp này chịu trách nhiệm:

- Khởi tạo DataSource
- Cấu hình TransactionManager
- Kết nối tới các database phân tán
- Quản lý transaction giữa các site

b) Package Controller

Package controller đóng vai trò tầng giao tiếp với phía client thông qua REST API.

Một số controller chính:

- AccountController.java
- TransferController.java
- QueryController.java

Nhiệm vụ chính:

- Nhận HTTP Request từ Frontend
- Validate dữ liệu đầu vào

- Gọi Service xử lý nghiệp vụ
- Trả kết quả JSON về cho client

c) Package service

Package service là tầng quan trọng nhất của hệ thống, nơi xử lý toàn bộ logic nghiệp vụ ngân hàng và transaction phân tán.

Các chức năng chính:

- Nạp tiền
- Rút tiền
- Chuyển khoản nội bộ
- Chuyển khoản liên chi nhánh
- Điều phối Two Phase Commit (2PC)
- Xử lý transaction đồng thời

Trong đó:

TransferService.java

đóng vai trò Coordinator trong cơ chế 2PC.

Lớp này chịu trách nhiệm:

- Gửi yêu cầu Prepare tới các site
- Kiểm tra trạng thái participant
- Ra quyết định Commit hoặc Rollback
- Đồng bộ transaction giữa các site

d) Package dto

Package dto chứa các lớp Data Transfer Object dùng để trao đổi dữ liệu giữa Frontend và Backend.

Ví dụ:

- DepositRequest.java
- WithdrawRequest.java
- InterBranchTransferRequest.java

DTO giúp:

- Tách biệt dữ liệu API và Entity
- Tăng tính bảo mật
- Dễ validate dữ liệu đầu vào

e) Package entity

Package entity chứa các lớp POJO ánh xạ dữ liệu từ MySQL.

Ví dụ:

- Account.java
- Customer.java
- TransactionHistory.java
- DistributedTransactionLog.java

Các entity giúp biểu diễn dữ liệu database dưới dạng object Java.

f) Package exception

Package exception dùng để xử lý ngoại lệ toàn cục của hệ thống.

Ví dụ:

GlobalExceptionHandler.java

Lớp này giúp:

- Chuẩn hóa response lỗi
- Trả mã HTTP phù hợp
- Giảm lặp code xử lý exception
- Tăng tính ổn định hệ thống

Điểm đặc biệt: Multi-DataSource

Khác với các hệ thống CRUD thông thường chỉ sử dụng một cơ sở dữ liệu duy nhất, hệ thống này triển khai mô hình Multi-DataSource nhằm kết nối đồng thời tới nhiều database độc lập.

Cơ chế này được hiện thực thông qua lớp:

SiteRouter.java

Lớp SiteRouter có nhiệm vụ:

- Xác định site cần truy cập
- Trả về JdbcTemplate tương ứng
- Trả về TransactionManager tương ứng

Ví dụ:

siteRouter.getJdbcTemplate(branchId)

siteRouter.getTransactionManager(branchId)

Thông qua cơ chế này, hệ thống có thể:

- Truy cập đúng database theo branch_id
- Điều phối transaction giữa nhiều site
- Hiện thực mô hình phân mảnh ngang (Horizontal Fragmentation)

Đây là thành phần quan trọng giúp hệ thống hoạt động như một cơ sở dữ liệu phân tán thực tế.

4.3.2 Cấu hình Multi-DataSource

Ứng dụng Spring Boot kết nối đồng thời tới ba database vật lý thông qua file cấu hình application.properties. Mỗi database đại diện cho một site trong hệ thống ngân hàng phân tán, tương ứng với các chi nhánh Hà Nội, Đà Nẵng và TP.HCM.

```properties
server.port=8080

# ============================================================
# SITE 1: MySQL Hà Nội (port 3307)
# ============================================================
app.datasource.hanoi.url=jdbc:mysql://localhost:3307/bank_hanoi?useSSL=false&allowPublicKeyRetrieval=true&serverTimezone=Asia/Ho_Chi_Minh
app.datasource.hanoi.username=root
app.datasource.hanoi.password=root
app.datasource.hanoi.driver-class-name=com.mysql.cj.jdbc.Driver

# ============================================================
# SITE 2: MySQL Đà Nẵng (port 3308)
# ============================================================
app.datasource.danang.url=jdbc:mysql://localhost:3308/bank_danang?useSSL=false&allowPublicKeyRetrieval=true&serverTimezone=Asia/Ho_Chi_Minh
app.datasource.danang.username=root
app.datasource.danang.password=root
app.datasource.danang.driver-class-name=com.mysql.cj.jdbc.Driver

# ============================================================
# SITE 3: MySQL TP.HCM (port 3309)
# ============================================================
app.datasource.hcm.url=jdbc:mysql://localhost:3309/bank_hcm?useSSL=false&allowPublicKeyRetrieval=true&serverTimezone=Asia/Ho_Chi_Minh
app.datasource.hcm.username=root
app.datasource.hcm.password=root
app.datasource.hcm.driver-class-name=com.mysql.cj.jdbc.Driver
```

***Hình 4.9 Cấu hình** Multi-DataSource*

Cấu hình DataSource

Ví dụ cấu hình DataSource cho site Hà Nội:

```java
@Configuration
public class HanoiDataSourceConfig {

    @Bean(name = "hanoiDataSource")
    @Primary
    public DataSource hanoiDataSource() {
        HikariDataSource ds = new HikariDataSource();
        ds.setJdbcUrl(
                "jdbc:mysql://localhost:3307/bank_hanoi?useSSL=false&allowPublicKeyRetrieval=true&serverTimezone=Asia/Ho_Chi_Minh");
        ds.setUsername("root");
        ds.setPassword("root");
        ds.setDriverClassName("com.mysql.cj.jdbc.Driver");
        ds.setConnectionTimeout(5000);
        ds.setMaximumPoolSize(10);
        ds.setPoolName("HanoiPool");
        return ds;
    }

    @Bean(name = "hanoiJdbcTemplate")
    @Primary
    public JdbcTemplate hanoiJdbcTemplate(@Qualifier("hanoiDataSource") DataSource ds) {
        return new JdbcTemplate(ds);
    }

    @Bean(name = "hanoiTransactionManager")
    @Primary
    public PlatformTransactionManager hanoiTransactionManager(
            @Qualifier("hanoiDataSource") DataSource ds) {

        DataSourceTransactionManager txManager = new DataSourceTransactionManager(ds);

        txManager.setTransactionSynchronization(
                DataSourceTransactionManager.SYNCHRONIZATION_NEVER);

        return txManager;
    }
}
```

Hình 4.10 Cấu hình datasource cho site Hà Nội

Mỗi site được cấu hình với:

- DataSource riêng
- JdbcTemplate riêng
- TransactionManager riêng
- TransactionTemplate riêng

Thiết kế này giúp hệ thống thao tác độc lập với từng database vật lý.

4.3.3 Cấu hình CORS

Do Frontend và Backend được triển khai trên hai cổng khác nhau:

- Frontend ReactJS chạy tại http://localhost:3000
- Backend Spring Boot chạy tại http://localhost:8080

nên trình duyệt sẽ áp dụng cơ chế bảo mật Same-Origin Policy và chặn các request khác domain nếu không được cấp quyền truy cập.

Để cho phép Frontend có thể gửi HTTP Request tới Backend thông qua REST API, hệ thống cấu hình CORS (Cross-Origin Resource Sharing) trong Spring Boot như sau:

```
@Configuration
public class CorsConfig {

    @Bean
    public WebMvcConfigurer corsConfigurer() {
        return new WebMvcConfigurer() {
            @Override
            public void addCorsMappings(CorsRegistry registry) {
                registry.addMapping("/api/**")
                        .allowedOrigins("*")
                        .allowedMethods("*")
                        .allowedHeaders("*");
            }
        };
    }
}
```

Hình 4.11 Cấu hình CORS

Giải thích cấu hình

Thành phần	**Ý nghĩa**
addMapping("/api/**")	Áp dụng CORS cho toàn bộ API
allowedOrigins("*")	Cho phép Frontend ReactJS truy cập
allowedMethods("*")	Cho phép tất cả HTTP methods
allowedHeaders("*")	Cho phép mọi request headers

Bảng 4.10 Giải thích cấu hình CORS

Ý nghĩa trong hệ thống

Cấu hình CORS đóng vai trò quan trọng trong mô hình Client–Server của hệ thống phân tán. Nhờ cấu hình này, ứng dụng Frontend có thể:

- Gửi request tới Backend
- Thực hiện CRUD dữ liệu
- Gọi API giao dịch ngân hàng
- Theo dõi kết quả transaction phân tán theo thời gian thực

mà không bị trình duyệt chặn do khác origin.

4.3.4 Cài đặt REST API

Hệ thống Backend được xây dựng theo kiến trúc RESTful API chuẩn hóa, đóng vai trò là cầu nối giao tiếp chính giữa Frontend (ReactJS) và Backend (Spring Boot). Để tăng khả năng bảo trì, kiểm thử độc lập và mở rộng quy mô trong tương lai, các Endpoint được phân tách rõ ràng thành nhiều Controller riêng biệt tương ứng với từng phân hệ nghiệp vụ.

Toàn bộ dữ liệu trao đổi giữa Client và Server đều được cấu trúc dưới định dạng JSON, tuân thủ nghiêm ngặt ngữ nghĩa của các phương thức HTTP chuẩn:

- GET: Truy vấn, đọc dữ liệu (Không làm thay đổi trạng thái hệ thống).
- POST: Tạo mới thực thể hoặc kích hoạt thực thi một luồng giao dịch tài chính.
- PUT: Cập nhật toàn diện dữ liệu của một thực thể hiện có.
- DELETE: Xóa bỏ dữ liệu (hoặc đánh dấu xóa logic).

Các Phân Hệ Controller Chính trong Hệ Thống

a) AccountController và TransferController (Phân hệ Giao dịch & Tài khoản)

Hai bộ điều phối này chịu trách nhiệm quản lý, xử lý các nghiệp vụ tài chính cốt lõi từ đơn giản (Cục bộ) đến phức tạp (Phân tán), tác động trực tiếp đến số dư và dòng tiền của hệ thống ngân hàng.

HTTP Method & API Endpoint	Chức năng nghiệp vụ	Tính chất xử lý (Transaction Type)
POST /api/transactions/deposit	Nạp tiền vào tài khoản	Local Transaction (Xử lý đơn nhiệm trên 1 site duy nhất)
POST /api/transactions/withdraw	Rút tiền khỏi tài khoản	Local Transaction (Xử lý đơn nhiệm trên 1 site duy nhất)
POST /api/transfers/internal	Chuyển khoản nội bộ cùng chi nhánh	Local Transaction (Xử lý đơn nhiệm trên 1 site duy nhất)
POST /api/transfers/inter-branch	Chuyển khoản liên chi nhánh	Distributed Transaction (Giao dịch phân tán, kích hoạt cơ chế Two-Phase Commit - 2PC)
POST /api/transactions/concurrent-withdraw	Mô phỏng xử lý giao dịch đồng thời	Concurrency Testing (Kiểm thử tranh chấp tài nguyên và hiện tượng Race Condition)

Bảng 4.11 Phân hệ Giao dịch & Tài khoản

Phân biệt cơ chế xử lý:

- Local Transaction: Hệ thống chỉ thiết lập kết nối (Connection) và mở Transaction trên một phân mảnh CSDL duy nhất chứa tài khoản đó.
- Distributed Transaction (/inter-branch): Kích hoạt tầng dịch vụ đóng vai trò là Coordinator, điều phối lệnh chuẩn bị (Prepare) và thực thi (Commit) đồng thời tới các site nguồn và site đích để đảm bảo tính toàn vẹn dữ liệu xuyên suốt hệ thống.

b) CustomerController và BranchController (Phân hệ Quản lý Danh mục - Master Data)

Hệ thống Controller này chịu trách nhiệm quản lý, tra cứu các thông tin nền tảng, danh mục dùng chung hoặc dữ liệu gốc của toàn hệ thống ngân hàng.

HTTP Method & API Endpoint	Chức năng nghiệp vụ	Vai trò trong hệ thống phân tán
GET /api/customers/branch/{branchId}	Lấy danh sách khách hàng theo chi nhánh	Hỗ trợ truy vấn và hiển thị dữ liệu đã được phân mảnh ngang.
POST /api/customers	Thêm mới hồ sơ khách hàng	Khởi tạo dữ liệu khách hàng vào đúng phân mảnh CSDL mục tiêu.

GET /api/branches	Lấy danh sách toàn bộ các chi nhánh	Hỗ trợ đồng bộ và tra cứu danh mục chi nhánh dùng chung giữa các site.

Bảng 4.12 Phân hệ Quản lý Danh mục - Master Data

c) QueryController (Phân hệ Truy vấn và Tổng hợp dữ liệu phân tán)

QueryController đảm nhận các tác vụ xử lý báo cáo, số liệu thống kê tổng hợp phục vụ giao diện Dashboard giám sát toàn quốc.

HTTP Method & API Endpoint	Chức năng nghiệp vụ	Cơ chế xử lý kỹ thuật
GET /api/stats/total-balance	Thống kê tổng số dư tiền gửi toàn hệ thống	Thực thi Distributed Query (Truy vấn phân tán), quét qua cả 3 site để tính tổng.
GET /api/stats/history/deposit	Truy vấn lịch sử nạp tiền từ tất cả các site	Gom và gộp dữ liệu lịch sử nạp tiền toàn quốc.

Bảng 4.13 Phân hệ Truy vấn và Tổng hợp dữ liệu phân tán

Luồng hoạt động của Distributed Query tại tầng API:

1. Nhận yêu cầu truy vấn từ Frontend.
2. Backend kích hoạt cơ chế bất đồng bộ hoặc tuần tự, phát lệnh SELECT tới cả 3 cơ sở dữ liệu độc lập (bank_hanoi, bank_danang, bank_hcm).
3. Thu thập và tập hợp (Aggregate) các tập kết quả (Result Set) trả về từ từng node CSDL.
4. Thực hiện tính toán tổng hợp (Gom cụm, tính tổng, sắp xếp lại theo thời gian) tại bộ nhớ của Application Server (Backend).
5. Đóng gói thành một đối tượng dữ liệu duy nhất, chuẩn hóa định dạng JSON và phản hồi về cho Frontend hiển thị.

Kiến Trúc Đặc Trưng của Hệ Thống REST API

Hệ thống REST API được thiết kế và vận hành dựa trên các nguyên tắc nền tảng sau:

- Tách biệt tuyệt đối giữa Giao diện và Logic (Decoupling): Frontend (ReactJS) độc lập hoàn toàn với Backend (Spring Boot), kết nối duy nhất qua các cổng API, tạo điều kiện phát triển song song giữa hai đội ngũ.
- Khả năng mở rộng tuyến tính (Scalability): Việc module hóa API theo từng nhóm Controller riêng biệt giúp hệ thống dễ dàng bổ sung thêm các endpoint nghiệp vụ mới hoặc tích hợp thêm chi nhánh (Site thứ 4, thứ 5...) mà không phá vỡ kiến trúc cũ.
- Hỗ trợ toàn vẹn giao dịch phân tán: Tích hợp sâu với các bộ quản lý giao dịch để xử lý an toàn các luồng tiền xuyên biên giới dữ liệu.
- Chuẩn hóa dữ liệu toàn hệ thống: Mọi kết quả (Dù thành công hay thất bại) đều được bọc trong một cấu trúc JSON đồng nhất, giúp ứng dụng Client dễ dàng bắt lỗi và hiển thị thông báo trực quan cho người dùng.

4.3.5 Cài đặt xử lý giao dịch phân tán

Để giải quyết bài toán xử lý đồng thời và đảm bảo tính nhất quán dữ liệu trong môi trường phân tán, hệ thống áp dụng nhiều cơ chế khóa và quản lý transaction trực tiếp tại tầng Backend.

Các kỹ thuật này giúp hệ thống:

- Ngăn chặn mất dữ liệu khi nhiều transaction thực hiện đồng thời
- Hạn chế hiện tượng deadlock
- Đảm bảo tính toàn vẹn dữ liệu
- Hỗ trợ triển khai cơ chế Two-Phase Commit (2PC)

a) Pessimistic Locking (Khóa bi quan)

Hệ thống sử dụng cơ chế khóa bi quan thông qua câu lệnh:

SELECT ... FOR UPDATE

để khóa dòng dữ liệu (*row-level lock*) trước khi cập nhật số dư tài khoản.

```java
// Ví dụ trong hàm withdraw() của AccountService
BigDecimal currentBalance = jdbc.queryForObject(
        "SELECT balance FROM account WHERE account_id = ? FOR UPDATE",
        BigDecimal.class, request.getAccountId());
```

Hình 4.12 Pessimistic Locking

Khi transaction thực hiện câu lệnh trên:

- Dòng dữ liệu tài khoản sẽ bị khóa
- Các transaction khác phải chờ cho đến khi transaction hiện tại commit hoặc rollback
- Không transaction nào có thể cập nhật cùng lúc trên cùng một tài khoản

Cơ chế này giúp ngăn chặn lỗi:

Lost Update trong trường hợp nhiều người dùng cùng thao tác trên một tài khoản tại cùng thời điểm.

b) Ordered Locking (Khóa theo thứ tự ưu tiên)

Trong chức năng chuyển khoản nội bộ (internalTransfer()), hệ thống áp dụng kỹ thuật khóa theo thứ tự nhằm tránh hiện tượng deadlock.

Ví dụ:

- Transaction A chuyển tiền từ tài khoản 1 → 2
- Transaction B chuyển tiền từ tài khoản 2 → 1

Nếu hai transaction khóa dữ liệu theo thứ tự khác nhau, hệ thống có thể xảy ra deadlock.

Để giải quyết vấn đề này, hệ thống ép buộc thứ tự khóa theo ID từ nhỏ đến lớn:

```
Long firstId = Math.min(request.getFromAccountId(), request.getToAccountId());
Long secondId = Math.max(request.getFromAccountId(), request.getToAccountId());

jdbc.queryForObject("SELECT balance FROM account WHERE account_id = ? FOR UPDATE", BigDecimal.class, firstId);
jdbc.queryForObject("SELECT balance FROM account WHERE account_id = ? FOR UPDATE", BigDecimal.class, secondId);
```

Hình 4.13 Ordered Locking

Cơ chế này đảm bảo:

- Tất cả transaction đều khóa dữ liệu theo cùng một thứ tự
- Loại bỏ khả năng chờ vòng tròn
- Giảm nguy cơ deadlock trong môi trường xử lý đồng thời

c) Native Transaction Management cho 2PC

Trong chức năng chuyển tiền liên chi nhánh (TransferService.interBranchTransfer()), hệ thống triển khai cơ chế Two-Phase Commit bằng cách quản lý nhiều transaction độc lập trên các kết nối database khác nhau.

Coordinator sử dụng PlatformTransactionManager để điều khiển transaction tại từng site.

Ví dụ:

PlatformTransactionManager sourceTxManager =

siteRouter.getTransactionManager(sourceBranch);

```
PlatformTransactionManager destTxManager =
    siteRouter.getTransactionManager(destBranch);
```

=> Mỗi TransactionManager tương ứng với một database vật lý độc lập.

Phase 1 – Prepare Phase

Coordinator khởi tạo transaction tại cả hai site nhưng chưa commit:

```
TransactionStatus sourceTx =
    sourceTxManager.getTransaction(
        new DefaultTransactionDefinition()
    );

TransactionStatus destTx =
    destTxManager.getTransaction(
        new DefaultTransactionDefinition()
    );
```

Trong giai đoạn này:

- Site nguồn thực hiện trừ tiền
- Site đích thực hiện cộng tiền
- Dữ liệu vẫn chưa được commit xuống database
- Các participant chuyển sang trạng thái PREPARED

Nếu toàn bộ participant đều thành công, Coordinator sẽ chuyển sang Commit Phase.

Phase 2 – Commit Phase

Coordinator gửi quyết định commit tới toàn bộ participant:

```
sourceTxManager.commit(sourceTx);
destTxManager.commit(destTx);
```

Khi đó:

- Dữ liệu được ghi vĩnh viễn xuống database
- Distributed transaction hoàn tất thành công

Nếu một site gặp lỗi trong quá trình Prepare hoặc Commit, hệ thống sẽ rollback transaction trên toàn bộ site nhằm đảm bảo tính nhất quán dữ liệu.

Rollback khi xảy ra lỗi

Trong trường hợp:

- Mất kết nối database
- Site bị crash
- Không đủ số dư
- Participant prepare thất bại

Coordinator sẽ thực hiện rollback() trên toàn bộ transaction đang mở để đưa hệ thống về trạng thái ban đầu.

Cơ chế này giúp đảm bảo:

- Không mất tiền
- Không commit một phần transaction
- Đảm bảo tính Atomicity trong môi trường phân tán

Vai trò của cơ chế xử lý transaction phân tán

Việc triển khai các kỹ thuật khóa và transaction management giúp hệ thống:

- Xử lý transaction đồng thời an toàn
- Ngăn chặn race condition
- Đảm bảo tính nhất quán dữ liệu giữa nhiều site
- Mô phỏng hoạt động thực tế của hệ quản trị cơ sở dữ liệu phân tán
- Hiện thực cơ chế Two-Phase Commit (2PC) trong hệ thống ngân hàng phân tán

4.3.6 Biên dịch và chạy Backend

Sau khi hoàn tất cấu hình hệ thống và cài đặt các dependency cần thiết, ứng dụng Backend được biên dịch và khởi chạy thông qua Gradle Wrapper tại thư mục:

backend/

Việc sử dụng Gradle Wrapper (gradlew) giúp hệ thống:

- Tự động tải đúng phiên bản Gradle
- Đảm bảo tính đồng nhất môi trường chạy
- Không yêu cầu cài Gradle thủ công trên máy người dùng

Tại thư mục backend, thực hiện lệnh:

```
./gradlew build
```

Lệnh này có nhiệm vụ:

- Tải các dependency từ Maven Central
- Biên dịch mã nguồn Java
- Kiểm tra lỗi biên dịch
- Build toàn bộ project Spring Boot

Nếu build thành công, Gradle sẽ tạo thư mục:

build/

chứa file .jar và các artifact cần thiết để chạy ứng dụng.

Bước 2: Khởi chạy ứng dụng Spring Boot

Sau khi build thành công, hệ thống được khởi chạy bằng lệnh:

./gradlew bootRun

Khi thực thi lệnh trên:

- Spring Boot sẽ khởi tạo toàn bộ Bean
- Tạo kết nối tới 3 database phân tán
- Khởi tạo Multi-DataSource
- Đăng ký REST API
- Kích hoạt TransactionManager cho từng site

Nếu quá trình khởi động thành công, terminal sẽ hiển thị:

Started BankingApplication in ...

Địa chỉ hoạt động của Backend

Sau khi khởi động hoàn tất, Backend sẽ hoạt động tại địa chỉ:

http://localhost:8080

Đây là cổng mặc định của ứng dụng Spring Boot.

Frontend ReactJS sẽ gửi HTTP Request tới địa chỉ này để thực hiện các chức năng:

- Nạp tiền
- Rút tiền
- Chuyển khoản
- Chuyển tiền liên chi nhánh

- Truy vấn thống kê phân tán

Vai trò của Backend trong hệ thống phân tán

Trong mô hình hệ thống ngân hàng phân tán, Backend Spring Boot đóng vai trò:

Coordinator

trong cơ chế Two-Phase Commit (2PC).

Backend chịu trách nhiệm:

- Điều phối transaction giữa các site
- Quản lý trạng thái transaction phân tán
- Gửi yêu cầu Prepare/Commit/Rollback
- Đồng bộ dữ liệu giữa các database
- Đảm bảo tính nhất quán dữ liệu toàn hệ thống

Nhờ đó, hệ thống có thể mô phỏng hoạt động của một hệ quản trị cơ sở dữ liệu phân tán thực tế trong môi trường ngân hàng.

4.4 Cài đặt Frontend

4.4.1 Cấu trúc thư mục Frontend

Phần Frontend của hệ thống được xây dựng bằng ReactJS kết hợp với công cụ build Vite theo mô hình Component-Based Architecture. Cấu trúc thư mục được tổ chức theo hướng module hóa nhằm tăng khả năng bảo trì, tái sử dụng và mở rộng chức năng trong tương lai.

Cấu trúc tổng quát của thư mục src/ được tổ chức như sau:

- api/: Cấu hình thư viện Axios và định nghĩa sẵn các phương thức gọi REST API (ví dụ: bankApi.js).
- components/: Chứa các component tái sử dụng nhiều lần và cả layout giao diện dùng chung cho toàn hệ thống (như file AppLayout.jsx chứa Sidebar và Header).
- pages/: Nơi chứa các màn hình chức năng chính của hệ thống ngân hàng phân tán. Mỗi file đại diện cho một module nghiệp vụ riêng biệt:
    - **Dashboard.jsx: Trang tổng quan hệ thống.**

- **Customers.jsx: Quản lý khách hàng.**
- **Accounts.jsx: Quản lý tài khoản.**
- **Transactions.jsx: Giao dịch cơ bản (Nạp, Rút tiền).**
- **Transfers.jsx: Chuyển tiền nội bộ và chuyển tiền phân tán liên chi nhánh (2PC).**
- **Branches.jsx: Xem danh sách các chi nhánh.**
- **Statistics.jsx: Bảng điều khiển thực hiện các truy vấn thống kê gom cụm phân tán.**

Đặc điểm kiến trúc Frontend

Frontend được xây dựng theo mô hình SPA (Single Page Application), trong đó:

- **ReactJS: Chịu trách nhiệm render giao diện phía Client.**
- **React Router DOM (App.jsx): Điều hướng mượt mà giữa các module mà không cần tải lại trang.**
- **Axios: Thực hiện giao tiếp bất đồng bộ với Backend.**
- **Ant Design: Cung cấp hệ thống UI Component chuyên nghiệp, nhất quán.**

4.4.2 Cấu hình Axios và Proxy

A. Cấu hình Proxy (vite.config.js)

Để giải quyết triệt để vấn đề CORS (Cross-Origin Resource Sharing) và Same-Origin Policy trong quá trình phát triển (Development), hệ thống không fix cứng địa chỉ host trong Axios mà sử dụng cơ chế Proxy của Vite.

```
export default defineConfig({
  plugins: [react()],
  server: {
    port: 3000,
    proxy: {
      '/api': {
        target: 'http://localhost:8080',
        changeOrigin: true,
      },
    },
  },
})
```

Hình 4.14 Cấu hình Proxy

Nhờ khai báo này, mọi request gửi tới đường dẫn /api/... trên Frontend sẽ được Vite tự động luân chuyển tới máy chủ Spring Boot ở cổng 8080.

B. Cấu hình Axios (src/api/bankApi.js)

Hệ thống tạo ra một instance duy nhất của Axios để tái sử dụng:

```
import axios from 'axios';

const API = axios.create({
  baseURL: '/api',
  timeout: 10000,
});
export default API;
```

Hình 4.15 Cấu hình Axios

C. Lợi ích của cấu trúc gọi API tập trung

Việc định nghĩa sẵn các endpoint (ví dụ: transferApi.interBranch, transactionApi.concurrentWithdraw) tại một nơi duy nhất giúp Frontend:

- Dễ thay đổi URL khi hệ thống Backend mở rộng.
- Giảm thiểu việc lặp lại các đoạn mã nối chuỗi URL trong các Component.
- Dễ dàng tích hợp các hệ thống phân tích lỗi (Interceptor) sau này.

4.4.3 Điều hướng ứng dụng

Hệ thống sử dụng React Router DOM để quản lý cây điều hướng tập trung trong tệp App.jsx. Lớp vỏ bọc <AppLayout> được cấu hình cố định tại tầng Root nhằm duy trì thanh điều hướng Sidebar và thành phần Header luôn hiển thị đồng bộ trên toàn bộ giao diện.

Các trang nghiệp vụ chính được thiết lập bản đồ tương ứng với các đường dẫn (Routes) cụ thể như sau:

- **/dashboard: Chuyển hướng tới trang Dashboard (Tổng quan hệ thống).**
- **/branches: Chuyển hướng tới màn hình quản lý Chi nhánh (Branches).**
- **/customers: Chuyển hướng tới màn hình quản lý danh sách Khách hàng (Customers).**
- **/accounts: Chuyển hướng tới màn hình quản lý Tài khoản (Accounts).**
- **/transactions: Chuyển hướng tới màn hình thực hiện giao dịch Rút/Nạp tiền.**
- **/transfers: Chuyển hướng tới màn hình điều phối Chuyển khoản phân tán.**
- **/statistics: Chuyển hướng tới màn hình Báo cáo và Thống kê dữ liệu.**

4.4.4 Các chức năng chính

Giao diện Frontend được thiết kế và xây dựng bám sát theo cấu trúc các API nghiệp vụ từ phía Backend, đảm bảo mô phỏng trực quan và sinh động nhất các hành vi đặc trưng của một hệ cơ sở dữ liệu phân tán:

a) Trang Customers và Accounts

- Chức năng: Hỗ trợ các tác vụ tìm kiếm, thêm mới, xem danh sách khách hàng và danh sách tài khoản ngân hàng.
- Đặc điểm phân tán: Vì dữ liệu hệ thống được tổ chức theo mô hình Phân mảnh ngang (Horizontal Fragmentation), giao diện được tích hợp sẵn các bộ chọn tham số lọc theo mã chi nhánh (branch_id). Cơ chế này giúp Frontend đính kèm tham số định tuyến chính xác để gửi request tới đúng site vật lý đang quản lý dữ liệu gốc.

b) Trang Transactions (Giao dịch cục bộ)

- Chức năng: Xử lý trực tiếp các nghiệp vụ tài chính cơ bản tại chỗ bao gồm nạp tiền và rút tiền.
- Điểm đặc biệt: Trang này tích hợp tính năng nâng cao Demo Xử lý tương tranh (Concurrency Withdraw). Chức năng này cho phép giả lập kịch bản hai người dùng hoặc hai tiến trình cùng thực hiện rút tiền trên một tài khoản tại một thời điểm, qua đó chứng minh trực quan cơ chế ngăn chặn lỗi mất cập nhật (Lost Update) nhờ vào giải pháp khóa bi quan (Pessimistic Locking) dưới tầng cơ sở dữ liệu.

c) Trang Transfers (Giao dịch phân tán)

- Chức năng: Đây là module trọng tâm và quan trọng nhất của đồ án, chịu trách nhiệm thực thi các luồng chuyển khoản nội bộ chi nhánh và chuyển khoản liên chi nhánh.
- Điểm đặc biệt: Cung cấp giao diện đồ họa trực quan minh họa từng bước vận hành của giao thức Two-Phase Commit (2PC). Hệ thống tích hợp sẵn cờ tùy chọn simulateCrash ngay trên giao diện để người dùng có thể chủ động gây lỗi hệ thống (ví dụ: giả lập sập mạng tại Site Đích). Từ đó, người vận hành có thể quan sát trực tiếp cách Nút điều phối (Coordinator) phát lệnh hủy bỏ và thực hiện Database Rollback đồng bộ để bảo vệ an toàn tài sản của khách hàng.

d) Trang Statistics (Thống kê phân tán)

- Chức năng: Thiết lập bảng điều khiển và biểu đồ để thực hiện các truy vấn phân tán nâng cao trên toàn bộ mạng lưới hệ thống (Distributed Queries).

- Ứng dụng thực tế: Cho phép tính toán tổng số dư tiền gửi toàn hệ thống, liệt kê top các giao dịch liên chi nhánh, hoặc xem lịch sử nạp/rút tiền được gom từ cả 3 site dữ liệu (bank_hanoi, bank_danang, bank_hcm).
- Cơ chế xử lý: Các tác vụ này buộc Nút điều phối Spring Boot phải phát lệnh quét dữ liệu đồng thời qua nhiều máy chủ MySQL độc lập, thực hiện gom cụm và tổng hợp kết quả (Aggregate) tại bộ nhớ đệm trước khi chuyển đổi thành dữ liệu biểu đồ trực quan trả về Frontend.

4.4.5 Chạy Frontend

Sau khi hoàn tất quá trình cài đặt toàn bộ các thư viện phụ thuộc (dependencies) bằng lệnh npm install, ứng dụng Frontend ReactJS sẽ được khởi chạy thông qua công cụ tối ưu hóa Vite Development Server.

Để vận hành hệ thống ở môi trường phát triển, thực hiện lệnh sau tại thư mục frontend:

Bash: npm run dev

Trong quá trình khởi chạy, trình quản lý Vite sẽ thực hiện tự động chuỗi tác vụ sau:

- Biên dịch mã nguồn: Biên dịch nhanh (Hot Module Replacement - HMR) toàn bộ mã nguồn ReactJS/JSX sang JavaScript thuần mà trình duyệt có thể hiểu được.
- Kích hoạt mạng lưới liên kết: Thiết lập luồng Proxy nội bộ hướng thẳng tới Backend Spring Boot (cổng 8080) để sẵn sàng xử lý các request API ngầm.
- Cấp phát cổng dịch vụ: Mở cổng phục vụ và lắng nghe phản hồi giao diện.

Địa chỉ truy cập hệ thống

Do cấu hình trong tệp vite.config.js đã chủ động ghi đè cổng mặc định của Vite, hệ thống Frontend sẽ khởi chạy thành công tại địa chỉ cố định: **http://localhost:3000**

Người dùng hoặc kỹ sư vận hành có thể sử dụng các trình duyệt web phổ biến (như Google Chrome, Microsoft Edge, Mozilla Firefox...) truy cập đường dẫn trên để trực tiếp tương tác với giao diện quản lý ngân hàng, đồng thời thực thi kiểm thử các kịch bản dòng chảy dữ liệu trên mạng lưới cơ sở dữ liệu phân tán.

4.5 Triển khai hệ thống

4.5.1 Quy trình triển khai

Quy trình triển khai hệ thống được tối ưu hóa theo mô hình tự động hóa tối đa nhằm vận hành trơn tru trong môi trường phát triển cục bộ (*Local Development*).

Yêu cầu tiên quyết về môi trường máy chủ (Prerequisites):

- Docker & Docker Compose v2 trở lên.
- Java Development Kit (JDK) phiên bản 17.
- Node.js (phiên bản LTS) và trình quản lý gói npm.

Các bước khởi chạy hệ thống được thực hiện tuần tự theo quy trình sau:

- **Bước 1: Khởi chạy cụm cơ sở dữ liệu phân tán** Mở terminal tại thư mục gốc của dự án (nơi chứa tệp cấu hình docker-compose.yml) và thực thi lệnh:
- Bash: ***docker compose up -d***
- *Cơ chế vận hành:* Lệnh này sẽ tải xuống Docker Image mysql:8 chính thức, khởi tạo và chạy ngầm (-d) song song 3 container độc lập tương ứng với 3 phân mảnh CSDL của 3 chi nhánh vùng miền.
- **Bước 2: Tự động khởi tạo cấu trúc dữ liệu (Auto-Init Schema)** Hệ thống áp dụng cơ chế tự động hóa bằng cách gắn kết (*Mounting*) trực tiếp các tệp cấu hình SQL vào thư mục đặc biệt /docker-entrypoint-initdb.d/ bên trong mỗi container. Ngay trong lần đầu tiên khởi chạy, hệ quản trị cơ sở dữ liệu MySQL sẽ tự động quét và thực thi tuần tự các tệp script script cấu trúc dữ liệu: init-hanoi.sql, init-danang.sql, và init-hcm.sql. Quá trình này tự động xây dựng cấu trúc hệ thống 6 bảng chính và nạp sẵn dữ liệu kiểm thử (Master Data) mà không cần lập trình viên phải thao tác thủ công.
- **Bước 3: Biên dịch và khởi chạy tầng Điều phối (Backend)** Mở một cửa sổ Terminal mới, di chuyển vào phân hệ backend và kích hoạt trình biên dịch Gradle của Spring Boot:
- Bash: ***cd backend***

  ./gradlew bootRun
- *Cơ chế vận hành:* Ứng dụng Backend Spring Boot sẽ thiết lập đồng thời cụm liên kết đa nguồn dữ liệu (*Multi-DataSource*) kết nối trực tiếp đến 3 node CSDL vật lý đã dựng ở Bước 1 thông qua cấu hình định tuyến động.

- **Bước 4: Biên dịch và khởi chạy tầng Giao diện (Frontend)** Mở một cửa sổ Terminal độc lập khác, di chuyển vào phân hệ frontend để cài đặt gói thư viện và kích hoạt máy chủ Vite:
- Bash

  cd frontend

  npm install

  npm run dev
- **Bước 5: Truy cập hệ thống** Sau khi các phân hệ thông báo khởi động thành công và trạng thái kiểm tra sức khỏe (*Healthcheck*) của các container chuyển sang màu xanh, người dùng mở trình duyệt web và truy cập vào địa chỉ đích: **http://localhost:3000**

4.5.2 Kiến trúc triển khai

Khi hệ thống vận hành thực tế, kiến trúc tổng thể được tổ chức chặt chẽ theo mô hình Kiến trúc 3 tầng (3-Tier Architecture), giúp phân tách rạch ròi các vai trò xử lý:

1. Tầng Giao diện (Presentation Tier - Frontend ReactJS): Vận hành trực tiếp trên trình duyệt của người dùng với vai trò là phía Client. Phân hệ này chịu trách nhiệm thu thập thao tác người dùng, render đồ họa ứng dụng đơn trang (SPA) và phát đi các luồng truy vấn dữ liệu không đồng bộ (HTTP REST API) thông qua Axios.
2. Tầng Điều phối & Nghiệp vụ (Logic Tier - Backend Spring Boot): Đóng vai trò là lớp trung gian (*Middleware*). Tầng này trực tiếp xử lý các luồng yêu cầu RESTful, phân tích logic tài chính, thực thi các thuật toán khóa dữ liệu tương tranh và đóng vai trò là Bộ điều phối trung tâm (2PC Coordinator) để kiểm soát các nhánh giao dịch phân tán.
3. Tầng Lưu trữ Dữ liệu (Data Tier - 3 MySQL Sites): Gồm 3 phân mảnh cơ sở dữ liệu vật lý hoạt động biệt lập (bank_hanoi, bank_danang, bank_hcm), quản lý cấu trúc dữ liệu phân mảnh ngang. Giao tiếp giữa tầng Backend nghiệp vụ và tầng lưu trữ được duy trì thông qua giao thức kết nối JDBC hiệu năng cao do bộ thư viện HikariCP tối ưu hóa.

4.5.3 Cổng mạng sử dụng

Để đảm bảo các dịch vụ, container và ứng dụng độc lập không xảy ra hiện tượng xung đột tài nguyên mạng khi chạy song song trên cùng một máy chủ vật lý, cấu hình hệ thống thực hiện phân định rạch ròi các cổng mạng (Ports) như sau:

Tên Dịch Vụ	**Công Nghệ Triển Khai**	**Cổng Hoạt Động (Port)**	**Chức Năng Hệ Thống**
Frontend	ReactJS / Vite Server	3000	Kết xuất đồ họa và hiển thị giao diện người dùng ứng dụng SPA.
Backend	Spring Boot Framework	8080	Tiếp nhận API, xử lý nghiệp vụ và làm nút điều phối giao dịch trung tâm.
MySQL HN	Docker Container	3307	Lưu trữ và quản lý CSDL vật lý của chi nhánh Hà Nội (bank_hanoi).
MySQL DN	Docker Container	3308	Lưu trữ và quản lý CSDL vật lý của chi nhánh Đà Nẵng (bank_danang).

MySQL HCM	Docker Container	3309	Lưu trữ và quản lý CSDL vật lý của chi nhánh TP.HCM (bank_hcm).

Bảng 4.14 Cổng mạng sử dụng

4.6 Kiểm thử hệ thống

4.6.1 Kiểm thử kết nối

Sau khi hoàn tất quá trình triển khai Backend và các cơ sở dữ liệu phân tán, hệ thống tiến hành kiểm thử kết nối nhằm đảm bảo Backend có thể giao tiếp thành công với các site MySQL và các REST API hoạt động bình thường.

Việc kiểm thử được thực hiện thông qua công cụ curl bằng cách gửi HTTP Request trực tiếp tới các endpoint của Backend Spring Boot.

Ví dụ kiểm tra API lấy danh sách chi nhánh: ***curl http://localhost:8080/branches***

Nếu hệ thống hoạt động, Backend sẽ trả về dữ liệu JSON chứa danh sách các chi nhánh ngân hàng:

```json
{
  "success": true,
  "message": "Success",
  "data": [
    {
      "branchId": "HN",
      "branchName": "Chi nhánh Hà Nội",
      "city": "Hà Nội",
      "createdAt": "2026-05-28T06:41:02"
    },
    {
      "branchId": "DN",
      "branchName": "Chi nhánh Đà Nẵng",
      "city": "Đà Nẵng",
      "createdAt": "2026-05-28T06:41:02"
    },
    {
      "branchId": "HCM",
      "branchName": "Chi nhánh TP. Hồ Chí Minh",
      "city": "TP. Hồ Chí Minh",
      "createdAt": "2026-05-28T06:41:02"
    }
  ]
}
```


Hình 4.16 Kiểm thử kết nối

Kiểm tra kết nối tới từng site

Tiếp theo, hệ thống tiến hành kiểm thử khả năng cô lập và đọc dữ liệu từ từng phân mảnh CSDL độc lập dựa trên tham số đường dẫn *branchId*.

Truy xuất dữ liệu phân mảnh Hà Nội (Site 1):

Bash: curl -s http://localhost:8080/api/branches/HN

Kết quả trả về:

```JSON
{
  "success": true,
  "message": "Success",
  "data": {
    "branchId": "HN",
    "branchName": "Chi nhánh Hà Nội",
    "city": "Hà Nội"
  }
}
```

Hình 4.17 Kiểm tra kết nối tới từng site

Thực hiện tương tự để xác nhận kết nối JDBC tới các site còn lại:

Bash

curl -s http://localhost:8080/api/branches/DN

curl -s http://localhost:8080/api/branches/HCM

Các request trên giúp xác nhận:

- Backend kết nối thành công tới nhiều DataSource
- Cơ chế định tuyến theo branchId hoạt động chính xác
- Dữ liệu được truy xuất đúng site tương ứng

Ý nghĩa của bước kiểm thử

Bước kiểm thử kết nối đóng vai trò quan trọng trong hệ thống cơ sở dữ liệu phân tán vì giúp xác minh:

- Các container MySQL đã hoạt động ổn định
- Backend Spring Boot kết nối thành công tới toàn bộ site
- REST API hoạt động bình thường
- Multi-DataSource được cấu hình chính xác

- Hệ thống sẵn sàng cho các giao dịch phân tán tiếp theo

Sau khi hoàn tất bước này, hệ thống có thể tiếp tục kiểm thử các nghiệp vụ như CRUD dữ liệu, chuyển khoản liên chi nhánh, xử lý đồng thời và rollback giao dịch.

4.6.2 Kiểm thử chức năng gửi tiền(deposit)

1. Mục tiêu kiểm thử

- Đảm bảo tính năng nạp tiền hoạt động chính xác tại một site cụ thể (Local Transaction).
- Chứng minh khả năng bảo toàn dữ liệu (Tính chất ACID) của hệ thống thông qua hai kịch bản: Giao dịch thành công trọn vẹn và Giao dịch bị lỗi ngang chừng (Rollback do Server Crash).

2. Chuẩn bị kịch bản chung

Tiến hành nạp 500.000 VNĐ vào tài khoản có ID 1 thuộc chi nhánh Hà Nội (HN). Số dư ban đầu (giả định) đang là 45.000.000 VNĐ.

Payload gửi lên **API POST /api/transactions/deposit:**

```
JSON

{
  "accountId": 1,
  "branchId": "HN",
  "amount": 500000
}
```

Hình 4.18 Payload gửi lên API POST /api/transactions/deposit

Trường hợp 1: Giao dịch chạy bình thường (Tắt cờ mô phỏng lỗi)

Trong trường hợp này, đoạn mã mô phỏng sự cố trong AccountService.java được đóng comment (tắt đi). Hệ thống sẽ thực thi trọn vẹn các bước.

Thực thi: Dùng công cụ lệnh curl gọi API Backend.

```bash
curl -s -X POST http://localhost:8080/api/transactions/deposit \
  -H "Content-Type: application/json" \
  -d '{"accountId": 1, "branchId": "HN", "amount": 500000}'
```

Hình 4.19 Lệnh curl gọi API Backend - deposit

Kết quả trả về: Backend thực hiện cập nhật lệnh UPDATE account và INSERT transaction_history, sau đó gọi txManager.commit(txStatus). Client nhận mã trạng thái HTTP 200 OK:

```json
{
    "success": true,
    "message": "Deposit successful",
    "data": {
        "accountId": 1,
        "branchId": "HN",
        "balance": 45500000.0,
        "status": "ACTIVE"
    }
}
```

Hình 4.20 Kết quả trả về - deposit

Các bước thực thi trên giao diện và trên CSDL:

B1. CSDL trước khi thực hiện

- Bảng account

account_id	customer_id	branch_id	balance	status	created_at
1	1	HN	45000000.00	ACTIVE	2026-05-28 06:41:02
2	1	HN	20000000.00	ACTIVE	2026-05-28 06:41:02
3	2	HN	75000000.00	ACTIVE	2026-05-28 06:41:02
4	3	HN	30000000.00	ACTIVE	2026-05-28 06:41:02
5	4	HN	100000000.00	ACTIVE	2026-05-28 06:41:02
6	5	HN	15000000.00	ACTIVE	2026-05-28 06:41:02
NULL	NULL	NULL	NULL	NULL	NULL

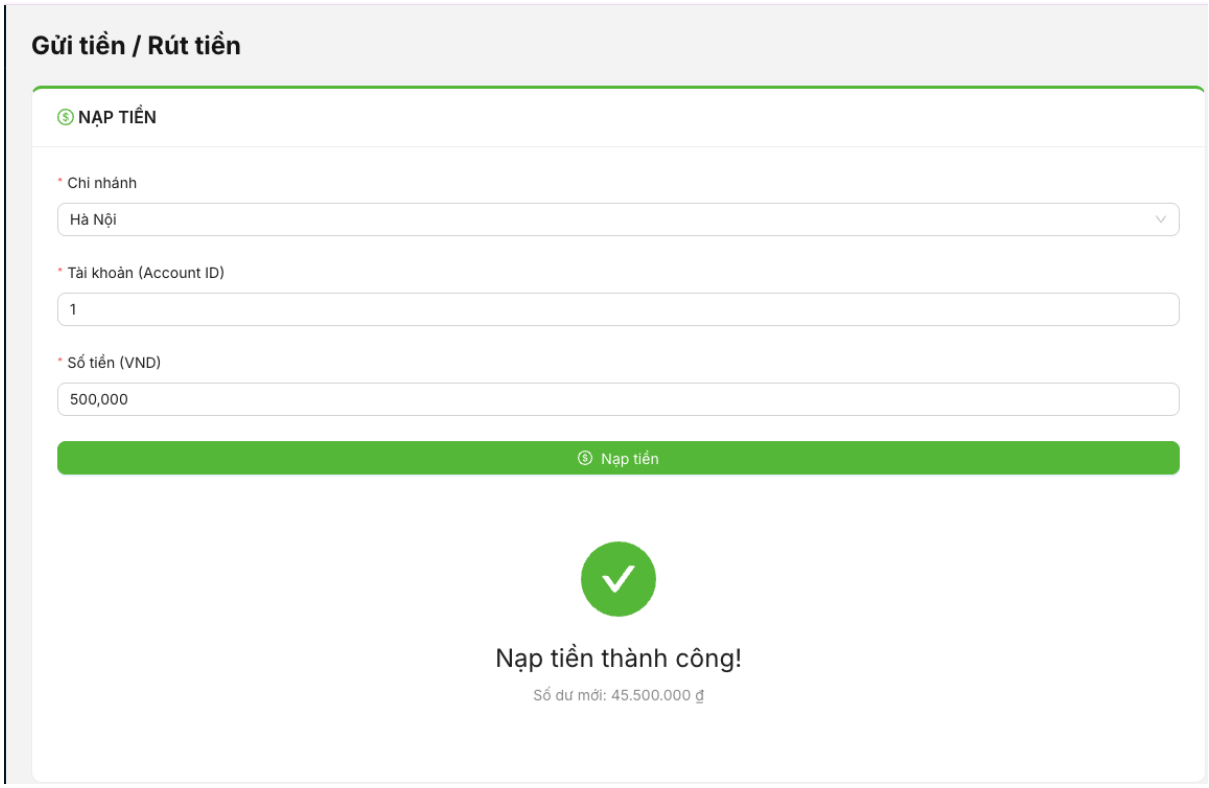
Hình 4.21 Bảng account trước khi thực hiện deposit

- Bảng transaction_history

transaction...	transaction_type	amount	account_id	related_account...	related_branch...	balance_after	status	distributed_txn_id	description	created_at
1	DEPOSIT	50000000.00	1	NULL	NULL	50000000.00	SUCCESS	NULL	Nạp tiền ban đầu	2026-05-28 06:41:02
2	DEPOSIT	20000000.00	2	NULL	NULL	20000000.00	SUCCESS	NULL	Nạp tiền ban đầu	2026-05-28 06:41:02
3	DEPOSIT	75000000.00	3	NULL	NULL	75000000.00	SUCCESS	NULL	Nạp tiền ban đầu	2026-05-28 06:41:02
4	DEPOSIT	30000000.00	4	NULL	NULL	30000000.00	SUCCESS	NULL	Nạp tiền ban đầu	2026-05-28 06:41:02
5	DEPOSIT	100000000.00	5	NULL	NULL	100000000.00	SUCCESS	NULL	Nạp tiền ban đầu	2026-05-28 06:41:02
6	DEPOSIT	15000000.00	6	NULL	NULL	15000000.00	SUCCESS	NULL	Nạp tiền ban đầu	2026-05-28 06:41:02
7	DEPOSIT	1000000.00	1	NULL	NULL	51000000.00	SUCCESS	NULL	Nạp tiền	2026-05-28 06:45:38
8	WITHDRAW	1000000.00	1	NULL	NULL	50000000.00	SUCCESS	NULL	Rút tiền	2026-05-28 06:53:24
9	INTER_BRANCH_OUT	5000000.00	1	1	HCM	45000000.00	SUCCESS	TXN-1779951226414	Inter branch transfer out	2026-05-28 06:53:46
10	INTER_BRANCH_OUT	5000000.00	1	1	HCM	45000000.00	FAILED	TXN-1779951230396	Inter branch transfer FAILED - HCM server bị cr...	2026-05-28 06:53:50
11	DEPOSIT	500000.00	1	NULL	NULL	45500000.00	SUCCESS	NULL	Nạp tiền	2026-05-28 10:52:01
12	WITHDRAW	500000.00	1	NULL	NULL	45000000.00	SUCCESS	NULL	Rút tiền	2026-05-28 10:58:46
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Hình 4.22 Bảng transaction_history trước khi thực hiện deposit

B2. Thao tác trên giao diện



Hình 4.23 Thao tác chức năng gửi tiền - deposit

B3. CSDL sau khi thao tác

- Bảng account

account_id	customer_id	branch_id	balance	status	created_at
1	1	HN	45500000.00	ACTIVE	2026-05-28 06:41:02
2	1	HN	20000000.00	ACTIVE	2026-05-28 06:41:02
3	2	HN	75000000.00	ACTIVE	2026-05-28 06:41:02
4	3	HN	30000000.00	ACTIVE	2026-05-28 06:41:02
5	4	HN	100000000.00	ACTIVE	2026-05-28 06:41:02
6	5	HN	15000000.00	ACTIVE	2026-05-28 06:41:02
NULL	NULL	NULL	NULL	NULL	NULL

Hình 4.24 Bảng account sau khi thực hiện deposit

- Bảng transaction_history

transaction...	transaction_type	amount	account_id	related_account...	related_branch...	balance_after	status	distributed_txn_id	description	created_at
1	DEPOSIT	50000000.00	1	NULL	NULL	50000000.00	SUCCESS	NULL	Nạp tiền ban đầu	2026-05-28 06:41:02
2	DEPOSIT	20000000.00	2	NULL	NULL	20000000.00	SUCCESS	NULL	Nạp tiền ban đầu	2026-05-28 06:41:02
3	DEPOSIT	75000000.00	3	NULL	NULL	75000000.00	SUCCESS	NULL	Nạp tiền ban đầu	2026-05-28 06:41:02
4	DEPOSIT	30000000.00	4	NULL	NULL	30000000.00	SUCCESS	NULL	Nạp tiền ban đầu	2026-05-28 06:41:02
5	DEPOSIT	100000000.00	5	NULL	NULL	100000000.00	SUCCESS	NULL	Nạp tiền ban đầu	2026-05-28 06:41:02
6	DEPOSIT	15000000.00	6	NULL	NULL	15000000.00	SUCCESS	NULL	Nạp tiền ban đầu	2026-05-28 06:41:02
7	DEPOSIT	1000000.00	1	NULL	NULL	51000000.00	SUCCESS	NULL	Nạp tiền	2026-05-28 06:45:38
8	WITHDRAW	1000000.00	1	NULL	NULL	50000000.00	SUCCESS	NULL	Rút tiền	2026-05-28 06:53:24
9	INTER_BRANCH_OUT	5000000.00	1	1	HCM	45000000.00	SUCCESS	TXN-1779951226414	Inter branch transfer out	2026-05-28 06:53:46
10	INTER_BRANCH_OUT	5000000.00	1	1	HCM	45000000.00	FAILED	TXN-1779951230396	Inter branch transfer FAILED - HCM server bị cr...	2026-05-28 06:53:50
11	DEPOSIT	500000.00	1	NULL	NULL	45500000.00	SUCCESS	NULL	Nạp tiền	2026-05-28 10:52:01
12	WITHDRAW	500000.00	1	NULL	NULL	45000000.00	SUCCESS	NULL	Rút tiền	2026-05-28 10:58:46
13	DEPOSIT	500000.00	1	NULL	NULL	45500000.00	SUCCESS	NULL	Nạp tiền	2026-05-28 11:02:25
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Hình 4.25 Bảng transaciton_history sau khi thực hiện deposit

Xác minh CSDL: Số dư đã tăng lên 45.500.000 VNĐ và bảng transaction_history xuất hiện bản ghi mới báo trạng thái SUCCESS.

Trường hợp 2: Mô phỏng sự cố Server Crash trước khi Commit

Để kiểm chứng tính an toàn của CSDL, hệ thống được can thiệp bằng cách mở comment (bật lên) đoạn mã sau trong hàm deposit() (file AccountService.java):

```
System.out.println();
System.out.println("SERVER CRASH BEFORE COMMIT!");

Thread.sleep(3000);

throw new RuntimeException("SERVER CRASH");
```

Hình 4.26 Đoạn code Mô phỏng sự cố Server Crash trước khi Commit

Đoạn code trên được đặt cố tình ở khe hở thời gian: Ngay sau khi đã chạy lệnh UPDATE trừ tiền và INSERT lịch sử, nhưng trước khi gọi lệnh txManager.commit().

- Thực thi: Gửi lại y hệt Payload nạp 500.000 VNĐ như trên thông qua curl.
- Diễn biến hệ thống:
    1. Hệ thống đã tiến hành tính toán và cập nhật số dư thành công trong bộ đệm (Memory) của MySQL.
    2. Bất ngờ lệnh throw new RuntimeException("SERVER CRASH"); bị kích hoạt, mô phỏng việc mất điện hoặc tiến trình bị giết chết.
    3. Ngoại lệ văng thẳng vào khối catch (Exception e).
    4. Lệnh giải cứu txManager.rollback(txStatus); lập tức được gọi ra.

Kết quả trả về cho Client:

```
JSON

{
    "success": false,
    "message": "SERVER CRASH",
    "data": null
}
```

Hình 4.27 Kết quả trả về khi server bị crash

Xác minh CSDL (Kiểm chứng tính toàn vẹn): Kiểm tra lại số dư tài khoản ID 1 và truy vấn toàn bộ lịch sử giao dịch

Các bước thực thi trên giao diện và trên CSDL:

B1. CSDL trước khi thực hiện

- Bảng account

account_id	customer_id	branch_id	balance	status	created_at
1	1	HN	45500000.00	ACTIVE	2026-05-28 06:41:02
2	1	HN	20000000.00	ACTIVE	2026-05-28 06:41:02
3	2	HN	75000000.00	ACTIVE	2026-05-28 06:41:02
4	3	HN	30000000.00	ACTIVE	2026-05-28 06:41:02
5	4	HN	100000000.00	ACTIVE	2026-05-28 06:41:02
6	5	HN	15000000.00	ACTIVE	2026-05-28 06:41:02
NULL	NULL	NULL	NULL	NULL	NULL

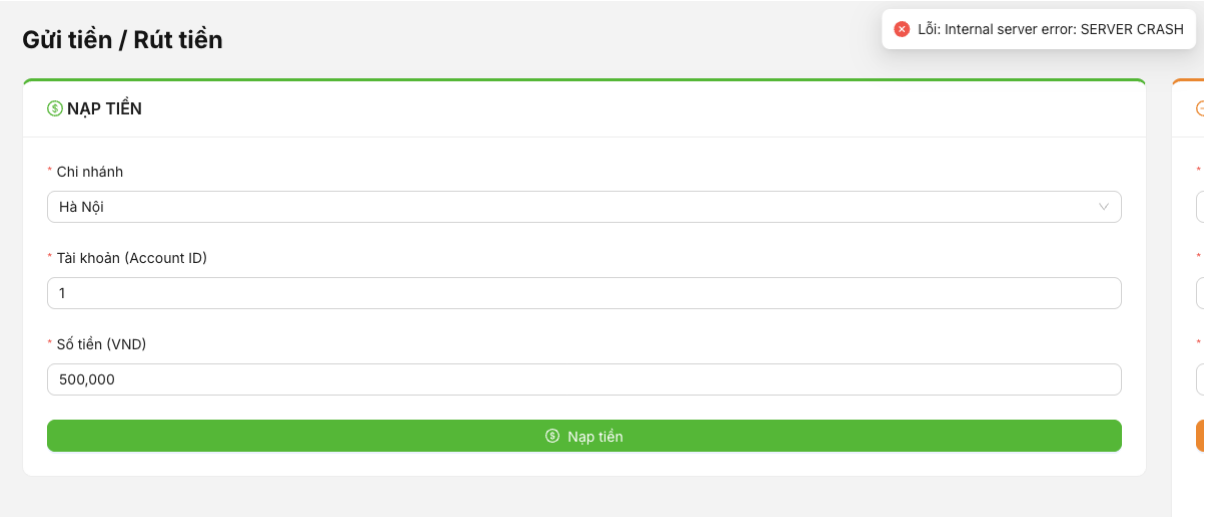
Hình 4.28 Bảng account trước khi deposit (server crassh)

- Bảng transaction_history

transaction...	transaction_type	amount	account_id	related_account...	related_branch...	balance_after	status	distributed_txn_id	description	created_at
1	DEPOSIT	50000000.00	1	NULL	NULL	50000000.00	SUCCESS	NULL	Nạp tiền ban đầu	2026-05-28 06:41:02
2	DEPOSIT	20000000.00	2	NULL	NULL	20000000.00	SUCCESS	NULL	Nạp tiền ban đầu	2026-05-28 06:41:02
3	DEPOSIT	75000000.00	3	NULL	NULL	75000000.00	SUCCESS	NULL	Nạp tiền ban đầu	2026-05-28 06:41:02
4	DEPOSIT	30000000.00	4	NULL	NULL	30000000.00	SUCCESS	NULL	Nạp tiền ban đầu	2026-05-28 06:41:02
5	DEPOSIT	100000000.00	5	NULL	NULL	100000000.00	SUCCESS	NULL	Nạp tiền ban đầu	2026-05-28 06:41:02
6	DEPOSIT	15000000.00	6	NULL	NULL	15000000.00	SUCCESS	NULL	Nạp tiền ban đầu	2026-05-28 06:41:02
7	DEPOSIT	1000000.00	1	NULL	NULL	51000000.00	SUCCESS	NULL	Nạp tiền	2026-05-28 06:45:38
8	WITHDRAW	1000000.00	1	NULL	NULL	50000000.00	SUCCESS	NULL	Rút tiền	2026-05-28 06:53:24
9	INTER_BRANCH_OUT	5000000.00	1	1	HCM	45000000.00	SUCCESS	TXN-1779951226414	Inter branch transfer out	2026-05-28 06:53:46
10	INTER_BRANCH_OUT	5000000.00	1	1	HCM	45000000.00	FAILED	TXN-1779951230396	Inter branch transfer FAILED - HCM server bị cr...	2026-05-28 06:53:50
11	DEPOSIT	500000.00	1	NULL	NULL	45500000.00	SUCCESS	NULL	Nạp tiền	2026-05-28 10:52:01
12	WITHDRAW	500000.00	1	NULL	NULL	45000000.00	SUCCESS	NULL	Rút tiền	2026-05-28 10:58:46
13	DEPOSIT	500000.00	1	NULL	NULL	45500000.00	SUCCESS	NULL	Nạp tiền	2026-05-28 11:02:25
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Hình 4.29 Bảng transaction_history trước khi deposit (server crash)

B2. Thao tác trên giao diện



Hình 4.30 Thao tác deposit - server crash

B3. CSDL sau khi thao tác

- Bảng account

account_id	customer_id	branch_id	balance	status	created_at
1	1	HN	45500000.00	ACTIVE	2026-05-28 06:41:02
2	1	HN	20000000.00	ACTIVE	2026-05-28 06:41:02
3	2	HN	75000000.00	ACTIVE	2026-05-28 06:41:02
4	3	HN	30000000.00	ACTIVE	2026-05-28 06:41:02
5	4	HN	100000000.00	ACTIVE	2026-05-28 06:41:02
6	5	HN	15000000.00	ACTIVE	2026-05-28 06:41:02
NULL	NULL	NULL	NULL	NULL	NULL

Hình 4.31 Bảng account sau khi deposit - server crash

- Bảng transaction_history

transaction...	transaction_type	amount	account_id	related_account...	related_branch...	balance_after	status	distributed_txn_id	description	created_at
1	DEPOSIT	50000000.00	1	NULL	NULL	50000000.00	SUCCESS	NULL	Nap tiền ban đầu	2026-05-28 06:41:02
2	DEPOSIT	20000000.00	2	NULL	NULL	20000000.00	SUCCESS	NULL	Nap tiền ban đầu	2026-05-28 06:41:02
3	DEPOSIT	75000000.00	3	NULL	NULL	75000000.00	SUCCESS	NULL	Nap tiền ban đầu	2026-05-28 06:41:02
4	DEPOSIT	30000000.00	4	NULL	NULL	30000000.00	SUCCESS	NULL	Nap tiền ban đầu	2026-05-28 06:41:02
5	DEPOSIT	100000000.00	5	NULL	NULL	100000000.00	SUCCESS	NULL	Nap tiền ban đầu	2026-05-28 06:41:02
6	DEPOSIT	15000000.00	6	NULL	NULL	15000000.00	SUCCESS	NULL	Nap tiền ban đầu	2026-05-28 06:41:02
7	DEPOSIT	1000000.00	1	NULL	NULL	51000000.00	SUCCESS	NULL	Nap tiền	2026-05-28 06:45:38
8	WITHDRAW	1000000.00	1	NULL	NULL	50000000.00	SUCCESS	NULL	Rút tiền	2026-05-28 06:53:24
9	INTER_BRANCH_OUT	5000000.00	1	1	HCM	45000000.00	SUCCESS	TXN-1779951226414	Inter branch transfer out	2026-05-28 06:53:46
10	INTER_BRANCH_OUT	5000000.00	1	1	HCM	45000000.00	FAILED	TXN-1779951230396	Inter branch transfer FAILED - HCM server bị cr...	2026-05-28 06:53:50
11	DEPOSIT	500000.00	1	NULL	NULL	45500000.00	SUCCESS	NULL	Nap tiền	2026-05-28 10:52:01
12	WITHDRAW	500000.00	1	NULL	NULL	45000000.00	SUCCESS	NULL	Rút tiền	2026-05-28 10:58:46
13	DEPOSIT	500000.00	1	NULL	NULL	45500000.00	SUCCESS	NULL	Nap tiền	2026-05-28 11:02:25
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Hình 4.32 Bảng transaction_history sau khi deposit - server crash

Kết quả in ra trên terminal:

```
DEPOSIT TRANSACTION

BEFORE DEPOSIT
Balance = 45500000.00

AFTER UPDATE (CHƯA COMMIT)
Deposit Amount = 500000
Balance = 46000000.00

TRANSACTION HISTORY INSERTED

SERVER CRASH BEFORE COMMIT!

ROLLBACK TRANSACTION

AFTER ROLLBACK
Balance = 45500000.00
```

Hình 4.33 Kết quả thực thi deposit từng bước khi server crash

Kết quả cho thấy số dư vẫn giữ nguyên ở mức cũ (45.500.000 VNĐ) và không hề có bất kỳ bản ghi lịch sử nạp tiền rác nào được chèn vào.

3. Kết luận

Trải qua 2 bài test, hệ thống chứng minh được **tính nguyên thủy (Atomicity)**. Tất cả thao tác dữ liệu được bọc chung trong một Transaction. Nếu thành công thì mọi thao tác cùng được lưu vĩnh viễn (Commit). Nếu xảy ra bất kỳ sự cố đứt gãy nào ở giữa chừng, toàn bộ thay đổi tạm thời sẽ bị hủy bỏ không dấu vết (Rollback), bảo đảm an toàn tuyệt đối cho tiền của khách hàng trong hệ thống phân tán.

4.6.3 Kiểm thử chức năng rút tiền và Xử lý tương tranh (Concurrency)

Tính năng rút tiền là một trong những nghiệp vụ nhạy cảm và quan trọng nhất của hệ thống ngân hàng. Đặc biệt, hệ thống phải đối mặt với bài toán xử lý tương tranh (Concurrency) khi có nhiều luồng (Threads) cùng truy cập và thực hiện thao tác rút tiền từ một tài khoản tại cùng một thời điểm.

1. Kiểm thử rút tiền cơ bản (Single Thread)

- Kịch bản: Tiến hành rút 500.000 VNĐ từ tài khoản ID 1 (Chi nhánh HN) đang có số dư 45.500.000 VNĐ.
- Thực thi:

```bash
curl -s -X POST http://localhost:8080/api/transactions/withdraw \
  -H "Content-Type: application/json" \
  -d '{"accountId": 1, "branchId": "HN", "amount": 500000}'
```

- Kết quả: Hệ thống trừ tiền thành công, số dư cập nhật về 45.000.000 VNĐ. Bản ghi lịch sử giao dịch mang trạng thái WITHDRAW được thêm tự động vào bảng transaction_history.

Các bước thực thi trên giao diện và trên CSDL:

B1. CSDL trước khi thực hiện

- Bảng account

account_id	customer_id	branch_id	balance	status	created_at
1	1	HN	45500000.00	ACTIVE	2026-05-28 06:41:02
2	1	HN	20000000.00	ACTIVE	2026-05-28 06:41:02
3	2	HN	75000000.00	ACTIVE	2026-05-28 06:41:02
4	3	HN	30000000.00	ACTIVE	2026-05-28 06:41:02
5	4	HN	100000000.00	ACTIVE	2026-05-28 06:41:02
6	5	HN	15000000.00	ACTIVE	2026-05-28 06:41:02
NULL	NULL	NULL	NULL	NULL	NULL

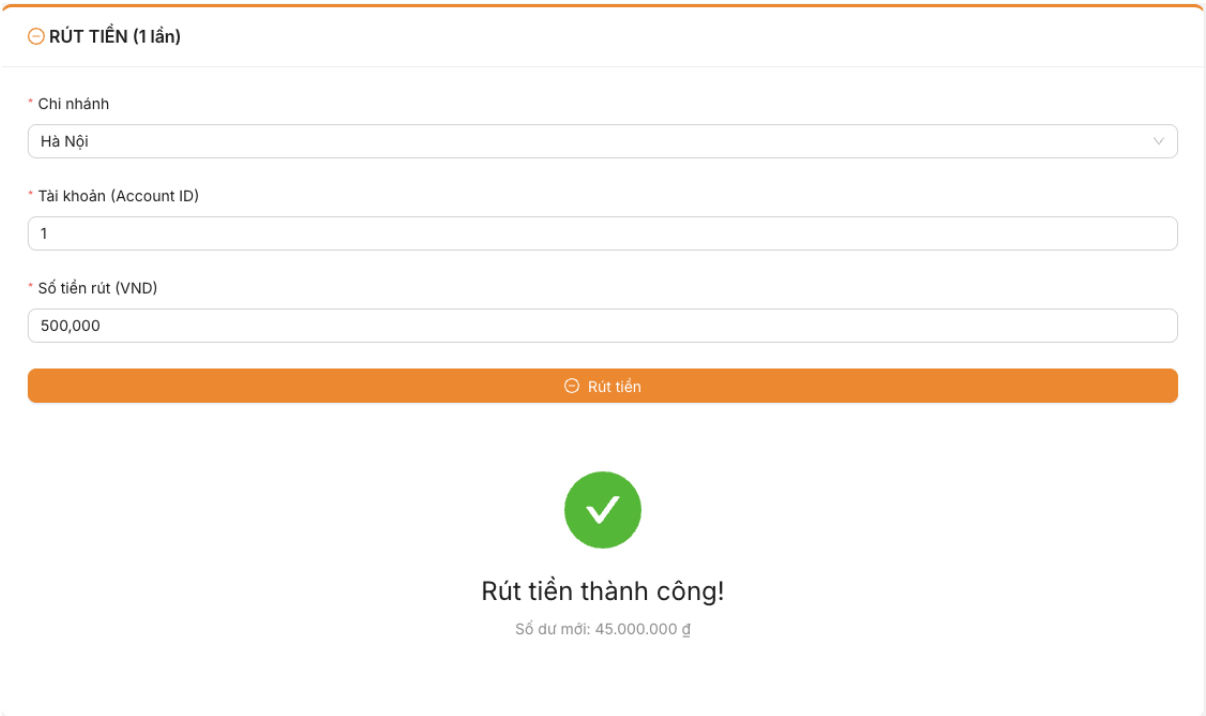
Hình 4.34 Bảng account trước khi thực hiện withdraw

- Bảng transaction_history

transaction...	transaction_type	amount	account_id	related_account...	related_branch...	balance_after	status	distributed_txn_id	description	created_at
1	DEPOSIT	50000000.00	1	NULL	NULL	50000000.00	SUCCESS	NULL	Nạp tiền ban đầu	2026-05-28 06:41:02
2	DEPOSIT	20000000.00	2	NULL	NULL	20000000.00	SUCCESS	NULL	Nạp tiền ban đầu	2026-05-28 06:41:02
3	DEPOSIT	75000000.00	3	NULL	NULL	75000000.00	SUCCESS	NULL	Nạp tiền ban đầu	2026-05-28 06:41:02
4	DEPOSIT	30000000.00	4	NULL	NULL	30000000.00	SUCCESS	NULL	Nạp tiền ban đầu	2026-05-28 06:41:02
5	DEPOSIT	100000000.00	5	NULL	NULL	100000000.00	SUCCESS	NULL	Nạp tiền ban đầu	2026-05-28 06:41:02
6	DEPOSIT	15000000.00	6	NULL	NULL	15000000.00	SUCCESS	NULL	Nạp tiền ban đầu	2026-05-28 06:41:02
7	DEPOSIT	1000000.00	1	NULL	NULL	51000000.00	SUCCESS	NULL	Nạp tiền	2026-05-28 06:45:38
8	WITHDRAW	1000000.00	1	NULL	NULL	50000000.00	SUCCESS	NULL	Rút tiền	2026-05-28 06:53:24
9	INTER_BRANCH_OUT	5000000.00	1	1	HCM	45000000.00	SUCCESS	TXN-1779951226414	Inter branch transfer out	2026-05-28 06:53:46
10	INTER_BRANCH_OUT	5000000.00	1	1	HCM	45000000.00	FAILED	TXN-1779951230396	Inter branch transfer FAILED - HCM server bị cr...	2026-05-28 06:53:50
11	DEPOSIT	500000.00	1	NULL	NULL	45500000.00	SUCCESS	NULL	Nạp tiền	2026-05-28 10:52:01
12	WITHDRAW	500000.00	1	NULL	NULL	45000000.00	SUCCESS	NULL	Rút tiền	2026-05-28 10:58:46
13	DEPOSIT	500000.00	1	NULL	NULL	45500000.00	SUCCESS	NULL	Nạp tiền	2026-05-28 11:02:25
20	WITHDRAW	500000.00	1	NULL	NULL	45000000.00	SUCCESS	NULL	Rút tiền	2026-05-28 11:41:45
21	WITHDRAW	10000000.00	1	NULL	NULL	35000000.00	SUCCESS	NULL	Rút tiền concurrent không lock	2026-05-28 11:42:22
22	WITHDRAW	10000000.00	1	NULL	NULL	35000000.00	SUCCESS	NULL	Rút tiền concurrent không lock	2026-05-28 11:42:22
23	WITHDRAW	10000000.00	1	NULL	NULL	25000000.00	SUCCESS	NULL	Rút tiền concurrent có lock	2026-05-28 11:43:11
24	WITHDRAW	10000000.00	1	NULL	NULL	15000000.00	SUCCESS	NULL	Rút tiền concurrent có lock	2026-05-28 11:43:11
27	DEPOSIT	30500000.00	1	NULL	NULL	45500000.00	SUCCESS	NULL	Nạp tiền	2026-05-28 12:07:24
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Hình 4.35 Bảng transaction_history sau trước khi thực hiện withdraw

B2. Thao tác trên giao diện



Hình 4.36 Thực hiện withdraw trên giao diện

B3. CSDL sau khi thao tác

- Bảng account

account_id	customer_id	branch_id	balance	status	created_at
1	1	HN	45000000.00	ACTIVE	2026-05-28 06:41:02
2	1	HN	20000000.00	ACTIVE	2026-05-28 06:41:02
3	2	HN	75000000.00	ACTIVE	2026-05-28 06:41:02
4	3	HN	30000000.00	ACTIVE	2026-05-28 06:41:02
5	4	HN	100000000.00	ACTIVE	2026-05-28 06:41:02
6	5	HN	15000000.00	ACTIVE	2026-05-28 06:41:02
NULL	NULL	NULL	NULL	NULL	NULL

Hình 4.37 Bảng account sau khi thực hiện withdraw

- Bảng transaction_history

transaction...	transaction_type	amount	account_id	related_account...	related_branch...	balance_after	status	distributed_txn_id	description	created_at
1	DEPOSIT	50000000.00	1	NULL	NULL	50000000.00	SUCCESS	NULL	Nạp tiền ban đầu	2026-05-28 06:41:02
2	DEPOSIT	20000000.00	2	NULL	NULL	20000000.00	SUCCESS	NULL	Nạp tiền ban đầu	2026-05-28 06:41:02
3	DEPOSIT	75000000.00	3	NULL	NULL	75000000.00	SUCCESS	NULL	Nạp tiền ban đầu	2026-05-28 06:41:02
4	DEPOSIT	30000000.00	4	NULL	NULL	30000000.00	SUCCESS	NULL	Nạp tiền ban đầu	2026-05-28 06:41:02
5	DEPOSIT	100000000.00	5	NULL	NULL	100000000.00	SUCCESS	NULL	Nạp tiền ban đầu	2026-05-28 06:41:02
6	DEPOSIT	15000000.00	6	NULL	NULL	15000000.00	SUCCESS	NULL	Nạp tiền ban đầu	2026-05-28 06:41:02
7	DEPOSIT	1000000.00	1	NULL	NULL	51000000.00	SUCCESS	NULL	Nạp tiền	2026-05-28 06:45:38
8	WITHDRAW	1000000.00	1	NULL	NULL	50000000.00	SUCCESS	NULL	Rút tiền	2026-05-28 06:53:24
9	INTER_BRANCH_OUT	5000000.00	1	1	HCM	45000000.00	SUCCESS	TXN-1779951226414	Inter branch transfer out	2026-05-28 06:53:46
10	INTER_BRANCH_OUT	5000000.00	1	1	HCM	45000000.00	FAILED	TXN-1779951230396	Inter branch transfer FAILED - HCM server bị cr...	2026-05-28 06:53:50
11	DEPOSIT	500000.00	1	NULL	NULL	45500000.00	SUCCESS	NULL	Nạp tiền	2026-05-28 10:52:01
12	WITHDRAW	500000.00	1	NULL	NULL	45000000.00	SUCCESS	NULL	Rút tiền	2026-05-28 10:58:46
13	DEPOSIT	500000.00	1	NULL	NULL	45500000.00	SUCCESS	NULL	Nạp tiền	2026-05-28 11:02:25
20	WITHDRAW	500000.00	1	NULL	NULL	45000000.00	SUCCESS	NULL	Rút tiền	2026-05-28 11:41:45
21	WITHDRAW	10000000.00	1	NULL	NULL	35000000.00	SUCCESS	NULL	Rút tiền concurrent không lock	2026-05-28 11:42:22
22	WITHDRAW	10000000.00	1	NULL	NULL	35000000.00	SUCCESS	NULL	Rút tiền concurrent không lock	2026-05-28 11:42:22
23	WITHDRAW	10000000.00	1	NULL	NULL	25000000.00	SUCCESS	NULL	Rút tiền concurrent có lock	2026-05-28 11:43:11
24	WITHDRAW	10000000.00	1	NULL	NULL	15000000.00	SUCCESS	NULL	Rút tiền concurrent có lock	2026-05-28 11:43:11
27	DEPOSIT	30500000.00	1	NULL	NULL	45500000.00	SUCCESS	NULL	Nạp tiền	2026-05-28 12:07:24
28	WITHDRAW	500000.00	1	NULL	NULL	45000000.00	SUCCESS	NULL	Rút tiền	2026-05-28 12:08:12

Hình 4.38 Bảng transaction_history sau khi thực hiện withdraw

Kết quả in ra trên terminal:

```
═══════════════════════════════════════
DEPOSIT TRANSACTION
═══════════════════════════════════════
BEFORE DEPOSIT
Balance = 15000000.00

AFTER UPDATE (CHƯA COMMIT)
Deposit Amount = 30500000
Balance = 45500000.00

TRANSACTION HISTORY INSERTED

COMMIT SUCCESS

AFTER COMMIT
Balance = 45500000.00
═══════════════════════════════════════

✅ [WITHDRAW] Account 1 at HN: -500000 → Balance = 45000000.00
```

Hình 4.39 Quá trình thực hiện rút tiền

2. Kiểm thử xử lý tương tranh: Trường hợp KHÔNG SỬ DỤNG Khóa (No Lock)

Để chứng minh sự nguy hiểm của việc thiết kế hệ thống thiếu an toàn, hệ thống cung cấp một API kiểm thử nhằm mô phỏng 2 tiến trình cùng lao vào rút tiền một lúc, trong đó cờ điều khiển useLock được đặt là false.

- Kịch bản: Số dư hiện tại dưới CSDL là 45.000.000 VNĐ. Hệ thống giả lập Luồng 1 (Thread 1) rút 10.000.000 VNĐ và Luồng 2 (Thread 2) rút 10.000.000 VNĐ

diễn ra hoàn toàn đồng thời. Nếu xử lý chuẩn xác, số dư cuối cùng phải là 25.000.000 VNĐ.

- Thực thi API mô phỏng:

```
curl -s -X POST http://localhost:8080/api/transactions/concurrent-withdraw \
  -H "Content-Type: application/json" \
  -d '{"accountId": 1, "branchId": "HN", "amountThread1": 10000000, "amountThread2": 10000000, "useLock": false}'
```

- Kết quả trả về:

```JSON
{
    "success": true,
    "message": "Concurrent withdrawal completed",
    "data": {
        "initialBalance": 45000000.0,
        "finalBalance": 35000000.0,
        "expectedBalance": 25000000.0,
        "lostUpdate": true,
        "logs": [
            "BẮT ĐẦU RÚT TIỀN ĐỒNG THỜI",
            "Chế độ = KHÔNG LOCK",
            "[T1] Đọc balance KHÔNG LOCK",
            "[T2] Đọc balance KHÔNG LOCK",
            "[T1] Balance đọc được = 45000000.00",
            "[T2] Balance đọc được = 45000000.00",
            "[T1] Balance sau update = 35000000.00",
            "[T2] Balance sau update = 35000000.00",
            "================================",
            "Số dư mong đợi = 25000000.00",
            "Số dư thực tế  = 35000000.00",
            "PHÁT HIỆN LOST UPDATE",
            "Một transaction đã bị ghi đè"
        ]
    }
}
```


Hình 4.40 Kết quả Json trả về rút tiền đồng thời không xử lý tương tranh

Các bước thực thi trên giao diện và trên CSDL:

B1. CSDL trước khi thực hiện

- Bảng account

account_id	customer_id	branch_id	balance	status	created_at
1	1	HN	45000000.00	ACTIVE	2026-05-28 06:41:02
2	1	HN	20000000.00	ACTIVE	2026-05-28 06:41:02
3	2	HN	75000000.00	ACTIVE	2026-05-28 06:41:02
4	3	HN	30000000.00	ACTIVE	2026-05-28 06:41:02
5	4	HN	100000000.00	ACTIVE	2026-05-28 06:41:02
6	5	HN	15000000.00	ACTIVE	2026-05-28 06:41:02
NULL	NULL	NULL	NULL	NULL	NULL

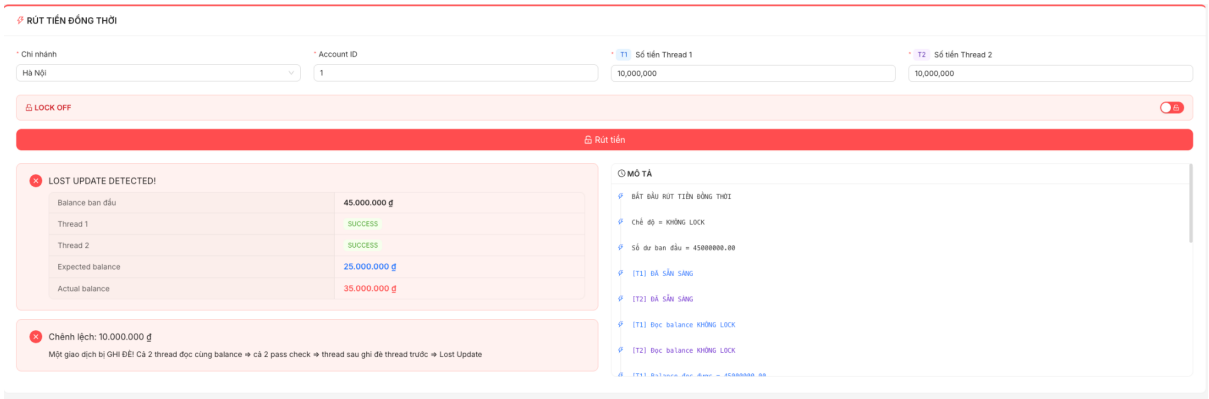
Hình 4.41 Bảng account trước khi rút tiền - không xử lý tương tranh

- Bảng transaction_history

transaction...	transaction_type	amount	account_id	related_account...	related_branch...	balance_after	status	distributed_txn_id	description	created_at
1	DEPOSIT	50000000.00	1	NULL	NULL	50000000.00	SUCCESS	NULL	Nạp tiền ban đầu	2026-05-28 06:41:02
2	DEPOSIT	20000000.00	2	NULL	NULL	20000000.00	SUCCESS	NULL	Nạp tiền ban đầu	2026-05-28 06:41:02
3	DEPOSIT	75000000.00	3	NULL	NULL	75000000.00	SUCCESS	NULL	Nạp tiền ban đầu	2026-05-28 06:41:02
4	DEPOSIT	30000000.00	4	NULL	NULL	30000000.00	SUCCESS	NULL	Nạp tiền ban đầu	2026-05-28 06:41:02
5	DEPOSIT	100000000.00	5	NULL	NULL	100000000.00	SUCCESS	NULL	Nạp tiền ban đầu	2026-05-28 06:41:02
6	DEPOSIT	15000000.00	6	NULL	NULL	15000000.00	SUCCESS	NULL	Nạp tiền ban đầu	2026-05-28 06:41:02
7	DEPOSIT	1000000.00	1	NULL	NULL	51000000.00	SUCCESS	NULL	Nạp tiền	2026-05-28 06:45:38
8	WITHDRAW	1000000.00	1	NULL	NULL	50000000.00	SUCCESS	NULL	Rút tiền	2026-05-28 06:53:24
9	INTER_BRANCH_OUT	5000000.00	1	1	HCM	45000000.00	SUCCESS	TXN-1779951226414	Inter branch transfer out	2026-05-28 06:53:46
10	INTER_BRANCH_OUT	5000000.00	1	1	HCM	45000000.00	FAILED	TXN-1779951230396	Inter branch transfer FAILED - HCM server bị cr...	2026-05-28 06:53:50
11	DEPOSIT	500000.00	1	NULL	NULL	45500000.00	SUCCESS	NULL	Nạp tiền	2026-05-28 10:52:01
12	WITHDRAW	500000.00	1	NULL	NULL	45000000.00	SUCCESS	NULL	Rút tiền	2026-05-28 10:58:46
13	DEPOSIT	500000.00	1	NULL	NULL	45500000.00	SUCCESS	NULL	Nạp tiền	2026-05-28 11:02:25
20	WITHDRAW	500000.00	1	NULL	NULL	45000000.00	SUCCESS	NULL	Rút tiền	2026-05-28 11:41:45
21	WITHDRAW	10000000.00	1	NULL	NULL	35000000.00	SUCCESS	NULL	Rút tiền concurrent không lock	2026-05-28 11:42:22
22	WITHDRAW	10000000.00	1	NULL	NULL	35000000.00	SUCCESS	NULL	Rút tiền concurrent không lock	2026-05-28 11:42:22
23	WITHDRAW	10000000.00	1	NULL	NULL	25000000.00	SUCCESS	NULL	Rút tiền concurrent có lock	2026-05-28 11:43:11
24	WITHDRAW	10000000.00	1	NULL	NULL	15000000.00	SUCCESS	NULL	Rút tiền concurrent có lock	2026-05-28 11:43:11
27	DEPOSIT	30500000.00	1	NULL	NULL	45500000.00	SUCCESS	NULL	Nạp tiền	2026-05-28 12:07:24
28	WITHDRAW	500000.00	1	NULL	NULL	45000000.00	SUCCESS	NULL	Rút tiền	2026-05-28 12:08:12

Hình 4.42 Bảng transaction_history trước khi thực hiện rút tiền đồng thời - không xử lý tương tranh

B2. Thao tác trên giao diện



Hình 4.43 Thực hiện rút tiền đồng thời - không xử lý tương tranh

B3. CSDL sau khi thao tác

- Bảng account

account_id	customer_id	branch_id	balance	status	created_at
1	1	HN	35000000.00	ACTIVE	2026-05-28 06:41:02
2	1	HN	20000000.00	ACTIVE	2026-05-28 06:41:02
3	2	HN	75000000.00	ACTIVE	2026-05-28 06:41:02
4	3	HN	30000000.00	ACTIVE	2026-05-28 06:41:02
5	4	HN	100000000.00	ACTIVE	2026-05-28 06:41:02
6	5	HN	15000000.00	ACTIVE	2026-05-28 06:41:02
NULL	NULL	NULL	NULL	NULL	NULL

Hình 4.44 Bảng account sau khi thực hiện rút tiền đồng thời - không xử lý trương tranh

- Bảng transaction_history

transaction...	transaction_type	amount	account_id	related_account...	related_branch...	balance_after	status	distributed_txn_id	description	created_at
1	DEPOSIT	50000000.00	1	NULL	NULL	50000000.00	SUCCESS	NULL	Nạp tiền ban đầu	2026-05-28 06:41:02
2	DEPOSIT	20000000.00	2	NULL	NULL	20000000.00	SUCCESS	NULL	Nạp tiền ban đầu	2026-05-28 06:41:02
3	DEPOSIT	75000000.00	3	NULL	NULL	75000000.00	SUCCESS	NULL	Nạp tiền ban đầu	2026-05-28 06:41:02
4	DEPOSIT	30000000.00	4	NULL	NULL	30000000.00	SUCCESS	NULL	Nạp tiền ban đầu	2026-05-28 06:41:02
5	DEPOSIT	100000000.00	5	NULL	NULL	100000000.00	SUCCESS	NULL	Nạp tiền ban đầu	2026-05-28 06:41:02
6	DEPOSIT	15000000.00	6	NULL	NULL	15000000.00	SUCCESS	NULL	Nạp tiền ban đầu	2026-05-28 06:41:02
7	DEPOSIT	1000000.00	1	NULL	NULL	51000000.00	SUCCESS	NULL	Nạp tiền	2026-05-28 06:45:38
8	WITHDRAW	1000000.00	1	NULL	NULL	50000000.00	SUCCESS	NULL	Rút tiền	2026-05-28 06:53:24
9	INTER_BRANCH_OUT	5000000.00	1	1	HCM	45000000.00	SUCCESS	TXN-1779951226414	Inter branch transfer out	2026-05-28 06:53:46
10	INTER_BRANCH_OUT	5000000.00	1	1	HCM	45000000.00	FAILED	TXN-1779951230396	Inter branch transfer FAILED - HCM server bị cr...	2026-05-28 06:53:50
11	DEPOSIT	500000.00	1	NULL	NULL	45500000.00	SUCCESS	NULL	Nạp tiền	2026-05-28 10:52:01
12	WITHDRAW	500000.00	1	NULL	NULL	45000000.00	SUCCESS	NULL	Rút tiền	2026-05-28 10:58:46
13	DEPOSIT	500000.00	1	NULL	NULL	45500000.00	SUCCESS	NULL	Nạp tiền	2026-05-28 11:02:25
20	WITHDRAW	500000.00	1	NULL	NULL	45000000.00	SUCCESS	NULL	Rút tiền	2026-05-28 11:41:45
21	WITHDRAW	10000000.00	1	NULL	NULL	35000000.00	SUCCESS	NULL	Rút tiền concurrent không lock	2026-05-28 11:42:22
22	WITHDRAW	10000000.00	1	NULL	NULL	35000000.00	SUCCESS	NULL	Rút tiền concurrent không lock	2026-05-28 11:42:22
23	WITHDRAW	10000000.00	1	NULL	NULL	25000000.00	SUCCESS	NULL	Rút tiền concurrent có lock	2026-05-28 11:43:11
24	WITHDRAW	10000000.00	1	NULL	NULL	15000000.00	SUCCESS	NULL	Rút tiền concurrent có lock	2026-05-28 11:43:11
27	DEPOSIT	30500000.00	1	NULL	NULL	45500000.00	SUCCESS	NULL	Nạp tiền	2026-05-28 12:07:24
28	WITHDRAW	500000.00	1	NULL	NULL	45000000.00	SUCCESS	NULL	Rút tiền	2026-05-28 12:08:12
29	WITHDRAW	10000000.00	1	NULL	NULL	35000000.00	SUCCESS	NULL	Rút tiền concurrent không lock	2026-05-28 12:10:06
30	WITHDRAW	10000000.00	1	NULL	NULL	35000000.00	SUCCESS	NULL	Rút tiền concurrent không lock	2026-05-28 12:10:06

Hình 4.45 Bảng transaction_history sau khi thực hiện rút tiền đồng thời - không xử lý tương tranh

Kết quả in ra trên terminal:

```
==========================================
MÔ PHỎNG RÚT TIỀN ĐỒNG THỜI
==========================================
Tài khoản          : 1
Chi nhánh          : HN
Chế độ             : KHÔNG LOCK
Số dư ban đầu      : 45000000.00
Thread 1 rút       : 10000000
Thread 2 rút       : 10000000
Tổng tiền rút      : 20000000
==========================================
[T1] Đọc balance KHÔNG LOCK
[T2] Đọc balance KHÔNG LOCK
[T1] Balance đọc được = 45000000.00
[T2] Balance đọc được = 45000000.00
[T2] Rút tiền thành công
[T1] Rút tiền thành công
[T1] Balance đọc được = 45000000.00
[T2] Balance đọc được = 45000000.00
[T2] Balance sau update = 35000000.00
[T1] Balance sau update = 35000000.00
------------------------------------------
Số dư mong đợi : 25000000.00
Số dư thực tế  : 35000000.00
PHÁT HIỆN LOST UPDATE
==========================================
```

Hình 4.46 Mô phỏng quá trình rút tiền đồng thời - không xử lý tương tranh

Đánh giá kỹ thuật: Dữ liệu hệ thống đã rơi vào trạng thái sai lệch nghiêm trọng do lỗi mất cập nhật (**Lost Update**). Do không có cơ chế chặn dòng, cả 2 luồng đều đọc chung số dư ban đầu là 45 triệu, cùng tính toán độc lập ra kết quả 35 triệu rồi lần lượt lưu xuống CSDL đè lên nhau. Hệ quả là khách hàng đã "rút trộm" thành công 10 triệu đồng mà CSDL không hề ghi nhận.

3. Kiểm thử xử lý tương tranh: Trường hợp SỬ DỤNG Khóa (Pessimistic Locking)

Để khắc phục triệt để lỗi Lost Update ở trên, mã nguồn ứng dụng đã được cấu hình bổ sung cơ chế khóa nghiêm ngặt thông qua lệnh tương tác dữ liệu SELECT ... FOR UPDATE. Hệ thống tiến hành lặp lại y hệt kịch bản tương tranh trên nhưng cờ useLock được bật thành true.

- **Kịch bản:** Số dư tài khoản lúc này đang ở mức 35.000.000 VNĐ sau lỗi của bài test trước. Hai Thread cùng quét vào hệ thống, mỗi bên thực hiện lệnh rút 10.000.000 VNĐ.
- **Thực thi API mô phỏng:**

- **curl -s -X POST http://localhost:8080/api/transactions/concurrent-withdraw **
- **-H "Content-Type: application/json" **
- **-d '{"accountId": 1, "branchId": "HN", "amountThread1": 10000000, "amountThread2": 10000000, "useLock": true}'**

- Kết quả trả về:


```JSON
{
    "success": true,
    "message": "Concurrent withdrawal completed",
    "data": {
        "initialBalance": 35000000.0,
        "finalBalance": 15000000.0,
        "expectedBalance": 15000000.0,
        "lostUpdate": false,
        "logs": [
            "BẮT ĐẦU RÚT TIỀN ĐỒNG THỜI",
            "Chế độ = CÓ LOCK",
            "[T2] Đang chờ lock...",
            "[T1] Đang chờ lock...",
            "[T2] Đã lock thành công",
            "[T2] Balance hiện tại = 35000000.00",
            "[T1] Đã lock thành công",
            "[T1] Balance hiện tại = 25000000.00",
            "[T2] Balance mới = 25000000.00",
            "[T1] Balance mới = 15000000.00",
            "================================",
            "Số dư mong đợi = 15000000.00",
            "Số dư thực tế  = 15000000.00",
            "Dữ liệu chính xác"
        ]
    }
}
```


Hình 4.47 Kết quả trả về khi thực hiện rút tiền đồng thời - SỬ DỤNG Khóa (Pessimistic Locking) dưới dạng JSON

Các bước thực thi trên giao diện và trên CSDL:

B1. CSDL trước khi thực hiện

- Bảng account

account_id	customer_id	branch_id	balance	status	created_at
1	1	HN	35000000.00	ACTIVE	2026-05-28 06:41:02
2	1	HN	20000000.00	ACTIVE	2026-05-28 06:41:02
3	2	HN	75000000.00	ACTIVE	2026-05-28 06:41:02
4	3	HN	30000000.00	ACTIVE	2026-05-28 06:41:02
5	4	HN	100000000.00	ACTIVE	2026-05-28 06:41:02
6	5	HN	15000000.00	ACTIVE	2026-05-28 06:41:02
NULL	NULL	NULL	NULL	NULL	NULL

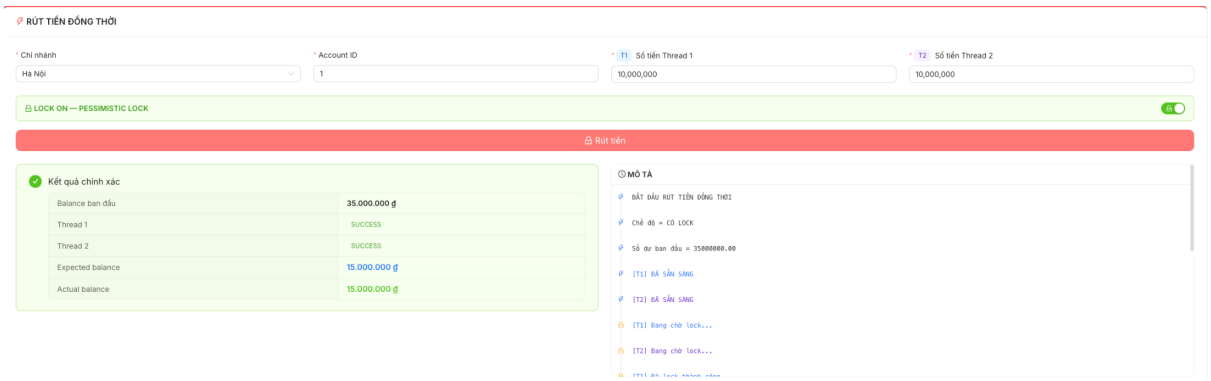
Hình 4.48 Bảng account trước khi thực hiện rút tiền đồng thời - sử dụng Pessimistic Lock

- Bảng transaction_history

transaction...	transaction_type	amount	account_id	related_account...	related_branch...	balance_after	status	distributed_txn_id	description	created_at
1	DEPOSIT	50000000.00	1	NULL	NULL	50000000.00	SUCCESS	NULL	Nạp tiền ban đầu	2026-05-28 06:41:02
2	DEPOSIT	20000000.00	2	NULL	NULL	20000000.00	SUCCESS	NULL	Nạp tiền ban đầu	2026-05-28 06:41:02
3	DEPOSIT	75000000.00	3	NULL	NULL	75000000.00	SUCCESS	NULL	Nạp tiền ban đầu	2026-05-28 06:41:02
4	DEPOSIT	30000000.00	4	NULL	NULL	30000000.00	SUCCESS	NULL	Nạp tiền ban đầu	2026-05-28 06:41:02
5	DEPOSIT	100000000.00	5	NULL	NULL	100000000.00	SUCCESS	NULL	Nạp tiền ban đầu	2026-05-28 06:41:02
6	DEPOSIT	15000000.00	6	NULL	NULL	15000000.00	SUCCESS	NULL	Nạp tiền ban đầu	2026-05-28 06:41:02
7	DEPOSIT	1000000.00	1	NULL	NULL	51000000.00	SUCCESS	NULL	Nạp tiền	2026-05-28 06:45:38
8	WITHDRAW	1000000.00	1	NULL	NULL	50000000.00	SUCCESS	NULL	Rút tiền	2026-05-28 06:53:24
9	INTER_BRANCH_OUT	5000000.00	1	1	HCM	45000000.00	SUCCESS	TXN-1779951226414	Inter branch transfer out	2026-05-28 06:53:46
10	INTER_BRANCH_OUT	5000000.00	1	1	HCM	45000000.00	FAILED	TXN-1779951230396	Inter branch transfer FAILED - HCM server bị cr...	2026-05-28 06:53:50
11	DEPOSIT	500000.00	1	NULL	NULL	45500000.00	SUCCESS	NULL	Nạp tiền	2026-05-28 10:52:01
12	WITHDRAW	500000.00	1	NULL	NULL	45000000.00	SUCCESS	NULL	Rút tiền	2026-05-28 10:58:46
13	DEPOSIT	500000.00	1	NULL	NULL	45500000.00	SUCCESS	NULL	Nạp tiền	2026-05-28 11:02:25
20	WITHDRAW	500000.00	1	NULL	NULL	45000000.00	SUCCESS	NULL	Rút tiền	2026-05-28 11:41:45
21	WITHDRAW	10000000.00	1	NULL	NULL	35000000.00	SUCCESS	NULL	Rút tiền concurrent không lock	2026-05-28 11:42:22
22	WITHDRAW	10000000.00	1	NULL	NULL	35000000.00	SUCCESS	NULL	Rút tiền concurrent không lock	2026-05-28 11:42:22
23	WITHDRAW	10000000.00	1	NULL	NULL	25000000.00	SUCCESS	NULL	Rút tiền concurrent có lock	2026-05-28 11:43:11
24	WITHDRAW	10000000.00	1	NULL	NULL	15000000.00	SUCCESS	NULL	Rút tiền concurrent có lock	2026-05-28 11:43:11
27	DEPOSIT	30500000.00	1	NULL	NULL	45500000.00	SUCCESS	NULL	Nạp tiền	2026-05-28 12:07:24
28	WITHDRAW	500000.00	1	NULL	NULL	45000000.00	SUCCESS	NULL	Rút tiền	2026-05-28 12:08:12
29	WITHDRAW	10000000.00	1	NULL	NULL	35000000.00	SUCCESS	NULL	Rút tiền concurrent không lock	2026-05-28 12:10:06
30	WITHDRAW	10000000.00	1	NULL	NULL	35000000.00	SUCCESS	NULL	Rút tiền concurrent không lock	2026-05-28 12:10:06

Hình 4.49 Bảng transaction_history trước khi thực hiện rút tiền đồng thời - sử dụng Pessimistic Lock

B2. Thao tác trên giao diện



Hình 4.50 Thực hiện rút tiền đồng thời - sử dụng Pessimistic Lock trên giao diện

B3. CSDL sau khi thao tác

- Bảng account

account_id	customer_id	branch_id	balance	status	created_at
1	1	HN	15000000.00	ACTIVE	2026-05-28 06:41:02
2	1	HN	20000000.00	ACTIVE	2026-05-28 06:41:02
3	2	HN	75000000.00	ACTIVE	2026-05-28 06:41:02
4	3	HN	30000000.00	ACTIVE	2026-05-28 06:41:02
5	4	HN	100000000.00	ACTIVE	2026-05-28 06:41:02
6	5	HN	15000000.00	ACTIVE	2026-05-28 06:41:02
NULL	NULL	NULL	NULL	NULL	NULL

Hình 4.51 Bảng account sau khi thực hiện rút tiền đồng thời - sử dụng Pessimistic LOCK

- Bảng transaction_history

transaction...	transaction_type	amount	account_id	related_account...	related_branch...	balance_after	status	distributed_txn_id	description	created_at
1	DEPOSIT	50000000.00	1	NULL	NULL	50000000.00	SUCCESS	NULL	Nạp tiền ban đầu	2026-05-28 06:41:02
2	DEPOSIT	20000000.00	2	NULL	NULL	20000000.00	SUCCESS	NULL	Nạp tiền ban đầu	2026-05-28 06:41:02
3	DEPOSIT	75000000.00	3	NULL	NULL	75000000.00	SUCCESS	NULL	Nạp tiền ban đầu	2026-05-28 06:41:02
4	DEPOSIT	30000000.00	4	NULL	NULL	30000000.00	SUCCESS	NULL	Nạp tiền ban đầu	2026-05-28 06:41:02
5	DEPOSIT	100000000.00	5	NULL	NULL	100000000.00	SUCCESS	NULL	Nạp tiền ban đầu	2026-05-28 06:41:02
6	DEPOSIT	15000000.00	6	NULL	NULL	15000000.00	SUCCESS	NULL	Nạp tiền ban đầu	2026-05-28 06:41:02
7	DEPOSIT	1000000.00	1	NULL	NULL	51000000.00	SUCCESS	NULL	Nạp tiền	2026-05-28 06:45:38
8	WITHDRAW	1000000.00	1	NULL	NULL	50000000.00	SUCCESS	NULL	Rút tiền	2026-05-28 06:53:24
9	INTER_BRANCH_OUT	5000000.00	1	1	HCM	45000000.00	SUCCESS	TXN-1779951226414	Inter branch transfer out	2026-05-28 06:53:46
10	INTER_BRANCH_OUT	5000000.00	1	1	HCM	45000000.00	FAILED	TXN-1779951230396	Inter branch transfer FAILED - HCM server bị cr...	2026-05-28 06:53:50
11	DEPOSIT	500000.00	1	NULL	NULL	45500000.00	SUCCESS	NULL	Nạp tiền	2026-05-28 10:52:01
12	WITHDRAW	500000.00	1	NULL	NULL	45000000.00	SUCCESS	NULL	Rút tiền	2026-05-28 10:58:46
13	DEPOSIT	500000.00	1	NULL	NULL	45500000.00	SUCCESS	NULL	Nạp tiền	2026-05-28 11:02:25
20	WITHDRAW	500000.00	1	NULL	NULL	45000000.00	SUCCESS	NULL	Rút tiền	2026-05-28 11:41:45
21	WITHDRAW	10000000.00	1	NULL	NULL	35000000.00	SUCCESS	NULL	Rút tiền concurrent không lock	2026-05-28 11:42:22
22	WITHDRAW	10000000.00	1	NULL	NULL	35000000.00	SUCCESS	NULL	Rút tiền concurrent không lock	2026-05-28 11:42:22
23	WITHDRAW	10000000.00	1	NULL	NULL	25000000.00	SUCCESS	NULL	Rút tiền concurrent có lock	2026-05-28 11:43:11
24	WITHDRAW	10000000.00	1	NULL	NULL	15000000.00	SUCCESS	NULL	Rút tiền concurrent có lock	2026-05-28 11:43:11
27	DEPOSIT	30500000.00	1	NULL	NULL	45500000.00	SUCCESS	NULL	Nạp tiền	2026-05-28 12:07:24
28	WITHDRAW	500000.00	1	NULL	NULL	45000000.00	SUCCESS	NULL	Rút tiền	2026-05-28 12:08:12
29	WITHDRAW	10000000.00	1	NULL	NULL	35000000.00	SUCCESS	NULL	Rút tiền concurrent không lock	2026-05-28 12:10:06
30	WITHDRAW	10000000.00	1	NULL	NULL	35000000.00	SUCCESS	NULL	Rút tiền concurrent không lock	2026-05-28 12:10:06
31	WITHDRAW	10000000.00	1	NULL	NULL	25000000.00	SUCCESS	NULL	Rút tiền concurrent có lock	2026-05-28 12:13:25
32	WITHDRAW	10000000.00	1	NULL	NULL	15000000.00	SUCCESS	NULL	Rút tiền concurrent có lock	2026-05-28 12:13:25
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Hình 4.52 Bảng transaction_history sau khi thực hiện rút tiền đồng thời - sử dụng Pessimistic Lock

Kết quả in ra trên terminal:

```
==========================================
MÔ PHỎNG RÚT TIỀN ĐỒNG THỜI
==========================================
Tài khoản          : 1
Chi nhánh          : HN
Chế độ             : CÓ LOCK
Số dư ban đầu      : 35000000.00
Thread 1 rút       : 10000000
Thread 2 rút       : 10000000
Tổng tiền rút      : 20000000
==========================================
[T1] Đang chờ lock...
[T2] Đang chờ lock...
[T1] Đã lock thành công
[T1] Balance hiện tại = 35000000.00
[T2] Đã lock thành công
[T2] Balance hiện tại = 25000000.00
[T1] Rút tiền thành công
[T1] Balance mới = 25000000.00
[T2] Rút tiền thành công
[T2] Balance mới = 15000000.00
------------------------------------------
Số dư mong đợi : 15000000.00
Số dư thực tế  : 15000000.00
DỮ LIỆU CHÍNH XÁC
==========================================
```

Hình 4.53 Mô phỏng các bước rút tiền đồng thời - sử dụng Pessimistic Lock

Đánh giá kỹ thuật: Hiện tượng lỗi Lost Update đã bị loại bỏ hoàn toàn. Bằng việc thực thi cơ chế khóa độc quyền ở tầng cơ sở dữ liệu (**Pessimistic Lock**), khi Thread 2 chiếm giữ khóa thành công thì Thread 1 ngay lập tức bị ép vào trạng thái chờ nhả khóa. Sau khi Thread 2 hoàn tất cập nhật số dư mới về mức 25 triệu và giải phóng lock, Thread 1 mới được phép đọc lại giá trị mới này để tiếp tục thực hiện phép trừ tiền của mình. Quy trình tuần tự hóa dữ liệu (Serialization) này đảm bảo số dư cuối cùng hoàn toàn trùng khớp với mong đợi, bảo toàn vẹn toàn **Tính độc lập (Isolation)** trong tiêu chuẩn ACID của hệ thống.

4.6.4 Kiểm thử chức năng chuyển tiền cùng chi nhánh (Internal Transfer)

Giao dịch chuyển tiền nội bộ là trường hợp đặc biệt khi cả tài khoản nguồn và tài khoản đích đều nằm trên cùng một chi nhánh (cùng chung một Site vật lý). Do đó, giao dịch này mang tính chất cục bộ (Local Transaction), chỉ yêu cầu thiết lập duy nhất một kết nối cơ sở dữ liệu (Database Connection) và thực thi trong một chu kỳ Transaction đơn lẻ mà không cần kích hoạt giao thức phân tán phức tạp 2PC.

1. Mục tiêu kiểm thử

- Đảm bảo hệ thống thực hiện trừ tiền tại tài khoản nguồn và cộng tiền tại tài khoản đích chính xác theo thời gian thực.
- Kiểm chứng cơ chế khóa theo thứ tự (Ordered Locking) hoạt động hiệu quả nhằm ngăn chặn triệt để hiện tượng xung đột khóa chéo (Deadlock).
- Đảm bảo hệ thống tự động sinh ra 2 bản ghi lịch sử giao dịch đối ứng chéo nhau (1 bản ghi gửi và 1 bản ghi nhận) trong bảng transaction_history.

2. Chuẩn bị kịch bản

- Tài khoản nguồn (From Account): ID 1 (Chi nhánh HN) — Số dư hiện tại dưới CSDL là 15.000.000 VNĐ.
- Tài khoản đích (To Account): ID 2 (Chi nhánh HN) — Số dư hiện tại dưới CSDL là 20.000.000 VNĐ.
- Yêu cầu nghiệp vụ: Thực hiện lệnh chuyển 5.000.000 VNĐ từ tài khoản ID 1 sang tài khoản ID 2.

Payload gửi lên API POST /api/transfers/internal:

```json
{
  "fromAccountId": 1,
  "toAccountId": 2,
  "branchId": "HN",
  "amount": 5000000
}
```

Hình 4.54 Payload gửi lên API POST /api/transfers/internal

3. Thực thi kiểm thử bằng API

Sử dụng công cụ dòng lệnh curl để phát HTTP Request trực tiếp tới máy chủ điều phối:

```bash
curl -s -X POST http://localhost:8080/api/transfers/internal \
  -H "Content-Type: application/json" \
  -d '{"fromAccountId": 1, "toAccountId": 2, "branchId": "HN", "amount": 5000000}'
```

4. Kết quả thực thi

Hệ thống xử lý nghiệp vụ thành công và phản hồi mã trạng thái HTTP 200 OK. Cấu trúc dữ liệu JSON Response chứng minh luồng dịch chuyển dòng tiền diễn ra hoàn hảo:

```json
{
    "success": true,
    "message": "Internal transfer completed",
    "data": {
        "status": "SUCCESS",
        "fromBranch": "HN",
        "fromAccountId": 1,
        "toBranch": "HN",
        "toAccountId": 2,
        "amount": 5000000,
        "sourceBalanceAfter": 10000000.0,
        "destBalanceAfter": 25000000.0,
        "message": "Chuyển tiền nội bộ thành công",
        "timestamp": "2026-05-28T19:16:32.256068"
    }
}
```

Hình 4.55 Kết quả trả về dạng JSON sau khi thực hiện chuyển tiền cùng chi nhánh

Đối soát trạng thái CSDL sau giao dịch:

- Số dư tài khoản nguồn ID 1 giảm trừ chính xác, còn 10.000.000 VNĐ.
- Số dư tài khoản đích ID 2 cộng tăng chính xác, lên mức 25.000.000 VNĐ.

5. Thực thi trên giao diện và trên CSDL

B1. CSDL trước khi thực hiện

- **Bảng account**

account_id	customer_id	branch_id	balance	status	created_at
1	1	HN	15000000.00	ACTIVE	2026-05-28 06:41:02
2	1	HN	20000000.00	ACTIVE	2026-05-28 06:41:02
3	2	HN	75000000.00	ACTIVE	2026-05-28 06:41:02
4	3	HN	30000000.00	ACTIVE	2026-05-28 06:41:02
5	4	HN	100000000.00	ACTIVE	2026-05-28 06:41:02
6	5	HN	15000000.00	ACTIVE	2026-05-28 06:41:02
NULL	NULL	NULL	NULL	NULL	NULL

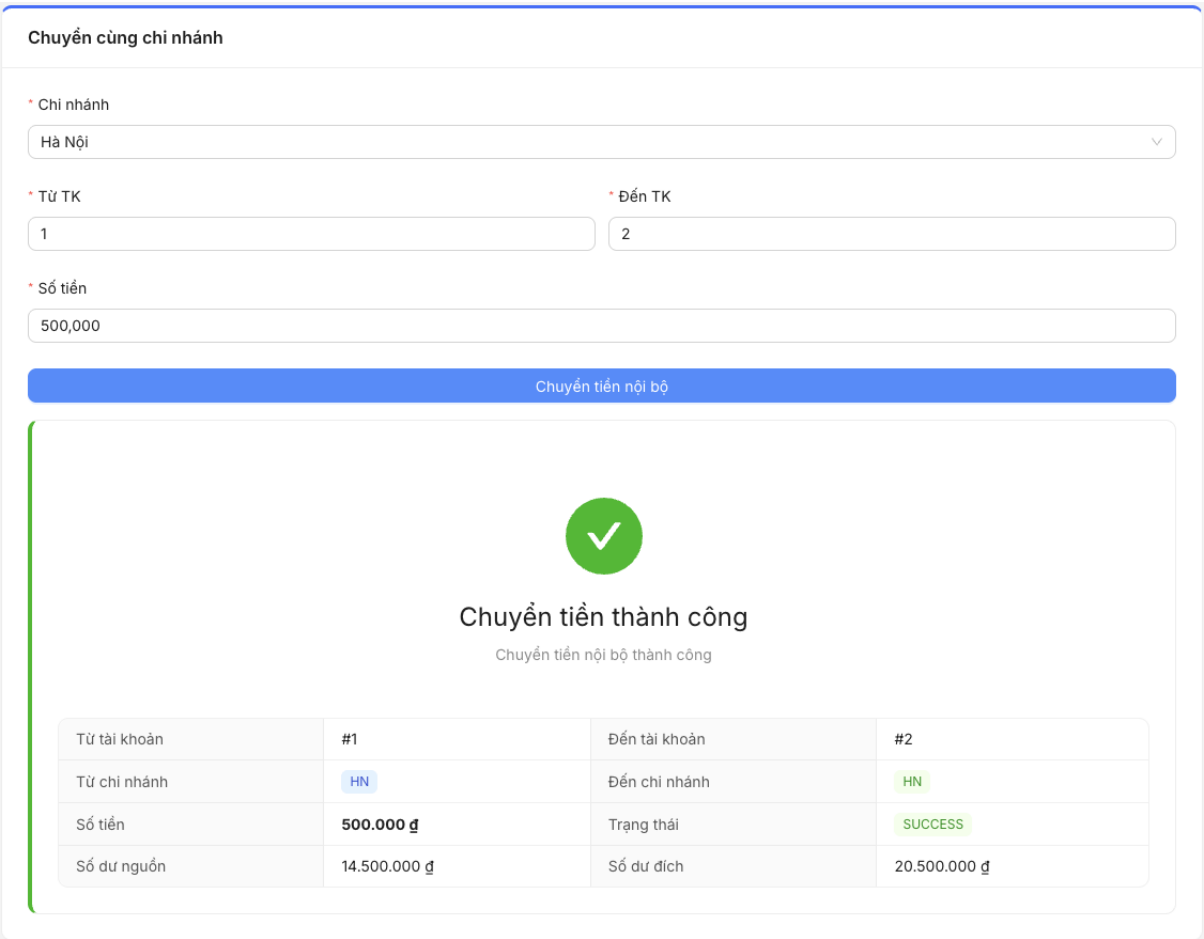
Hình 4.56 Bảng account trước khi thực hiện chuyển tiền cùng chi nhánh

- **Bảng transaction_history**

transaction...	transaction_type	amount	account_id	related_account...	related_branch...	balance_after	status	distributed_txn_id	description	created_at
1	DEPOSIT	50000000.00	1	NULL	NULL	50000000.00	SUCCESS	NULL	Nạp tiền ban đầu	2026-05-28 06:41:02
2	DEPOSIT	20000000.00	2	NULL	NULL	20000000.00	SUCCESS	NULL	Nạp tiền ban đầu	2026-05-28 06:41:02
3	DEPOSIT	75000000.00	3	NULL	NULL	75000000.00	SUCCESS	NULL	Nạp tiền ban đầu	2026-05-28 06:41:02
4	DEPOSIT	30000000.00	4	NULL	NULL	30000000.00	SUCCESS	NULL	Nạp tiền ban đầu	2026-05-28 06:41:02
5	DEPOSIT	100000000.00	5	NULL	NULL	100000000.00	SUCCESS	NULL	Nạp tiền ban đầu	2026-05-28 06:41:02
6	DEPOSIT	15000000.00	6	NULL	NULL	15000000.00	SUCCESS	NULL	Nạp tiền ban đầu	2026-05-28 06:41:02
7	DEPOSIT	1000000.00	1	NULL	NULL	51000000.00	SUCCESS	NULL	Nạp tiền	2026-05-28 06:45:38
8	WITHDRAW	1000000.00	1	NULL	NULL	50000000.00	SUCCESS	NULL	Rút tiền	2026-05-28 06:53:24
9	INTER_BRANCH_OUT	5000000.00	1	1	HCM	45000000.00	SUCCESS	TXN-1779951226414	Inter branch transfer out	2026-05-28 06:53:46
10	INTER_BRANCH_OUT	5000000.00	1	1	HCM	45000000.00	FAILED	TXN-1779951230396	Inter branch transfer FAILED - HCM server bị cr...	2026-05-28 06:53:50
11	DEPOSIT	500000.00	1	NULL	NULL	45500000.00	SUCCESS	NULL	Nạp tiền	2026-05-28 10:52:01
12	WITHDRAW	500000.00	1	NULL	NULL	45000000.00	SUCCESS	NULL	Rút tiền	2026-05-28 10:58:46
13	DEPOSIT	500000.00	1	NULL	NULL	45500000.00	SUCCESS	NULL	Nạp tiền	2026-05-28 11:02:25
20	WITHDRAW	500000.00	1	NULL	NULL	45000000.00	SUCCESS	NULL	Rút tiền	2026-05-28 11:41:45
21	WITHDRAW	10000000.00	1	NULL	NULL	35000000.00	SUCCESS	NULL	Rút tiền concurrent không lock	2026-05-28 11:42:22
22	WITHDRAW	10000000.00	1	NULL	NULL	35000000.00	SUCCESS	NULL	Rút tiền concurrent không lock	2026-05-28 11:42:22
23	WITHDRAW	10000000.00	1	NULL	NULL	25000000.00	SUCCESS	NULL	Rút tiền concurrent có lock	2026-05-28 11:43:11
24	WITHDRAW	10000000.00	1	NULL	NULL	15000000.00	SUCCESS	NULL	Rút tiền concurrent có lock	2026-05-28 11:43:11
27	DEPOSIT	30500000.00	1	NULL	NULL	45500000.00	SUCCESS	NULL	Nạp tiền	2026-05-28 12:07:24
28	WITHDRAW	500000.00	1	NULL	NULL	45000000.00	SUCCESS	NULL	Rút tiền	2026-05-28 12:08:12
29	WITHDRAW	10000000.00	1	NULL	NULL	35000000.00	SUCCESS	NULL	Rút tiền concurrent không lock	2026-05-28 12:10:06
30	WITHDRAW	10000000.00	1	NULL	NULL	35000000.00	SUCCESS	NULL	Rút tiền concurrent không lock	2026-05-28 12:10:06
31	WITHDRAW	10000000.00	1	NULL	NULL	25000000.00	SUCCESS	NULL	Rút tiền concurrent có lock	2026-05-28 12:13:25
32	WITHDRAW	10000000.00	1	NULL	NULL	15000000.00	SUCCESS	NULL	Rút tiền concurrent có lock	2026-05-28 12:13:25
33	TRANSFER_OUT	5000000.00	1	2	NULL	10000000.00	SUCCESS	NULL	Chuyển tiền nội bộ	2026-05-28 12:16:32
34	TRANSFER_IN	5000000.00	2	1	NULL	25000000.00	SUCCESS	NULL	Nhận tiền nội bộ	2026-05-28 12:16:32
35	TRANSFER_OUT	100000.00	1	2	NULL	9900000.00	SUCCESS	NULL	Chuyển tiền đồng thời [T1] không lock	2026-05-28 15:35:02
36	TRANSFER_IN	100000.00	2	1	NULL	25100000.00	SUCCESS	NULL	Nhận tiền đồng thời [T1] không lock	2026-05-28 15:35:02
37	TRANSFER_OUT	1000000.00	1	2	NULL	8900000.00	SUCCESS	NULL	Chuyển tiền đồng thời [T1] không lock	2026-05-28 15:46:09
38	TRANSFER_IN	1000000.00	2	1	NULL	26100000.00	SUCCESS	NULL	Nhận tiền đồng thời [T1] không lock	2026-05-28 15:46:09
39	TRANSFER_OUT	1000000.00	1	2	NULL	7900000.00	SUCCESS	NULL	Chuyển tiền đồng thời [T1] không lock	2026-05-28 15:49:52
40	TRANSFER_IN	1000000.00	2	1	NULL	27100000.00	SUCCESS	NULL	Nhận tiền đồng thời [T1] không lock	2026-05-28 15:49:52
41	TRANSFER_OUT	1000000.00	1	2	NULL	6800000.00	SUCCESS	NULL	Chuyển tiền nội bộ	2026-05-28 17:27:57
42	TRANSFER_IN	1000000.00	2	1	NULL	28200000.00	SUCCESS	NULL	Nhận tiền nội bộ	2026-05-28 17:27:57
43	TRANSFER_OUT	1000000.00	1	2	NULL	9800000.00	SUCCESS	NULL	Deadlock demo: A→B thành công	2026-05-28 17:51:16
44	TRANSFER_IN	1000000.00	2	1	NULL	25200000.00	SUCCESS	NULL	Deadlock demo: nhận tiền từ A→B	2026-05-28 17:51:16
45	TRANSFER_OUT	1000000.00	1	2	NULL	8800000.00	SUCCESS	NULL	Deadlock demo: A→B thành công	2026-05-28 17:51:50
46	TRANSFER_IN	1000000.00	2	1	NULL	26200000.00	SUCCESS	NULL	Deadlock demo: nhận tiền từ A→B	2026-05-28 17:51:50
47	TRANSFER_OUT	1000000.00	1	2	NULL	7800000.00	SUCCESS	NULL	Deadlock demo: A→B thành công	2026-05-28 17:53:26
48	TRANSFER_IN	1000000.00	2	1	NULL	27200000.00	SUCCESS	NULL	Deadlock demo: nhận tiền từ A→B	2026-05-28 17:53:26
49	TRANSFER_OUT	2000000.00	1	2	NULL	5800000.00	SUCCESS	NULL	Deadlock demo: B→A thành công	2026-05-28 17:54:04
50	TRANSFER_IN	2000000.00	2	1	NULL	29200000.00	SUCCESS	NULL	Deadlock demo: nhận tiền từ B→A	2026-05-28 17:54:04
51	INTER_BRANCH_OUT	50000000.00	1	2	DN	5800000.00	FAILED	TXN-1779990874822	Inter branch transfer FAILED - Không đủ số dư	2026-05-28 17:54:34
52	INTER_BRANCH_OUT	5000000.00	1	2	DN	800000.00	SUCCESS	TXN-1779990895204	Inter branch transfer out	2026-05-28 17:54:55
53	DEPOSIT	14200000.00	1	NULL	NULL	15000000.00	SUCCESS	NULL	Nạp tiền	2026-05-29 09:36:39
54	WITHDRAW	9200000.00	1	NULL	NULL	5800000.00	SUCCESS	NULL	Rút tiền	2026-05-29 09:36:56
55	DEPOSIT	9200000.00	1	NULL	NULL	15000000.00	SUCCESS	NULL	Nạp tiền	2026-05-29 09:37:19
56	WITHDRAW	9200000.00	2	NULL	NULL	20000000.00	SUCCESS	NULL	Rút tiền	2026-05-29 09:40:31
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Hình 4.57 Bảng transaction_history trước khi thực hiện chuyển tiền cùng chi nhánh

B2. Thao tác trên giao diện



Hình 4.58 Thực hiện rút tiền cùng chi nhánh trên giao diện

B3. CSDL sau khi thao tác

- **Bảng account**

account_id	customer_id	branch_id	balance	status	created_at
1	1	HN	14500000.00	ACTIVE	2026-05-28 06:41:02
2	1	HN	20500000.00	ACTIVE	2026-05-28 06:41:02
3	2	HN	75000000.00	ACTIVE	2026-05-28 06:41:02
4	3	HN	30000000.00	ACTIVE	2026-05-28 06:41:02
5	4	HN	100000000.00	ACTIVE	2026-05-28 06:41:02
6	5	HN	15000000.00	ACTIVE	2026-05-28 06:41:02
NULL	NULL	NULL	NULL	NULL	NULL

Hình 4.59 Bảng account sau khi thực hiện rút tiền cùng chi nhánh

- **Bảng transaction_history**

transaction...	transaction_type	amount	account_id	related_account...	related_branch...	balance_after	status	distributed_txn_id	description	created_at
1	DEPOSIT	50000000.00	1	NULL	NULL	50000000.00	SUCCESS	NULL	Nạp tiền ban đầu	2026-05-28 06:41:02
2	DEPOSIT	20000000.00	2	NULL	NULL	20000000.00	SUCCESS	NULL	Nạp tiền ban đầu	2026-05-28 06:41:02
3	DEPOSIT	75000000.00	3	NULL	NULL	75000000.00	SUCCESS	NULL	Nạp tiền ban đầu	2026-05-28 06:41:02
4	DEPOSIT	30000000.00	4	NULL	NULL	30000000.00	SUCCESS	NULL	Nạp tiền ban đầu	2026-05-28 06:41:02
5	DEPOSIT	100000000.00	5	NULL	NULL	100000000.00	SUCCESS	NULL	Nạp tiền ban đầu	2026-05-28 06:41:02
6	DEPOSIT	15000000.00	6	NULL	NULL	15000000.00	SUCCESS	NULL	Nạp tiền ban đầu	2026-05-28 06:41:02
7	DEPOSIT	1000000.00	1	NULL	NULL	51000000.00	SUCCESS	NULL	Nạp tiền	2026-05-28 06:45:38
8	WITHDRAW	1000000.00	1	NULL	NULL	50000000.00	SUCCESS	NULL	Rút tiền	2026-05-28 06:53:24
9	INTER_BRANCH_OUT	5000000.00	1	1	HCM	45000000.00	SUCCESS	TXN-1779951226414	Inter branch transfer out	2026-05-28 06:53:46
10	INTER_BRANCH_OUT	5000000.00	1	1	HCM	45000000.00	FAILED	TXN-1779951230396	Inter branch transfer FAILED - HCM server bị cr...	2026-05-28 06:53:50
11	DEPOSIT	500000.00	1	NULL	NULL	45500000.00	SUCCESS	NULL	Nạp tiền	2026-05-28 10:52:01
12	WITHDRAW	500000.00	1	NULL	NULL	45000000.00	SUCCESS	NULL	Rút tiền	2026-05-28 10:58:46
13	DEPOSIT	500000.00	1	NULL	NULL	45500000.00	SUCCESS	NULL	Nạp tiền	2026-05-28 11:02:25
20	WITHDRAW	500000.00	1	NULL	NULL	45000000.00	SUCCESS	NULL	Rút tiền	2026-05-28 11:41:45
21	WITHDRAW	10000000.00	1	NULL	NULL	35000000.00	SUCCESS	NULL	Rút tiền concurrent không lock	2026-05-28 11:42:22
22	WITHDRAW	10000000.00	1	NULL	NULL	35000000.00	SUCCESS	NULL	Rút tiền concurrent không lock	2026-05-28 11:42:22
23	WITHDRAW	10000000.00	1	NULL	NULL	25000000.00	SUCCESS	NULL	Rút tiền concurrent có lock	2026-05-28 11:43:11
24	WITHDRAW	10000000.00	1	NULL	NULL	15000000.00	SUCCESS	NULL	Rút tiền concurrent có lock	2026-05-28 11:43:11
27	DEPOSIT	30500000.00	1	NULL	NULL	45500000.00	SUCCESS	NULL	Nạp tiền	2026-05-28 12:07:24
28	WITHDRAW	500000.00	1	NULL	NULL	45000000.00	SUCCESS	NULL	Rút tiền	2026-05-28 12:08:12
29	WITHDRAW	10000000.00	1	NULL	NULL	35000000.00	SUCCESS	NULL	Rút tiền concurrent không lock	2026-05-28 12:10:06
30	WITHDRAW	10000000.00	1	NULL	NULL	35000000.00	SUCCESS	NULL	Rút tiền concurrent không lock	2026-05-28 12:10:06
31	WITHDRAW	10000000.00	1	NULL	NULL	25000000.00	SUCCESS	NULL	Rút tiền concurrent có lock	2026-05-28 12:13:25
32	WITHDRAW	10000000.00	1	NULL	NULL	15000000.00	SUCCESS	NULL	Rút tiền concurrent có lock	2026-05-28 12:13:25
33	TRANSFER_OUT	5000000.00	1	2	NULL	10000000.00	SUCCESS	NULL	Chuyển tiền nội bộ	2026-05-28 12:16:32
34	TRANSFER_IN	5000000.00	2	1	NULL	25000000.00	SUCCESS	NULL	Nhận tiền nội bộ	2026-05-28 12:16:32
35	TRANSFER_OUT	100000.00	1	2	NULL	9900000.00	SUCCESS	NULL	Chuyển tiền đồng thời [T1] không lock	2026-05-28 15:35:02
36	TRANSFER_IN	100000.00	2	1	NULL	25100000.00	SUCCESS	NULL	Nhận tiền đồng thời [T1] không lock	2026-05-28 15:35:02
37	TRANSFER_OUT	1000000.00	1	2	NULL	8900000.00	SUCCESS	NULL	Chuyển tiền đồng thời [T1] không lock	2026-05-28 15:46:09
38	TRANSFER_IN	1000000.00	2	1	NULL	26100000.00	SUCCESS	NULL	Nhận tiền đồng thời [T1] không lock	2026-05-28 15:46:09
39	TRANSFER_OUT	1000000.00	1	2	NULL	7900000.00	SUCCESS	NULL	Chuyển tiền đồng thời [T1] không lock	2026-05-28 15:49:52
40	TRANSFER_IN	1000000.00	2	1	NULL	27100000.00	SUCCESS	NULL	Nhận tiền đồng thời [T1] không lock	2026-05-28 15:49:52
41	TRANSFER_OUT	1000000.00	1	2	NULL	6800000.00	SUCCESS	NULL	Chuyển tiền nội bộ	2026-05-28 17:27:57
42	TRANSFER_IN	1000000.00	2	1	NULL	28200000.00	SUCCESS	NULL	Nhận tiền nội bộ	2026-05-28 17:27:57
43	TRANSFER_OUT	1000000.00	1	2	NULL	9800000.00	SUCCESS	NULL	Deadlock demo: A→B thành công	2026-05-28 17:51:16
44	TRANSFER_IN	1000000.00	2	1	NULL	25200000.00	SUCCESS	NULL	Deadlock demo: nhận tiền từ A→B	2026-05-28 17:51:16
45	TRANSFER_OUT	1000000.00	1	2	NULL	8800000.00	SUCCESS	NULL	Deadlock demo: A→B thành công	2026-05-28 17:51:50
46	TRANSFER_IN	1000000.00	2	1	NULL	26200000.00	SUCCESS	NULL	Deadlock demo: nhận tiền từ A→B	2026-05-28 17:51:50
47	TRANSFER_OUT	1000000.00	1	2	NULL	7800000.00	SUCCESS	NULL	Deadlock demo: A→B thành công	2026-05-28 17:53:26
48	TRANSFER_IN	1000000.00	2	1	NULL	27200000.00	SUCCESS	NULL	Deadlock demo: nhận tiền từ A→B	2026-05-28 17:53:26
49	TRANSFER_OUT	2000000.00	1	2	NULL	5800000.00	SUCCESS	NULL	Deadlock demo: B→A thành công	2026-05-28 17:54:04
50	TRANSFER_IN	2000000.00	2	1	NULL	29200000.00	SUCCESS	NULL	Deadlock demo: nhận tiền từ B→A	2026-05-28 17:54:04
51	INTER_BRANCH_OUT	50000000.00	1	2	DN	5800000.00	FAILED	TXN-1779990874822	Inter branch transfer FAILED - Không đủ số dư	2026-05-28 17:54:34
52	INTER_BRANCH_OUT	5000000.00	1	2	DN	800000.00	SUCCESS	TXN-1779990895204	Inter branch transfer out	2026-05-28 17:54:55
53	DEPOSIT	14200000.00	1	NULL	NULL	15000000.00	SUCCESS	NULL	Nạp tiền	2026-05-29 09:36:39
54	WITHDRAW	9200000.00	1	NULL	NULL	5800000.00	SUCCESS	NULL	Rút tiền	2026-05-29 09:36:56
55	DEPOSIT	9200000.00	1	NULL	NULL	15000000.00	SUCCESS	NULL	Nạp tiền	2026-05-29 09:37:19
56	WITHDRAW	9200000.00	2	NULL	NULL	20000000.00	SUCCESS	NULL	Rút tiền	2026-05-29 09:40:31
57	TRANSFER_OUT	500000.00	1	2	NULL	14500000.00	SUCCESS	NULL	Chuyển tiền nội bộ	2026-05-29 09:43:07
58	TRANSFER_IN	500000.00	2	1	NULL	20500000.00	SUCCESS	NULL	Nhận tiền nội bộ	2026-05-29 09:43:07
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Hình 4.60 Bảng transaction_history sau khi thực hiện rút tiền cùng chi nhánh

Kết quả in ra trên terminal:

```
================================================
CHUYỂN TIỀN NỘI BỘ
================================================
Chi nhánh       : HN
Tài khoản gửi   : 1
Tài khoản nhận  : 2
Số tiền chuyển  : 500000 VND
------------------------------------------------
CHUYỂN TIỀN THÀNH CÔNG
Số dư tài khoản gửi  : 14500000.00
Số dư tài khoản nhận : 20500000.00
================================================
```

Hình 4.61 Mô phỏng quá trình thực hiện rút tiền cùng chi nhánh

6. Đánh giá cơ chế Ordered Locking (Ngăn chặn Deadlock)

Mặc dù là giao dịch cục bộ trong cùng một node, hệ thống vẫn bắt buộc phải sử dụng cơ chế khóa bi quan SELECT ... FOR UPDATE nhằm khóa đồng thời cả 2 dòng tài khoản trong MySQL để phòng ngừa lỗi mất cập nhật (Lost Update).

Tuy nhiên, trong môi trường đa luồng, một lỗ hổng nghiêm trọng về khóa chéo (Deadlock) sẽ xuất hiện nếu xảy ra kịch bản tương tranh sau:

- Tiến trình 1: Người dùng A (ID 1) thực hiện chuyển tiền cho người dùng B (ID 2) => Hệ thống chiếm giữ khóa tài khoản A và chờ để khóa tiếp tài khoản B.
- Tiến trình 2: Hoàn toàn đồng thời, người dùng B (ID 2) thao tác chuyển ngược tiền lại cho A (ID 1) => Hệ thống chiếm giữ khóa tài khoản B và chờ để khóa tiếp tài khoản A.

Hai tiến trình này sẽ rơi vào trạng thái bế tắc vô hạn (Deadlock) do giữ khóa của nhau và cùng chờ tài nguyên của đối phương nhả ra.

Để giải quyết triệt để rủi ro kỹ thuật này, đồ án đã cài đặt thuật toán ép buộc thứ tự khóa (Ordered Locking) từ ID nhỏ đến ID lớn tại lớp xử lý nghiệp vụ nghiệp vụ trung tâm (tệp TransferService.java):

```
Long firstId = Math.min(request.getFromAccountId(), request.getToAccountId());
Long secondId = Math.max(request.getFromAccountId(), request.getToAccountId());
// Luôn thực hiện chiếm giữ khóa dòng có ID nhỏ trước
jdbc.queryForObject("SELECT balance FROM account WHERE account_id = ? FOR UPDATE", BigDecimal.class, firstId);
// Thực hiện chiếm giữ khóa dòng có ID lớn sau
jdbc.queryForObject("SELECT balance FROM account WHERE account_id = ? FOR UPDATE", BigDecimal.class, secondId);
```

Nguyên lý bảo vệ hệ thống: Nhờ áp dụng giải pháp định tuyến thứ tự khóa cố định này, bất chấp mọi hướng dịch chuyển của dòng tiền (dù A chuyển cho B hay B chuyển cho A), hệ thống luôn thực thi một chuỗi khóa tuần tự duy nhất (Luôn luôn khóa thực thể ID 1 trước, sau đó mới tiến hành khóa thực thể ID 2).

Nếu có tương tranh xảy ra, tiến trình đến sau bắt buộc phải rơi vào trạng thái xếp hàng chờ đợi luồng xử lý trước giải phóng hoàn toàn cặp khóa, loại bỏ 100% nguy cơ xảy ra hiện tượng Deadlock vòng lặp, đem lại sự ổn định và an toàn tuyệt đối cho hệ thống Core Banking.

4.6.5 Kiểm thử chức năng chuyển liên chi nhánh (Internal Transfer)

Giao dịch chuyển tiền liên chi nhánh là kịch bản phức tạp nhất trong hệ thống ngân hàng phân tán, đòi hỏi dòng tiền phải được dịch chuyển giữa hai nút mạng vật lý hoàn toàn độc lập (ví dụ: Từ máy chủ Hà Nội vào máy chủ TP.HCM).

Để đảm bảo nguyên tắc ACID trong môi trường phân tán (đặc biệt là tính nhất quán - Consistency và tính nguyên tử - Atomicity), hệ thống áp dụng giao thức Two-Phase Commit (2PC) do Spring Boot Coordinator điều phối.

Kịch bản 1: Giao dịch diễn ra thuận lợi, không xảy ra sự cố (2PC Commit)

1. Mục tiêu kiểm thử

- Đảm bảo luồng thực thi 2-Phase Commit (Phase 1: Prepare -> Phase 2: Commit) hoạt động chính xác.
- Tiền được trừ tại chi nhánh nguồn và cộng tại chi nhánh đích một cách nhất quán.
- Hai bản ghi lịch sử INTER_BRANCH_OUT và INTER_BRANCH_IN được lưu ở hai máy chủ khác nhau nhưng chia sẻ chung một distributed_txn_id.

2. Chuẩn bị kịch bản

- Chi nhánh nguồn: Hà Nội (HN), Tài khoản ID 1, Số dư hiện tại 15.000.000 VNĐ.
- Chi nhánh đích: TP.HCM (HCM), Tài khoản ID 5, Số dư hiện tại 3.000.000 VNĐ.
- Yêu cầu nghiệp vụ: Chuyển 5.000.000 VNĐ. Không kích hoạt cờ giả lập lỗi.

3. Thực thi kiểm thử (API Request)

```bash
curl -X POST http://localhost:8080/api/transfers/inter-branch \
-H "Content-Type: application/json" \
-d '{
  "fromBranch": "HN",
  "fromAccountId": 1,
  "toBranch": "HCM",
  "toAccountId": 5,
  "amount": 5000000,
  "simulateCrash": false
}'
```

Hình 4.62 Kết quả dưới dạng JSON sau khi thực hiện chuyển tiền liên chi nhánh - không xảy ra sự cố

4. Kết quả và Luồng thực thi Hệ thống xử lý thành công, trả về HTTP 200 OK.

Dựa trên kết xuất log từ Coordinator, tiến trình 2PC diễn ra như sau:

- PHASE 1 (PREPARE):
  - Hệ thống khóa tài khoản ID 1 tại HN, xác nhận số dư đủ, tiến hành trừ 5.000.000 (giảm xuống 10.000.000). Trạng thái Transaction: Chưa Commit.
  - Hệ thống khóa tài khoản ID 5 tại HCM, tiến hành cộng 5.000.000 (tăng lên 8.000.000). Trạng thái Transaction: Chưa Commit.
- PHASE 2 (COMMIT):
  - Do cả 2 Node đều phản hồi Phase 1 thành công, máy chủ điều phối phát lệnh Commit.
  - Database HN Commit thành công. Database HCM Commit thành công.

Đối soát số dư thực tế: Số dư ID 1 (HN) là 10.000.000 VNĐ. Số dư ID 5 (HCM) là 8.000.000 VNĐ. Dữ liệu hoàn toàn nhất quán.

5. Thực thi trên giao diện và trên CSDL

B1. CSDL trước khi thực hiện

- Bảng account

  +Branch HN

account_id	customer_id	branch_id	balance	status	created_at
1	1	HN	15000000.00	ACTIVE	2026-05-28 06:41:02
2	1	HN	20500000.00	ACTIVE	2026-05-28 06:41:02
3	2	HN	75000000.00	ACTIVE	2026-05-28 06:41:02
4	3	HN	30000000.00	ACTIVE	2026-05-28 06:41:02
5	4	HN	100000000.00	ACTIVE	2026-05-28 06:41:02
6	5	HN	15000000.00	ACTIVE	2026-05-28 06:41:02
NULL	NULL	NULL	NULL	NULL	NULL

Hình 4.63 Bảng account tại site HNt trước khi thực hiện chuyển tiền liên chi nhánh - không xảy ra sự cố

+Branch HCM

account_id	customer_id	branch_id	balance	status	created_at
1	1	HCM	95000000.00	ACTIVE	2026-05-28 06:41:02
2	2	HCM	55000000.00	ACTIVE	2026-05-28 06:41:02
3	3	HCM	120000000.00	ACTIVE	2026-05-28 06:41:02
4	4	HCM	55000000.00	ACTIVE	2026-05-28 06:41:02
5	5	HCM	70000000.00	ACTIVE	2026-05-28 06:41:02
NULL	NULL	NULL	NULL	NULL	NULL

Hình 4.64 Bảng account tại site HCM trước khi thực hiện chuyển tiền liên chi nhánh - không xảy ra sự cố

- Bảng transaction_history

+Branch HN

transaction...	transaction_type	amount	account_id	related_account...	related_branch...	balance_after	status	distributed_txn_id	description	created_at
1	DEPOSIT	50000000.00	1	NULL	NULL	50000000.00	SUCCESS	NULL	Nạp tiền ban đầu	2026-05-28 06:41:02
2	DEPOSIT	20000000.00	2	NULL	NULL	20000000.00	SUCCESS	NULL	Nạp tiền ban đầu	2026-05-28 06:41:02
3	DEPOSIT	75000000.00	3	NULL	NULL	75000000.00	SUCCESS	NULL	Nạp tiền ban đầu	2026-05-28 06:41:02
4	DEPOSIT	30000000.00	4	NULL	NULL	30000000.00	SUCCESS	NULL	Nạp tiền ban đầu	2026-05-28 06:41:02
5	DEPOSIT	100000000.00	5	NULL	NULL	100000000.00	SUCCESS	NULL	Nạp tiền ban đầu	2026-05-28 06:41:02
6	DEPOSIT	15000000.00	6	NULL	NULL	15000000.00	SUCCESS	NULL	Nạp tiền ban đầu	2026-05-28 06:41:02
7	DEPOSIT	1000000.00	1	NULL	NULL	51000000.00	SUCCESS	NULL	Nạp tiền	2026-05-28 06:45:38
8	WITHDRAW	1000000.00	1	NULL	NULL	50000000.00	SUCCESS	NULL	Rút tiền	2026-05-28 06:53:24
9	INTER_BRANCH_OUT	5000000.00	1	1	HCM	45000000.00	SUCCESS	TXN-1779951226414	Inter branch transfer out	2026-05-28 06:53:46
10	INTER_BRANCH_OUT	5000000.00	1	1	HCM	45000000.00	FAILED	TXN-1779951230396	Inter branch transfer FAILED - HCM server bị cr...	2026-05-28 06:53:50
11	DEPOSIT	500000.00	1	NULL	NULL	45500000.00	SUCCESS	NULL	Nạp tiền	2026-05-28 10:52:01
12	WITHDRAW	500000.00	1	NULL	NULL	45000000.00	SUCCESS	NULL	Rút tiền	2026-05-28 10:58:46
13	DEPOSIT	500000.00	1	NULL	NULL	45500000.00	SUCCESS	NULL	Nạp tiền	2026-05-28 11:02:25
20	WITHDRAW	500000.00	1	NULL	NULL	45000000.00	SUCCESS	NULL	Rút tiền	2026-05-28 11:41:45
21	WITHDRAW	10000000.00	1	NULL	NULL	35000000.00	SUCCESS	NULL	Rút tiền concurrent không lock	2026-05-28 11:42:22
22	WITHDRAW	10000000.00	1	NULL	NULL	35000000.00	SUCCESS	NULL	Rút tiền concurrent không lock	2026-05-28 11:42:22
23	WITHDRAW	10000000.00	1	NULL	NULL	25000000.00	SUCCESS	NULL	Rút tiền concurrent có lock	2026-05-28 11:43:11
24	WITHDRAW	10000000.00	1	NULL	NULL	15000000.00	SUCCESS	NULL	Rút tiền concurrent có lock	2026-05-28 11:43:11
27	DEPOSIT	30500000.00	1	NULL	NULL	45500000.00	SUCCESS	NULL	Nạp tiền	2026-05-28 12:07:24
28	WITHDRAW	500000.00	1	NULL	NULL	45000000.00	SUCCESS	NULL	Rút tiền	2026-05-28 12:08:12
29	WITHDRAW	10000000.00	1	NULL	NULL	35000000.00	SUCCESS	NULL	Rút tiền concurrent không lock	2026-05-28 12:10:06
30	WITHDRAW	10000000.00	1	NULL	NULL	35000000.00	SUCCESS	NULL	Rút tiền concurrent không lock	2026-05-28 12:10:06
31	WITHDRAW	10000000.00	1	NULL	NULL	25000000.00	SUCCESS	NULL	Rút tiền concurrent có lock	2026-05-28 12:13:25
32	WITHDRAW	10000000.00	1	NULL	NULL	15000000.00	SUCCESS	NULL	Rút tiền concurrent có lock	2026-05-28 12:13:25
33	TRANSFER_OUT	5000000.00	1	2	NULL	10000000.00	SUCCESS	NULL	Chuyển tiền nội bộ	2026-05-28 12:16:32
34	TRANSFER_IN	5000000.00	2	1	NULL	25000000.00	SUCCESS	NULL	Nhận tiền nội bộ	2026-05-28 12:16:32
35	TRANSFER_OUT	100000.00	1	2	NULL	9900000.00	SUCCESS	NULL	Chuyển tiền đồng thời [T1] không lock	2026-05-28 15:35:02
36	TRANSFER_IN	100000.00	2	1	NULL	25100000.00	SUCCESS	NULL	Nhận tiền đồng thời [T1] không lock	2026-05-28 15:35:02
37	TRANSFER_OUT	1000000.00	1	2	NULL	8900000.00	SUCCESS	NULL	Chuyển tiền đồng thời [T1] không lock	2026-05-28 15:46:09
38	TRANSFER_IN	1000000.00	2	1	NULL	26100000.00	SUCCESS	NULL	Nhận tiền đồng thời [T1] không lock	2026-05-28 15:46:09
39	TRANSFER_OUT	1000000.00	1	2	NULL	7900000.00	SUCCESS	NULL	Chuyển tiền đồng thời [T1] không lock	2026-05-28 15:49:52
40	TRANSFER_IN	1000000.00	2	1	NULL	27100000.00	SUCCESS	NULL	Nhận tiền đồng thời [T1] không lock	2026-05-28 15:49:52
41	TRANSFER_OUT	1000000.00	1	2	NULL	6800000.00	SUCCESS	NULL	Chuyển tiền nội bộ	2026-05-28 17:27:57
42	TRANSFER_IN	1000000.00	2	1	NULL	28200000.00	SUCCESS	NULL	Nhận tiền nội bộ	2026-05-28 17:27:57
43	TRANSFER_OUT	1000000.00	1	2	NULL	9800000.00	SUCCESS	NULL	Deadlock demo: A→B thành công	2026-05-28 17:51:16
44	TRANSFER_IN	1000000.00	2	1	NULL	25200000.00	SUCCESS	NULL	Deadlock demo: nhận tiền từ A→B	2026-05-28 17:51:16
45	TRANSFER_OUT	1000000.00	1	2	NULL	8800000.00	SUCCESS	NULL	Deadlock demo: A→B thành công	2026-05-28 17:51:50
46	TRANSFER_IN	1000000.00	2	1	NULL	26200000.00	SUCCESS	NULL	Deadlock demo: nhận tiền từ A→B	2026-05-28 17:51:50
47	TRANSFER_OUT	1000000.00	1	2	NULL	7800000.00	SUCCESS	NULL	Deadlock demo: A→B thành công	2026-05-28 17:53:26
48	TRANSFER_IN	1000000.00	2	1	NULL	27200000.00	SUCCESS	NULL	Deadlock demo: nhận tiền từ A→B	2026-05-28 17:53:26
49	TRANSFER_OUT	2000000.00	1	2	NULL	5800000.00	SUCCESS	NULL	Deadlock demo: B→A thành công	2026-05-28 17:54:04
50	TRANSFER_IN	2000000.00	2	1	NULL	29200000.00	SUCCESS	NULL	Deadlock demo: nhận tiền từ B→A	2026-05-28 17:54:04
51	INTER_BRANCH_OUT	50000000.00	1	2	DN	5800000.00	FAILED	TXN-1779990874822	Inter branch transfer FAILED - Không đủ số dư	2026-05-28 17:54:34
52	INTER_BRANCH_OUT	5000000.00	1	2	DN	800000.00	SUCCESS	TXN-1779990895204	Inter branch transfer out	2026-05-28 17:54:55
53	DEPOSIT	14200000.00	1	NULL	NULL	15000000.00	SUCCESS	NULL	Nạp tiền	2026-05-29 09:36:39
54	WITHDRAW	9200000.00	1	NULL	NULL	5800000.00	SUCCESS	NULL	Rút tiền	2026-05-29 09:36:56
55	DEPOSIT	9200000.00	1	NULL	NULL	15000000.00	SUCCESS	NULL	Nạp tiền	2026-05-29 09:37:19
56	WITHDRAW	9200000.00	2	NULL	NULL	20000000.00	SUCCESS	NULL	Rút tiền	2026-05-29 09:40:31
57	TRANSFER_OUT	500000.00	1	2	NULL	14500000.00	SUCCESS	NULL	Chuyển tiền nội bộ	2026-05-29 09:43:07
58	TRANSFER_IN	500000.00	2	1	NULL	20500000.00	SUCCESS	NULL	Nhận tiền nội bộ	2026-05-29 09:43:07
59	INTER_BRANCH_OUT	5000000.00	1	2	HCM	9500000.00	SUCCESS	TXN-1780048217000	Inter branch transfer out	2026-05-29 09:50:17
60	INTER_BRANCH_OUT	5000000.00	1	2	HCM	4500000.00	SUCCESS	TXN-1780048610090	Inter branch transfer out	2026-05-29 09:56:50
61	DEPOSIT	11500000.00	1	NULL	NULL	16000000.00	SUCCESS	NULL	Nạp tiền	2026-05-29 09:57:20
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Hình 4.65 Bảng transaction_history tại site HN trước khi thực hiện chuyển tiền liên chi nhánh - không xảy ra sự cố

+Branch HCM

transaction...	transaction_type	amount	account_id	related_account...	related_branch...	balance_after	status	distributed_txn_id	description	created_at
1	DEPOSIT	90000000.00	1	NULL	NULL	90000000.00	SUCCESS	NULL	Nạp tiền ban đầu	2026-05-28 06:41:02
2	DEPOSIT	45000000.00	2	NULL	NULL	45000000.00	SUCCESS	NULL	Nạp tiền ban đầu	2026-05-28 06:41:02
3	DEPOSIT	120000000.00	3	NULL	NULL	120000000.00	SUCCESS	NULL	Nạp tiền ban đầu	2026-05-28 06:41:02
4	DEPOSIT	55000000.00	4	NULL	NULL	55000000.00	SUCCESS	NULL	Nạp tiền ban đầu	2026-05-28 06:41:02
5	DEPOSIT	70000000.00	5	NULL	NULL	70000000.00	SUCCESS	NULL	Nạp tiền ban đầu	2026-05-28 06:41:02
6	INTER_BRANCH_IN	5000000.00	1	1	HN	95000000.00	SUCCESS	TXN-1779951226414	Inter branch transfer in	2026-05-28 06:53:46
7	INTER_BRANCH_IN	5000000.00	1	1	HN	95000000.00	FAILED	TXN-1779951230396	Inter branch transfer FAILED - HCM server bị cr...	2026-05-28 06:53:50
8	INTER_BRANCH_IN	5000000.00	2	1	HN	50000000.00	SUCCESS	TXN-1780048217000	Inter branch transfer in	2026-05-29 09:50:17
9	INTER_BRANCH_IN	5000000.00	2	1	HN	55000000.00	SUCCESS	TXN-1780048610090	Inter branch transfer in	2026-05-29 09:56:50
10	INTER_BRANCH_IN	5000000.00	5	1	HN	75000000.00	SUCCESS	TXN-1780049021441	Inter branch transfer in	2026-05-29 10:03:41
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Hình 4.66 Bảng transaction_history tại site HCM trước khi thực hiện chuyển tiền liên chi nhánh - không xảy ra sự cố

- Bảng distributed_transaction_log

+Branch HN

txn_id	txn_type	status	source_bran...	dest_branch	amount	source_account...	dest_account...	error_message	created_at	updated_at
TXN-1779951226414	INTER_BRANCH_TRANSFER	COMMITTED	HN	HCM	5000000.00	1	1	NULL	2026-05-28 06:53:46	2026-05-28 06:53:46
TXN-1779951230396	INTER_BRANCH_TRANSFER	ABORTED	HN	HCM	5000000.00	1	1	HCM server bị crash	2026-05-28 06:53:50	2026-05-28 06:53:50
TXN-1779990874822	INTER_BRANCH_TRANSFER	ABORTED	HN	DN	50000000.00	1	2	Không đủ số dư	2026-05-28 17:54:34	2026-05-28 17:54:34
TXN-1779990895204	INTER_BRANCH_TRANSFER	COMMITTED	HN	DN	5000000.00	1	2	NULL	2026-05-28 17:54:55	2026-05-28 17:54:55
TXN-1780048217000	INTER_BRANCH_TRANSFER	COMMITTED	HN	HCM	5000000.00	1	2	NULL	2026-05-29 09:50:17	2026-05-29 09:50:17
TXN-1780048610090	INTER_BRANCH_TRANSFER	COMMITTED	HN	HCM	5000000.00	1	2	NULL	2026-05-29 09:56:50	2026-05-29 09:56:50
TXN-1780049021441	INTER_BRANCH_TRANSFER	COMMITTED	HN	HCM	5000000.00	1	5	NULL	2026-05-29 10:03:41	2026-05-29 10:03:41
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Hình 4.67 Bảng distributed_transaction_log tại site HN trước khi thực hiện chuyển tiền liên chi nhánh - không xảy ra sự cố

+Branch HCM

txn_id	txn_type	status	source_bran...	dest_branch	amount	source_account...	dest_account...	error_message	created_at	updated_at
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Hình 4.68 Bảng distributed_transaction_log tại site HCM trước khi thực hiện chuyển tiền liên chi nhánh - không xảy ra sự cố

- Bảng transaction_participant

  +Branch HN

id	txn_id	branch_id	role	status	action	created_at
1	TXN-1780048610090	HN	SOURCE	COMMITTED	DEBIT	2026-05-29 09:56:50
2	TXN-1780049021441	HN	SOURCE	COMMITTED	DEBIT	2026-05-29 10:03:41
NULL	NULL	NULL	NULL	NULL	NULL	NULL

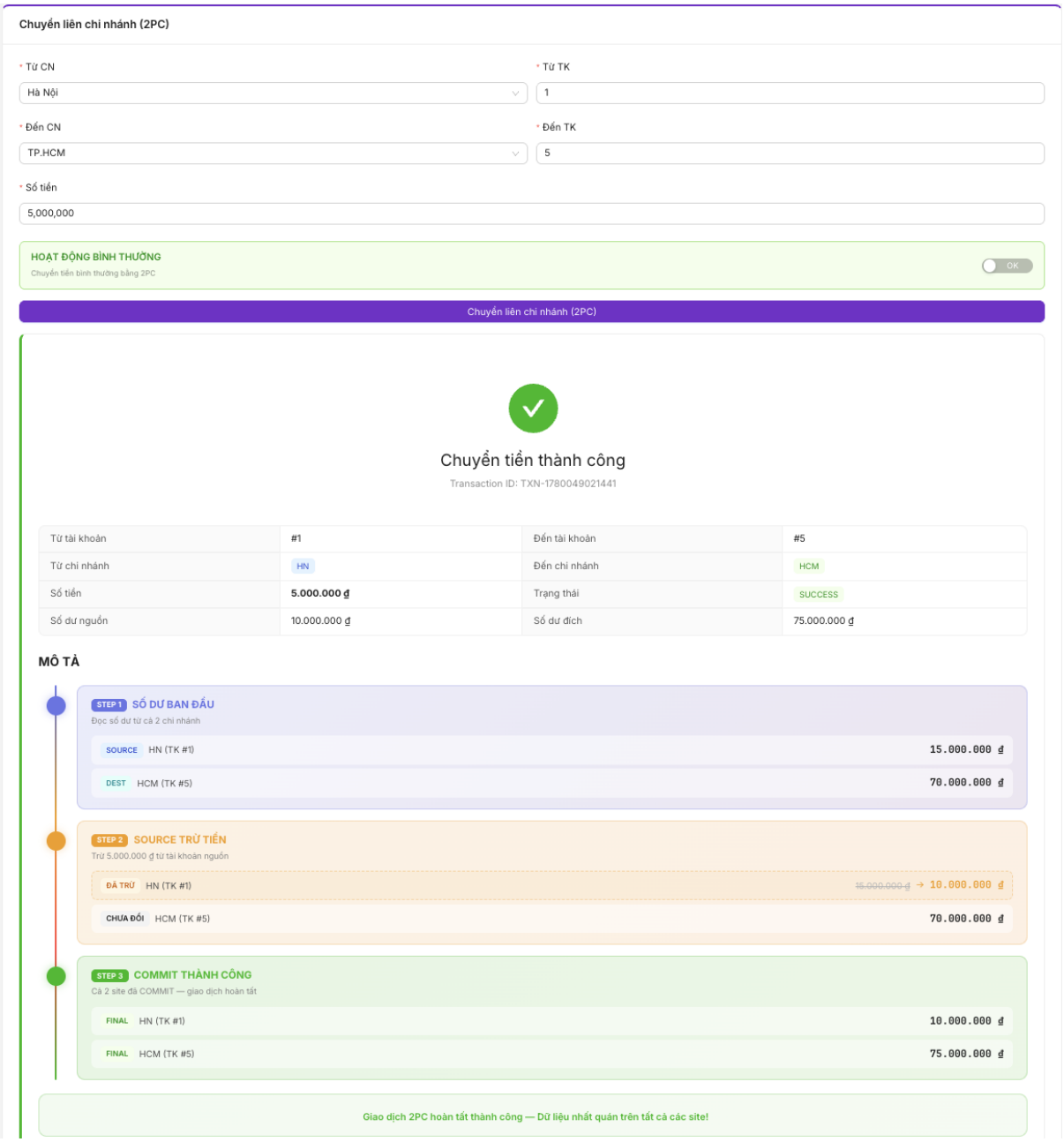
Hình 4.69 Bảng transaction_participant tại site HN trước khi chuyển tiền liên chi nhánh - không xảy ra sự cố

+Branch HCM

id	txn_id	branch_id	role	status	action	created_at
NULL	NULL	NULL	NULL	NULL	NULL	NULL

Hình 4.70 Bảng transaction_participant tại site HCM trước khi chuyển tiền liên chi nhánh - không xảy ra sự cố

B2. Thao tác trên giao diện



Hình 4.71 Thực hiện chuyển tiền liên chi nhánh trên giao diện - không xảy ra sự cố

B3. CSDL sau khi thao tác

- Bảng account

  +Branch HN

account_id	customer_id	branch_id	balance	status	created_at
1	1	HN	10000000.00	ACTIVE	2026-05-28 06:41:02
2	1	HN	20500000.00	ACTIVE	2026-05-28 06:41:02
3	2	HN	75000000.00	ACTIVE	2026-05-28 06:41:02
4	3	HN	30000000.00	ACTIVE	2026-05-28 06:41:02
5	4	HN	100000000.00	ACTIVE	2026-05-28 06:41:02
6	5	HN	15000000.00	ACTIVE	2026-05-28 06:41:02

Hình 4.72 Bảng account tại site HNt sau khi thực hiện chuyển tiền liên chi nhánh - không xảy ra sự cố

+Branch HCM

account_id	customer_id	branch_id	balance	status	created_at
1	1	HCM	95000000.00	ACTIVE	2026-05-28 06:41:02
2	2	HCM	55000000.00	ACTIVE	2026-05-28 06:41:02
3	3	HCM	120000000.00	ACTIVE	2026-05-28 06:41:02
4	4	HCM	55000000.00	ACTIVE	2026-05-28 06:41:02
5	5	HCM	8000000.00	ACTIVE	2026-05-28 06:41:02
NULL	NULL	NULL	NULL	NULL	NULL

Hình 4.73 Bảng account tại site HCM sau khi thực hiện chuyển tiền liên chi nhánh - không xảy ra sự cố

- Bảng transaction_history

+Branch HN

transaction...	transaction_type	amount	account_id	related_account...	related_branch...	balance_after	status	distributed_txn_id	description	created_at
1	DEPOSIT	50000000.00	1	NULL	NULL	50000000.00	SUCCESS	NULL	Nạp tiền ban đầu	2026-05-28 06:41:02
2	DEPOSIT	20000000.00	2	NULL	NULL	20000000.00	SUCCESS	NULL	Nạp tiền ban đầu	2026-05-28 06:41:02
3	DEPOSIT	75000000.00	3	NULL	NULL	75000000.00	SUCCESS	NULL	Nạp tiền ban đầu	2026-05-28 06:41:02
4	DEPOSIT	30000000.00	4	NULL	NULL	30000000.00	SUCCESS	NULL	Nạp tiền ban đầu	2026-05-28 06:41:02
5	DEPOSIT	100000000.00	5	NULL	NULL	100000000.00	SUCCESS	NULL	Nạp tiền ban đầu	2026-05-28 06:41:02
6	DEPOSIT	15000000.00	6	NULL	NULL	15000000.00	SUCCESS	NULL	Nạp tiền ban đầu	2026-05-28 06:41:02
7	DEPOSIT	1000000.00	1	NULL	NULL	51000000.00	SUCCESS	NULL	Nạp tiền	2026-05-28 06:45:38
8	WITHDRAW	1000000.00	1	NULL	NULL	50000000.00	SUCCESS	NULL	Rút tiền	2026-05-28 06:53:24
9	INTER_BRANCH_OUT	5000000.00	1	1	HCM	45000000.00	SUCCESS	TXN-1779951226414	Inter branch transfer out	2026-05-28 06:53:46
10	INTER_BRANCH_OUT	5000000.00	1	1	HCM	45000000.00	FAILED	TXN-1779951230396	Inter branch transfer FAILED - HCM server bị cr...	2026-05-28 06:53:50
11	DEPOSIT	500000.00	1	NULL	NULL	45500000.00	SUCCESS	NULL	Nạp tiền	2026-05-28 10:52:01
12	WITHDRAW	500000.00	1	NULL	NULL	45000000.00	SUCCESS	NULL	Rút tiền	2026-05-28 10:58:46
13	DEPOSIT	500000.00	1	NULL	NULL	45500000.00	SUCCESS	NULL	Nạp tiền	2026-05-28 11:02:25
20	WITHDRAW	500000.00	1	NULL	NULL	45000000.00	SUCCESS	NULL	Rút tiền	2026-05-28 11:41:45
21	WITHDRAW	10000000.00	1	NULL	NULL	35000000.00	SUCCESS	NULL	Rút tiền concurrent không lock	2026-05-28 11:42:22
22	WITHDRAW	10000000.00	1	NULL	NULL	35000000.00	SUCCESS	NULL	Rút tiền concurrent không lock	2026-05-28 11:42:22
23	WITHDRAW	10000000.00	1	NULL	NULL	25000000.00	SUCCESS	NULL	Rút tiền concurrent có lock	2026-05-28 11:43:11
24	WITHDRAW	10000000.00	1	NULL	NULL	15000000.00	SUCCESS	NULL	Rút tiền concurrent có lock	2026-05-28 11:43:11
27	DEPOSIT	30500000.00	1	NULL	NULL	45500000.00	SUCCESS	NULL	Nạp tiền	2026-05-28 12:07:24
28	WITHDRAW	500000.00	1	NULL	NULL	45000000.00	SUCCESS	NULL	Rút tiền	2026-05-28 12:08:12
29	WITHDRAW	10000000.00	1	NULL	NULL	35000000.00	SUCCESS	NULL	Rút tiền concurrent không lock	2026-05-28 12:10:06
30	WITHDRAW	10000000.00	1	NULL	NULL	35000000.00	SUCCESS	NULL	Rút tiền concurrent không lock	2026-05-28 12:10:06
31	WITHDRAW	10000000.00	1	NULL	NULL	25000000.00	SUCCESS	NULL	Rút tiền concurrent có lock	2026-05-28 12:13:25
32	WITHDRAW	10000000.00	1	NULL	NULL	15000000.00	SUCCESS	NULL	Rút tiền concurrent có lock	2026-05-28 12:13:25
33	TRANSFER_OUT	5000000.00	1	2	NULL	10000000.00	SUCCESS	NULL	Chuyển tiền nội bộ	2026-05-28 12:16:32
34	TRANSFER_IN	5000000.00	2	1	NULL	25000000.00	SUCCESS	NULL	Nhận tiền nội bộ	2026-05-28 12:16:32
35	TRANSFER_OUT	100000.00	1	2	NULL	9900000.00	SUCCESS	NULL	Chuyển tiền đồng thời [T1] không lock	2026-05-28 15:35:02
36	TRANSFER_IN	100000.00	2	1	NULL	25100000.00	SUCCESS	NULL	Nhận tiền đồng thời [T1] không lock	2026-05-28 15:35:02
37	TRANSFER_OUT	1000000.00	1	2	NULL	8900000.00	SUCCESS	NULL	Chuyển tiền đồng thời [T1] không lock	2026-05-28 15:46:09
38	TRANSFER_IN	1000000.00	2	1	NULL	26100000.00	SUCCESS	NULL	Nhận tiền đồng thời [T1] không lock	2026-05-28 15:46:09
39	TRANSFER_OUT	1000000.00	1	2	NULL	7900000.00	SUCCESS	NULL	Chuyển tiền đồng thời [T1] không lock	2026-05-28 15:49:52
40	TRANSFER_IN	1000000.00	2	1	NULL	27100000.00	SUCCESS	NULL	Nhận tiền đồng thời [T1] không lock	2026-05-28 15:49:52
41	TRANSFER_OUT	1000000.00	1	2	NULL	6800000.00	SUCCESS	NULL	Chuyển tiền nội bộ	2026-05-28 17:27:57
42	TRANSFER_IN	1000000.00	2	1	NULL	28200000.00	SUCCESS	NULL	Nhận tiền nội bộ	2026-05-28 17:27:57
43	TRANSFER_OUT	1000000.00	1	2	NULL	9800000.00	SUCCESS	NULL	Deadlock demo: A→B thành công	2026-05-28 17:51:16
44	TRANSFER_IN	1000000.00	2	1	NULL	25200000.00	SUCCESS	NULL	Deadlock demo: nhận tiền từ A→B	2026-05-28 17:51:16
45	TRANSFER_OUT	1000000.00	1	2	NULL	8800000.00	SUCCESS	NULL	Deadlock demo: A→B thành công	2026-05-28 17:51:50
46	TRANSFER_IN	1000000.00	2	1	NULL	26200000.00	SUCCESS	NULL	Deadlock demo: nhận tiền từ A→B	2026-05-28 17:51:50
47	TRANSFER_OUT	1000000.00	1	2	NULL	7800000.00	SUCCESS	NULL	Deadlock demo: A→B thành công	2026-05-28 17:53:26
48	TRANSFER_IN	1000000.00	2	1	NULL	27200000.00	SUCCESS	NULL	Deadlock demo: nhận tiền từ A→B	2026-05-28 17:53:26
49	TRANSFER_OUT	2000000.00	1	2	NULL	5800000.00	SUCCESS	NULL	Deadlock demo: B→A thành công	2026-05-28 17:54:04
50	TRANSFER_IN	2000000.00	2	1	NULL	29200000.00	SUCCESS	NULL	Deadlock demo: nhận tiền từ B→A	2026-05-28 17:54:04
51	INTER_BRANCH_OUT	50000000.00	1	2	DN	5800000.00	FAILED	TXN-1779990874822	Inter branch transfer FAILED - Không đủ số dư	2026-05-28 17:54:34
52	INTER_BRANCH_OUT	5000000.00	1	2	DN	800000.00	SUCCESS	TXN-1779990895204	Inter branch transfer out	2026-05-28 17:54:55
53	DEPOSIT	14200000.00	1	NULL	NULL	15000000.00	SUCCESS	NULL	Nạp tiền	2026-05-29 09:36:39
54	WITHDRAW	9200000.00	1	NULL	NULL	5800000.00	SUCCESS	NULL	Rút tiền	2026-05-29 09:36:56
55	DEPOSIT	9200000.00	1	NULL	NULL	15000000.00	SUCCESS	NULL	Nạp tiền	2026-05-29 09:37:19
56	WITHDRAW	9200000.00	2	NULL	NULL	20000000.00	SUCCESS	NULL	Rút tiền	2026-05-29 09:40:31
57	TRANSFER_OUT	500000.00	1	2	NULL	14500000.00	SUCCESS	NULL	Chuyển tiền nội bộ	2026-05-29 09:43:07
58	TRANSFER_IN	500000.00	2	1	NULL	20500000.00	SUCCESS	NULL	Nhận tiền nội bộ	2026-05-29 09:43:07
59	INTER_BRANCH_OUT	5000000.00	1	2	HCM	9500000.00	SUCCESS	TXN-1780048217000	Inter branch transfer out	2026-05-29 09:50:17
60	INTER_BRANCH_OUT	5000000.00	1	2	HCM	4500000.00	SUCCESS	TXN-1780048610090	Inter branch transfer out	2026-05-29 09:56:50
61	DEPOSIT	11500000.00	1	NULL	NULL	16000000.00	SUCCESS	NULL	Nạp tiền	2026-05-29 09:57:20
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Hình 4.74 Bảng transaction_history tại site HN sau khi thực hiện chuyển tiền liên chi nhánh - không xảy ra sự cố

+Branch HCM

transaction...	transaction_type	amount	account_id	related_account...	related_branch...	balance_after	status	distributed_txn_id	description	created_at
1	DEPOSIT	90000000.00	1	NULL	NULL	90000000.00	SUCCESS	NULL	Nạp tiền ban đầu	2026-05-28 06:41:02
2	DEPOSIT	45000000.00	2	NULL	NULL	45000000.00	SUCCESS	NULL	Nạp tiền ban đầu	2026-05-28 06:41:02
3	DEPOSIT	120000000.00	3	NULL	NULL	120000000.00	SUCCESS	NULL	Nạp tiền ban đầu	2026-05-28 06:41:02
4	DEPOSIT	55000000.00	4	NULL	NULL	55000000.00	SUCCESS	NULL	Nạp tiền ban đầu	2026-05-28 06:41:02
5	DEPOSIT	70000000.00	5	NULL	NULL	70000000.00	SUCCESS	NULL	Nạp tiền ban đầu	2026-05-28 06:41:02
6	INTER_BRANCH_IN	5000000.00	1	1	HN	95000000.00	SUCCESS	TXN-1779951226414	Inter branch transfer in	2026-05-28 06:53:46
7	INTER_BRANCH_IN	5000000.00	1	1	HN	95000000.00	FAILED	TXN-1779951230396	Inter branch transfer FAILED - HCM server bị cr...	2026-05-28 06:53:50
8	INTER_BRANCH_IN	5000000.00	2	1	HN	50000000.00	SUCCESS	TXN-1780048217000	Inter branch transfer in	2026-05-29 09:50:17
9	INTER_BRANCH_IN	5000000.00	2	1	HN	55000000.00	SUCCESS	TXN-1780048610090	Inter branch transfer in	2026-05-29 09:56:50
10	INTER_BRANCH_IN	5000000.00	5	1	HN	75000000.00	SUCCESS	TXN-1780049021441	Inter branch transfer in	2026-05-29 10:03:41
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Hình 4.75 Bảng transaction_history tại site HCM sau khi thực hiện chuyển tiền liên chi nhánh - không xảy ra sự cố

- Bảng distributed_transaction_log

+Branch HN

txn_id	txn_type	status	source_bran...	dest_branch	amount	source_account...	dest_account...	error_message	created_at	updated_at
TXN-1779951226414	INTER_BRANCH_TRANSFER	COMMITTED	HN	HCM	5000000.00	1	1	NULL	2026-05-28 06:53:46	2026-05-28 06:53:46
TXN-1779951230396	INTER_BRANCH_TRANSFER	ABORTED	HN	HCM	5000000.00	1	1	HCM server bị crash	2026-05-28 06:53:50	2026-05-28 06:53:50
TXN-1779990874822	INTER_BRANCH_TRANSFER	ABORTED	HN	DN	50000000.00	1	2	Không đủ số dư	2026-05-28 17:54:34	2026-05-28 17:54:34
TXN-1779990895204	INTER_BRANCH_TRANSFER	COMMITTED	HN	DN	5000000.00	1	2	NULL	2026-05-28 17:54:55	2026-05-28 17:54:55
TXN-1780048217000	INTER_BRANCH_TRANSFER	COMMITTED	HN	HCM	5000000.00	1	2	NULL	2026-05-29 09:50:17	2026-05-29 09:50:17
TXN-1780048610090	INTER_BRANCH_TRANSFER	COMMITTED	HN	HCM	5000000.00	1	2	NULL	2026-05-29 09:56:50	2026-05-29 09:56:50
TXN-1780049021441	INTER_BRANCH_TRANSFER	COMMITTED	HN	HCM	5000000.00	1	5	NULL	2026-05-29 10:03:41	2026-05-29 10:03:41
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Hình 4.76 Bảng distributed_transaction_log tại site HN sau khi thực hiện chuyển tiền liên chi nhánh - không xảy ra sự cố

+Branch HCM

txn_id	txn_type	status	source_bran...	dest_branch	amount	source_account...	dest_account...	error_message	created_at	updated_at
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Hình 4.77 Bảng distributed_transaction_log tại site HCM sau khi thực hiện chuyển tiền liên chi nhánh - không xảy ra sự cố

- Bảng transaction_participant

  +Branch HN

id	txn_id	branch_id	role	status	action	created_at
1	TXN-1780048610090	HN	SOURCE	COMMITTED	DEBIT	2026-05-29 09:56:50
2	TXN-1780049021441	HN	SOURCE	COMMITTED	DEBIT	2026-05-29 10:03:41
3	TXN-1780049359647	HN	SOURCE	COMMITTED	DEBIT	2026-05-29 10:09:19
4	TXN-1780049359647	HCM	DESTINATION	COMMITTED	CREDIT	2026-05-29 10:09:19

Hình 4.78 Bảng transaction_participant tại site HN sau khi chuyển tiền liên chi nhánh - không xảy ra sự cố

+Branch HCM

id	txn_id	branch_id	role	status	action	created_at
NULL	NULL	NULL	NULL	NULL	NULL	NULL

Hình 4.79 Bảng transaction_participant tại site HCM sau khi chuyển tiền liên chi nhánh - không xảy ra sự cố

Kết quả in ra trên terminal:

```
==========================================
GIAO DỊCH 2PC LIÊN CHI NHÁNH
==========================================
Transaction ID : TXN-1780049853731
Từ chi nhánh   : HN
Đến chi nhánh  : HCM
Số tiền        : 5000000
==========================================
[TransactionLogger] Executed: 1 row(s) affected | SQL: INSERT INTO distributed_transaction_log (txn_id, txn_type, s...
[TransactionLogger] Executed: 1 row(s) affected | SQL: INSERT INTO transaction_participant (txn_id, branch_id, role...
[TransactionLogger] Executed: 1 row(s) affected | SQL: INSERT INTO transaction_participant (txn_id, branch_id, role...

STEP 1: SỐ DƯ BAN ĐẦU
HN - Account 1: 15000000.00
HCM - Account 5: 3000000.00

STEP 2: SOURCE TRỪ TIỀN
HN: 15000000.00 → 10000000.00
HCM: 3000000.00 (chưa thay đổi)
PREPARE SOURCE THÀNH CÔNG
[TransactionLogger] Executed: 1 row(s) affected | SQL: UPDATE transaction_participant SET status = ? WHERE txn_id =...

STEP 3: DEST CỘNG TIỀN
HN: 10000000.00
HCM: 3000000.00 → 8000000.00
PREPARE DEST THÀNH CÔNG
[TransactionLogger] Executed: 1 row(s) affected | SQL: UPDATE transaction_participant SET status = ? WHERE txn_id =...

TẤT CẢ SITE ĐÃ PREPARE THÀNH CÔNG
BẮT ĐẦU COMMIT
COMMIT SOURCE THÀNH CÔNG
[TransactionLogger] Executed: 1 row(s) affected | SQL: UPDATE transaction_participant SET status = ? WHERE txn_id =...
COMMIT DEST THÀNH CÔNG
[TransactionLogger] Executed: 1 row(s) affected | SQL: UPDATE transaction_participant SET status = ? WHERE txn_id =...
[TransactionLogger] Executed: 1 row(s) affected | SQL: UPDATE distributed_transaction_log SET status = ? WHERE txn_...

STEP 4: SỐ DƯ SAU COMMIT
HN - Account 1: 10000000.00
HCM - Account 5: 8000000.00
==========================================
TRANSACTION COMMITTED
==========================================
```

Hình 4.80 Kết quả mô phỏng quá trình chuyển tiền liên chi nhánh - không xảy ra sự cố

Kịch bản 2: Máy chủ đích (Destination) gặp sự cố mạng/Crash trong Phase 1 (2PC Rollback)

1. Mục tiêu kiểm thử

- Kiểm chứng khả năng chịu lỗi (Fault Tolerance) của giao thức 2PC.
- Đảm bảo nếu một trong các Database tham gia giao dịch bị sập hoặc mất kết nối, hệ thống phải tự động hoàn tác (Rollback) toàn cục để tránh hiện tượng mất tiền hoặc tiền bị treo lơ lửng.

2. Chuẩn bị kịch bản

- Chi nhánh nguồn: Hà Nội (HN), Tài khoản ID 1, Số dư hiện tại 10.000.000 VNĐ.
- Chi nhánh đích: TP.HCM (HCM), Tài khoản ID 5, Số dư hiện tại 8.000.000 VNĐ.

- Yêu cầu nghiệp vụ: Chuyển 2.000.000 VNĐ. Kích hoạt cờ giả lập lỗi simulateCrash = true tại nhánh đích.

3. Thực thi kiểm thử (API Request)

```bash
curl -X POST http://localhost:8080/api/transfers/inter-branch \
-H "Content-Type: application/json" \
-d '{
  "fromBranch": "HN",
  "fromAccountId": 1,
  "toBranch": "HCM",
  "toAccountId": 5,
  "amount": 2000000,
  "simulateCrash": true
}'
```

Hình 4.81 Kết quả trả về dưới dạng JSON thực hiện chuyển tiền liên chi nhánh - xảy ra lỗi

4. Kết quả và Luồng thực thi tự bảo vệ Hệ thống phát hiện lỗi và lập tức chặn giao dịch, trả về HTTP 200 OK (với cờ success: false trong payload báo hiệu nghiệp vụ thất bại). Log hệ thống phản ánh quá trình Rollback:

- **PHASE 1 (PREPARE):**
  - Máy chủ điều phối tiến hành trừ tiền ở chi nhánh nguồn (HN): Tài khoản ID 1 giảm từ 10.000.000 xuống 8.000.000. Lệnh UPDATE lưu vào bộ đệm của Transaction HN.
  - Tiến trình chuyển sang chi nhánh đích (HCM). Lúc này, Giả lập Crash kích hoạt (Ném ra ngoại lệ RuntimeException: HCM server bị crash). Giai đoạn PREPARE tại HCM thất bại hoàn toàn.
- PHASE 2 (ABORT & ROLLBACK):
  - Coordinator nhận được tín hiệu lỗi từ chi nhánh HCM trong Phase 1, lập tức kích hoạt chu trình Abort toàn cục.
  - Gửi lệnh rollback() tới chi nhánh HCM (Transaction bị hủy).
  - Gửi lệnh rollback() tới chi nhánh HN (Hoàn tác lệnh UPDATE ban nãy).

Đối soát số dư thực tế sau khi Rollback:

- Số dư tài khoản ID 1 (HN) lập tức khôi phục lại nguyên vẹn 10.000.000 VNĐ (Không hề bị mất đi 2.000.000 VNĐ dù lệnh UPDATE đã từng chạy qua).
- Số dư tài khoản ID 5 (HCM) vẫn giữ nguyên 8.000.000 VNĐ.
- Trạng thái bản ghi theo dõi giao dịch phân tán (Transaction Logger) cập nhật thành ABORTED.

5. Thực thi trên giao diện và CSDL

B1. CSDL trước khi thực hiện

- Bảng account

  +Branch HN

account_id	customer_id	branch_id	balance	status	created_at
1	1	HN	10000000.00	ACTIVE	2026-05-28 06:41:02
2	1	HN	20500000.00	ACTIVE	2026-05-28 06:41:02
3	2	HN	75000000.00	ACTIVE	2026-05-28 06:41:02
4	3	HN	30000000.00	ACTIVE	2026-05-28 06:41:02
5	4	HN	100000000.00	ACTIVE	2026-05-28 06:41:02
6	5	HN	15000000.00	ACTIVE	2026-05-28 06:41:02

Hình 4.82 Bảng account tại site HN trước khi thực hiện chuyển tiền liên chi nhánh - xảy ra lỗi

  +Branch HCM

account_id	customer_id	branch_id	balance	status	created_at
1	1	HCM	95000000.00	ACTIVE	2026-05-28 06:41:02
2	2	HCM	55000000.00	ACTIVE	2026-05-28 06:41:02
3	3	HCM	120000000.00	ACTIVE	2026-05-28 06:41:02
4	4	HCM	55000000.00	ACTIVE	2026-05-28 06:41:02
5	5	HCM	8000000.00	ACTIVE	2026-05-28 06:41:02
NULL	NULL	NULL	NULL	NULL	NULL

Hình 4.83 Bảng account tại site HCM trước khi thực hiện chuyển tiền liên chi nhánh - xảy ra lỗi

- Bảng transaction_history

  +Branch HN

transaction...	transaction_type	amount	account_id	related_account...	related_branch...	balance_after	status	distributed_txn_id	description	created_at
1	DEPOSIT	50000000.00	1	NULL	NULL	50000000.00	SUCCESS	NULL	Nạp tiền ban đầu	2026-05-28 06:41:02
2	DEPOSIT	20000000.00	2	NULL	NULL	20000000.00	SUCCESS	NULL	Nạp tiền ban đầu	2026-05-28 06:41:02
3	DEPOSIT	75000000.00	3	NULL	NULL	75000000.00	SUCCESS	NULL	Nạp tiền ban đầu	2026-05-28 06:41:02
4	DEPOSIT	30000000.00	4	NULL	NULL	30000000.00	SUCCESS	NULL	Nạp tiền ban đầu	2026-05-28 06:41:02
5	DEPOSIT	100000000.00	5	NULL	NULL	100000000.00	SUCCESS	NULL	Nạp tiền ban đầu	2026-05-28 06:41:02
6	DEPOSIT	15000000.00	6	NULL	NULL	15000000.00	SUCCESS	NULL	Nạp tiền ban đầu	2026-05-28 06:41:02
7	DEPOSIT	1000000.00	1	NULL	NULL	51000000.00	SUCCESS	NULL	Nạp tiền	2026-05-28 06:45:38
8	WITHDRAW	1000000.00	1	NULL	NULL	50000000.00	SUCCESS	NULL	Rút tiền	2026-05-28 06:53:24
9	INTER_BRANCH_OUT	5000000.00	1	1	HCM	45000000.00	SUCCESS	TXN-1779951226414	Inter branch transfer out	2026-05-28 06:53:46
10	INTER_BRANCH_OUT	5000000.00	1	1	HCM	45000000.00	FAILED	TXN-1779951230396	Inter branch transfer FAILED - HCM server bị cr...	2026-05-28 06:53:50
11	DEPOSIT	500000.00	1	NULL	NULL	45500000.00	SUCCESS	NULL	Nạp tiền	2026-05-28 10:52:01
12	WITHDRAW	500000.00	1	NULL	NULL	45000000.00	SUCCESS	NULL	Rút tiền	2026-05-28 10:58:46
13	DEPOSIT	500000.00	1	NULL	NULL	45500000.00	SUCCESS	NULL	Nạp tiền	2026-05-28 11:02:25
20	WITHDRAW	500000.00	1	NULL	NULL	45000000.00	SUCCESS	NULL	Rút tiền	2026-05-28 11:41:45
21	WITHDRAW	10000000.00	1	NULL	NULL	35000000.00	SUCCESS	NULL	Rút tiền concurrent không lock	2026-05-28 11:42:22
22	WITHDRAW	10000000.00	1	NULL	NULL	35000000.00	SUCCESS	NULL	Rút tiền concurrent không lock	2026-05-28 11:42:22
23	WITHDRAW	10000000.00	1	NULL	NULL	25000000.00	SUCCESS	NULL	Rút tiền concurrent có lock	2026-05-28 11:43:11
24	WITHDRAW	10000000.00	1	NULL	NULL	15000000.00	SUCCESS	NULL	Rút tiền concurrent có lock	2026-05-28 11:43:11
27	DEPOSIT	30500000.00	1	NULL	NULL	45500000.00	SUCCESS	NULL	Nạp tiền	2026-05-28 12:07:24
28	WITHDRAW	500000.00	1	NULL	NULL	45000000.00	SUCCESS	NULL	Rút tiền	2026-05-28 12:08:12
29	WITHDRAW	10000000.00	1	NULL	NULL	35000000.00	SUCCESS	NULL	Rút tiền concurrent không lock	2026-05-28 12:10:06
30	WITHDRAW	10000000.00	1	NULL	NULL	35000000.00	SUCCESS	NULL	Rút tiền concurrent không lock	2026-05-28 12:10:06
31	WITHDRAW	10000000.00	1	NULL	NULL	25000000.00	SUCCESS	NULL	Rút tiền concurrent có lock	2026-05-28 12:13:25
32	WITHDRAW	10000000.00	1	NULL	NULL	15000000.00	SUCCESS	NULL	Rút tiền concurrent có lock	2026-05-28 12:13:25
33	TRANSFER_OUT	5000000.00	1	2	NULL	10000000.00	SUCCESS	NULL	Chuyển tiền nội bộ	2026-05-28 12:16:32
34	TRANSFER_IN	5000000.00	2	1	NULL	25000000.00	SUCCESS	NULL	Nhận tiền nội bộ	2026-05-28 12:16:32
35	TRANSFER_OUT	100000.00	1	2	NULL	9900000.00	SUCCESS	NULL	Chuyển tiền đồng thời [T1] không lock	2026-05-28 15:35:02
36	TRANSFER_IN	100000.00	2	1	NULL	25100000.00	SUCCESS	NULL	Nhận tiền đồng thời [T1] không lock	2026-05-28 15:35:02
37	TRANSFER_OUT	1000000.00	1	2	NULL	8900000.00	SUCCESS	NULL	Chuyển tiền đồng thời [T1] không lock	2026-05-28 15:46:09
38	TRANSFER_IN	1000000.00	2	1	NULL	26100000.00	SUCCESS	NULL	Nhận tiền đồng thời [T1] không lock	2026-05-28 15:46:09
39	TRANSFER_OUT	1000000.00	1	2	NULL	7900000.00	SUCCESS	NULL	Chuyển tiền đồng thời [T1] không lock	2026-05-28 15:49:52
40	TRANSFER_IN	1000000.00	2	1	NULL	27100000.00	SUCCESS	NULL	Nhận tiền đồng thời [T1] không lock	2026-05-28 15:49:52
41	TRANSFER_OUT	1000000.00	1	2	NULL	6800000.00	SUCCESS	NULL	Chuyển tiền nội bộ	2026-05-28 17:27:57
42	TRANSFER_IN	1000000.00	2	1	NULL	28200000.00	SUCCESS	NULL	Nhận tiền nội bộ	2026-05-28 17:27:57
43	TRANSFER_OUT	1000000.00	1	2	NULL	9800000.00	SUCCESS	NULL	Deadlock demo: A→B thành công	2026-05-28 17:51:16
44	TRANSFER_IN	1000000.00	2	1	NULL	25200000.00	SUCCESS	NULL	Deadlock demo: nhận tiền từ A→B	2026-05-28 17:51:16
45	TRANSFER_OUT	1000000.00	1	2	NULL	8800000.00	SUCCESS	NULL	Deadlock demo: A→B thành công	2026-05-28 17:51:50
46	TRANSFER_IN	1000000.00	2	1	NULL	26200000.00	SUCCESS	NULL	Deadlock demo: nhận tiền từ A→B	2026-05-28 17:51:50
47	TRANSFER_OUT	1000000.00	1	2	NULL	7800000.00	SUCCESS	NULL	Deadlock demo: A→B thành công	2026-05-28 17:53:26
48	TRANSFER_IN	1000000.00	2	1	NULL	27200000.00	SUCCESS	NULL	Deadlock demo: nhận tiền từ A→B	2026-05-28 17:53:26
49	TRANSFER_OUT	2000000.00	1	2	NULL	5800000.00	SUCCESS	NULL	Deadlock demo: B→A thành công	2026-05-28 17:54:04
50	TRANSFER_IN	2000000.00	2	1	NULL	29200000.00	SUCCESS	NULL	Deadlock demo: nhận tiền từ B→A	2026-05-28 17:54:04
51	INTER_BRANCH_OUT	50000000.00	1	2	DN	5800000.00	FAILED	TXN-1779990874822	Inter branch transfer FAILED - Không đủ số dư	2026-05-28 17:54:34
52	INTER_BRANCH_OUT	5000000.00	1	2	DN	800000.00	SUCCESS	TXN-1779990895204	Inter branch transfer out	2026-05-28 17:54:55
53	DEPOSIT	14200000.00	1	NULL	NULL	15000000.00	SUCCESS	NULL	Nạp tiền	2026-05-29 09:36:39
54	WITHDRAW	9200000.00	1	NULL	NULL	5800000.00	SUCCESS	NULL	Rút tiền	2026-05-29 09:36:56
55	DEPOSIT	9200000.00	1	NULL	NULL	15000000.00	SUCCESS	NULL	Nạp tiền	2026-05-29 09:37:19
56	WITHDRAW	9200000.00	2	NULL	NULL	20000000.00	SUCCESS	NULL	Rút tiền	2026-05-29 09:40:31
57	TRANSFER_OUT	500000.00	1	2	NULL	14500000.00	SUCCESS	NULL	Chuyển tiền nội bộ	2026-05-29 09:43:07
58	TRANSFER_IN	500000.00	2	1	NULL	20500000.00	SUCCESS	NULL	Nhận tiền nội bộ	2026-05-29 09:43:07
59	INTER_BRANCH_OUT	5000000.00	1	2	HCM	9500000.00	SUCCESS	TXN-1780048217000	Inter branch transfer out	2026-05-29 09:50:17
60	INTER_BRANCH_OUT	5000000.00	1	2	HCM	4500000.00	SUCCESS	TXN-1780048610090	Inter branch transfer out	2026-05-29 09:56:50
61	DEPOSIT	11500000.00	1	NULL	NULL	16000000.00	SUCCESS	NULL	Nạp tiền	2026-05-29 09:57:20
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Hình 4.84 Bảng transaction_history tại site HN trước khi thực hiện chuyển tiền liên chi nhánh - xảy ra lỗi

+Branch HCM

transaction...	transaction_type	amount	account_id	related_account...	related_branch...	balance_after	status	distributed_txn_id	description	created_at
1	DEPOSIT	90000000.00	1	NULL	NULL	90000000.00	SUCCESS	NULL	Nạp tiền ban đầu	2026-05-28 06:41:02
2	DEPOSIT	45000000.00	2	NULL	NULL	45000000.00	SUCCESS	NULL	Nạp tiền ban đầu	2026-05-28 06:41:02
3	DEPOSIT	120000000.00	3	NULL	NULL	120000000.00	SUCCESS	NULL	Nạp tiền ban đầu	2026-05-28 06:41:02
4	DEPOSIT	55000000.00	4	NULL	NULL	55000000.00	SUCCESS	NULL	Nạp tiền ban đầu	2026-05-28 06:41:02
5	DEPOSIT	70000000.00	5	NULL	NULL	70000000.00	SUCCESS	NULL	Nạp tiền ban đầu	2026-05-28 06:41:02
6	INTER_BRANCH_IN	5000000.00	1	1	HN	95000000.00	SUCCESS	TXN-1779951226414	Inter branch transfer in	2026-05-28 06:53:46
7	INTER_BRANCH_IN	5000000.00	1	1	HN	95000000.00	FAILED	TXN-1779951230396	Inter branch transfer FAILED - HCM server bị cr...	2026-05-28 06:53:50
8	INTER_BRANCH_IN	5000000.00	2	1	HN	50000000.00	SUCCESS	TXN-1780048217000	Inter branch transfer in	2026-05-29 09:50:17
9	INTER_BRANCH_IN	5000000.00	2	1	HN	55000000.00	SUCCESS	TXN-1780048610090	Inter branch transfer in	2026-05-29 09:56:50
10	INTER_BRANCH_IN	5000000.00	5	1	HN	75000000.00	SUCCESS	TXN-1780049021441	Inter branch transfer in	2026-05-29 10:03:41
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Hình 4.85 Bảng transaction_history tại site HCM trước khi thực hiện chuyển tiền liên chi nhánh - xảy ra lỗi

- Bảng distributed_transaction_log

+Branch HN

txn_id	txn_type	status	source_bran...	dest_branch	amount	source_account...	dest_account...	error_message	created_at	updated_at
TXN-1779951226414	INTER_BRANCH_TRANSFER	COMMITTED	HN	HCM	5000000.00	1	1	NULL	2026-05-28 06:53:46	2026-05-28 06:53:46
TXN-1779951230396	INTER_BRANCH_TRANSFER	ABORTED	HN	HCM	5000000.00	1	1	HCM server bị crash	2026-05-28 06:53:50	2026-05-28 06:53:50
TXN-1779990874822	INTER_BRANCH_TRANSFER	ABORTED	HN	DN	50000000.00	1	2	Không đủ số dư	2026-05-28 17:54:34	2026-05-28 17:54:34
TXN-1779990895204	INTER_BRANCH_TRANSFER	COMMITTED	HN	DN	5000000.00	1	2	NULL	2026-05-28 17:54:55	2026-05-28 17:54:55
TXN-1780048217000	INTER_BRANCH_TRANSFER	COMMITTED	HN	HCM	5000000.00	1	2	NULL	2026-05-29 09:50:17	2026-05-29 09:50:17
TXN-1780048610090	INTER_BRANCH_TRANSFER	COMMITTED	HN	HCM	5000000.00	1	2	NULL	2026-05-29 09:56:50	2026-05-29 09:56:50
TXN-1780049021441	INTER_BRANCH_TRANSFER	COMMITTED	HN	HCM	5000000.00	1	5	NULL	2026-05-29 10:03:41	2026-05-29 10:03:41
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Hình 4.86 Bảng distributed_transaction_log tại site HN trước khi thực hiện chuyển tiền liên chi nhánh - xảy ra lỗi

+Branch HCM

txn_id	txn_type	status	source_bran...	dest_branch	amount	source_account...	dest_account...	error_message	created_at	updated_at
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Hình 4.87 Bảng distributed_transaction_log tại site HCM trước khi thực hiện chuyển tiền liên chi nhánh - xảy ra lỗi

- Bảng transaction_participant

+Branch HN

id	txn_id	branch_id	role	status	action	created_at
1	TXN-1780048610090	HN	SOURCE	COMMITTED	DEBIT	2026-05-29 09:56:50
2	TXN-1780049021441	HN	SOURCE	COMMITTED	DEBIT	2026-05-29 10:03:41
3	TXN-1780049359647	HN	SOURCE	COMMITTED	DEBIT	2026-05-29 10:09:19
4	TXN-1780049359647	HCM	DESTINATION	COMMITTED	CREDIT	2026-05-29 10:09:19

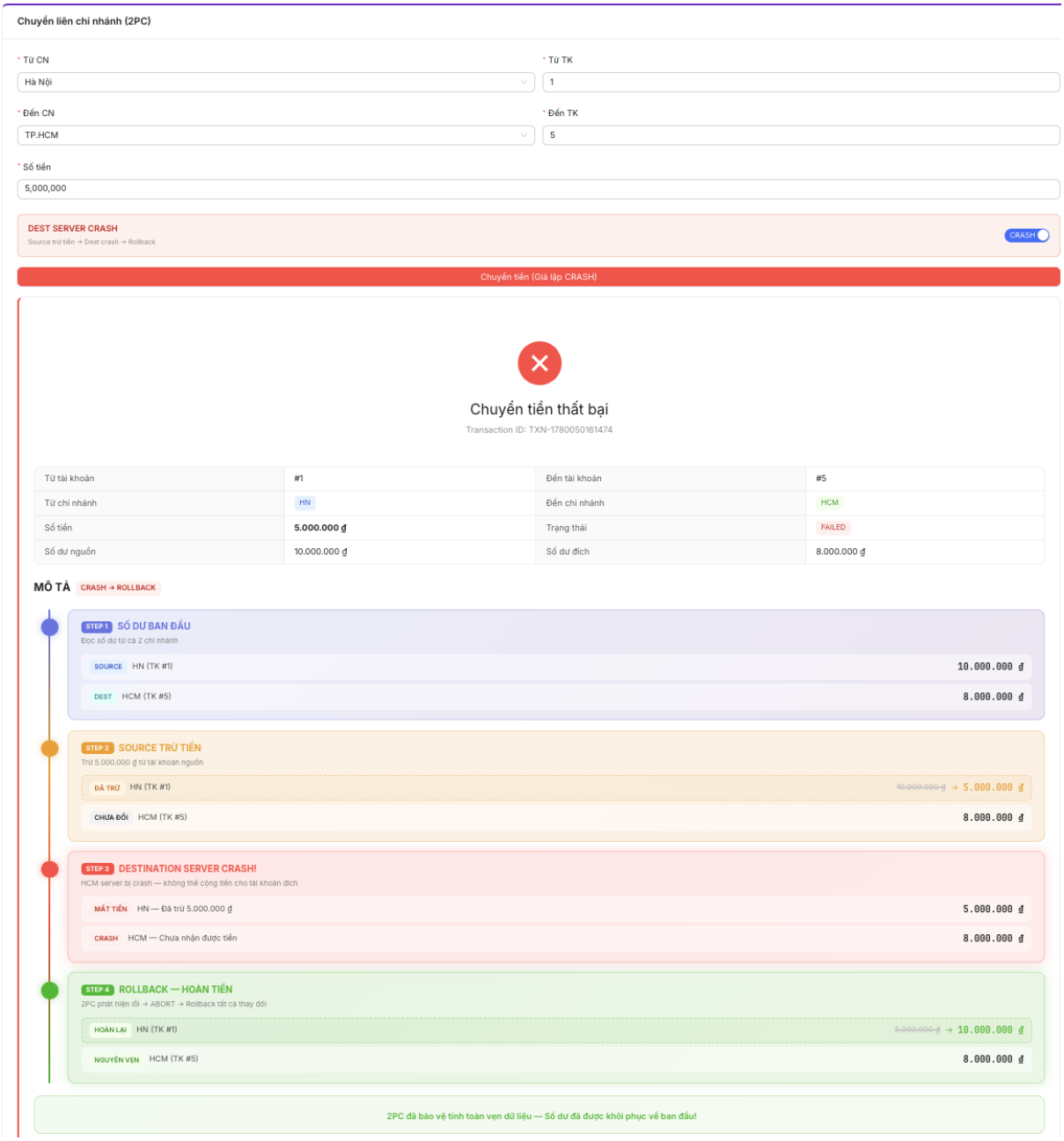
Hình 4.88 Bảng transaction_participant tại site HN trước khi thực hiện chuyển tiền liên chi nhánh - xảy ra lỗi

+Branch HCM

id	txn_id	branch_id	role	status	action	created_at
NULL	NULL	NULL	NULL	NULL	NULL	NULL

Hình 4.89 Bảng transaction_participant tại site HCM trước khi thực hiện chuyển tiền liên chi nhánh - xảy ra lỗi

B2. Thao tác trên giao diện



Hình 4.90 Thực hiện chuyển tiền liên chi nhánh trên giao diện - xảy ra lỗi

B3. CSDL sau khi thao tác

- Bảng account

  +Branch HN

account_id	customer_id	branch_id	balance	status	created_at
1	1	HN	10000000.00	ACTIVE	2026-05-28 06:41:02
2	1	HN	20500000.00	ACTIVE	2026-05-28 06:41:02
3	2	HN	75000000.00	ACTIVE	2026-05-28 06:41:02
4	3	HN	30000000.00	ACTIVE	2026-05-28 06:41:02
5	4	HN	100000000.00	ACTIVE	2026-05-28 06:41:02
6	5	HN	15000000.00	ACTIVE	2026-05-28 06:41:02

Hình 4.91 Bảng account tại site HN sau khi thực hiện chuyển tiền liên chi nhánh - xảy ra lỗi

+Branch HCM

account_id	customer_id	branch_id	balance	status	created_at
1	1	HCM	95000000.00	ACTIVE	2026-05-28 06:41:02
2	2	HCM	55000000.00	ACTIVE	2026-05-28 06:41:02
3	3	HCM	120000000.00	ACTIVE	2026-05-28 06:41:02
4	4	HCM	55000000.00	ACTIVE	2026-05-28 06:41:02
5	5	HCM	8000000.00	ACTIVE	2026-05-28 06:41:02
NULL	NULL	NULL	NULL	NULL	NULL

Hình 4.92 Bảng account tại site HCM sau khi thực hiện chuyển tiền liên chi nhánh - xảy ra lỗi

- Bảng transaction_history

+Branch HN

transaction...	transaction_type	amount	account_id	related_account...	related_branch...	balance_after	status	distributed_txn_id	description	created_at
5	DEPOSIT	100000000.00	5	NULL	NULL	100000000.00	SUCCESS	NULL	Nạp tiền ban đầu	2026-05-28 06:41:02
6	DEPOSIT	15000000.00	6	NULL	NULL	15000000.00	SUCCESS	NULL	Nạp tiền ban đầu	2026-05-28 06:41:02
7	DEPOSIT	1000000.00	1	NULL	NULL	51000000.00	SUCCESS	NULL	Nạp tiền	2026-05-28 06:45:38
8	WITHDRAW	1000000.00	1	NULL	NULL	50000000.00	SUCCESS	NULL	Rút tiền	2026-05-28 06:53:24
9	INTER_BRANCH_OUT	5000000.00	1	1	HCM	45000000.00	SUCCESS	TXN-1779951226414	Inter branch transfer out	2026-05-28 06:53:46
10	INTER_BRANCH_OUT	5000000.00	1	1	HCM	45000000.00	FAILED	TXN-1779951230396	Inter branch transfer FAILED - HCM server bị cr...	2026-05-28 06:53:50
11	DEPOSIT	500000.00	1	NULL	NULL	45500000.00	SUCCESS	NULL	Nạp tiền	2026-05-28 10:52:01
12	WITHDRAW	500000.00	1	NULL	NULL	45000000.00	SUCCESS	NULL	Rút tiền	2026-05-28 10:58:46
13	DEPOSIT	500000.00	1	NULL	NULL	45500000.00	SUCCESS	NULL	Nạp tiền	2026-05-28 11:02:25
20	WITHDRAW	500000.00	1	NULL	NULL	45000000.00	SUCCESS	NULL	Rút tiền	2026-05-28 11:41:45
21	WITHDRAW	10000000.00	1	NULL	NULL	35000000.00	SUCCESS	NULL	Rút tiền concurrent không lock	2026-05-28 11:42:22
22	WITHDRAW	10000000.00	1	NULL	NULL	35000000.00	SUCCESS	NULL	Rút tiền concurrent không lock	2026-05-28 11:42:22
23	WITHDRAW	10000000.00	1	NULL	NULL	25000000.00	SUCCESS	NULL	Rút tiền concurrent có lock	2026-05-28 11:43:11
24	WITHDRAW	10000000.00	1	NULL	NULL	15000000.00	SUCCESS	NULL	Rút tiền concurrent có lock	2026-05-28 11:43:11
27	DEPOSIT	30500000.00	1	NULL	NULL	45500000.00	SUCCESS	NULL	Nạp tiền	2026-05-28 12:07:24
28	WITHDRAW	500000.00	1	NULL	NULL	45000000.00	SUCCESS	NULL	Rút tiền	2026-05-28 12:08:12
29	WITHDRAW	10000000.00	1	NULL	NULL	35000000.00	SUCCESS	NULL	Rút tiền concurrent không lock	2026-05-28 12:10:06
30	WITHDRAW	10000000.00	1	NULL	NULL	35000000.00	SUCCESS	NULL	Rút tiền concurrent không lock	2026-05-28 12:10:06
31	WITHDRAW	10000000.00	1	NULL	NULL	25000000.00	SUCCESS	NULL	Rút tiền concurrent có lock	2026-05-28 12:13:25
32	WITHDRAW	10000000.00	1	NULL	NULL	15000000.00	SUCCESS	NULL	Rút tiền concurrent có lock	2026-05-28 12:13:25
33	TRANSFER_OUT	5000000.00	1	2	NULL	10000000.00	SUCCESS	NULL	Chuyển tiền nội bộ	2026-05-28 12:16:32
34	TRANSFER_IN	5000000.00	2	1	NULL	25000000.00	SUCCESS	NULL	Nhận tiền nội bộ	2026-05-28 12:16:32
35	TRANSFER_OUT	100000.00	1	2	NULL	9900000.00	SUCCESS	NULL	Chuyển tiền đồng thời [T1] không lock	2026-05-28 15:35:02
36	TRANSFER_IN	100000.00	2	1	NULL	25100000.00	SUCCESS	NULL	Nhận tiền đồng thời [T1] không lock	2026-05-28 15:35:02
37	TRANSFER_OUT	1000000.00	1	2	NULL	8900000.00	SUCCESS	NULL	Chuyển tiền đồng thời [T1] không lock	2026-05-28 15:46:09
38	TRANSFER_IN	1000000.00	2	1	NULL	26100000.00	SUCCESS	NULL	Nhận tiền đồng thời [T1] không lock	2026-05-28 15:46:09
39	TRANSFER_OUT	1000000.00	1	2	NULL	7900000.00	SUCCESS	NULL	Chuyển tiền đồng thời [T1] không lock	2026-05-28 15:49:52
40	TRANSFER_IN	1000000.00	2	1	NULL	27100000.00	SUCCESS	NULL	Nhận tiền đồng thời [T1] không lock	2026-05-28 15:49:52
41	TRANSFER_OUT	1000000.00	1	2	NULL	6800000.00	SUCCESS	NULL	Chuyển tiền nội bộ	2026-05-28 17:27:57
42	TRANSFER_IN	1000000.00	2	1	NULL	28200000.00	SUCCESS	NULL	Nhận tiền nội bộ	2026-05-28 17:27:57
43	TRANSFER_OUT	1000000.00	1	2	NULL	9800000.00	SUCCESS	NULL	Deadlock demo: A→B thành công	2026-05-28 17:51:16
44	TRANSFER_IN	1000000.00	2	1	NULL	25200000.00	SUCCESS	NULL	Deadlock demo: nhận tiền từ A→B	2026-05-28 17:51:16
45	TRANSFER_OUT	1000000.00	1	2	NULL	8800000.00	SUCCESS	NULL	Deadlock demo: A→B thành công	2026-05-28 17:51:50
46	TRANSFER_IN	1000000.00	2	1	NULL	26200000.00	SUCCESS	NULL	Deadlock demo: nhận tiền từ A→B	2026-05-28 17:51:50
47	TRANSFER_OUT	1000000.00	1	2	NULL	7800000.00	SUCCESS	NULL	Deadlock demo: A→B thành công	2026-05-28 17:53:26
48	TRANSFER_IN	1000000.00	2	1	NULL	27200000.00	SUCCESS	NULL	Deadlock demo: nhận tiền từ A→B	2026-05-28 17:53:26
49	TRANSFER_OUT	2000000.00	1	2	NULL	5800000.00	SUCCESS	NULL	Deadlock demo: B→A thành công	2026-05-28 17:54:04
50	TRANSFER_IN	2000000.00	2	1	NULL	29200000.00	SUCCESS	NULL	Deadlock demo: nhận tiền từ B→A	2026-05-28 17:54:04
51	INTER_BRANCH_OUT	50000000.00	1	2	DN	5800000.00	FAILED	TXN-1779990874822	Inter branch transfer FAILED - Không đủ số dư	2026-05-28 17:54:34
52	INTER_BRANCH_OUT	5000000.00	1	2	DN	800000.00	SUCCESS	TXN-1779990895204	Inter branch transfer out	2026-05-28 17:54:55
53	DEPOSIT	14200000.00	1	NULL	NULL	15000000.00	SUCCESS	NULL	Nạp tiền	2026-05-29 09:36:39
54	WITHDRAW	9200000.00	1	NULL	NULL	5800000.00	SUCCESS	NULL	Rút tiền	2026-05-29 09:36:56
55	DEPOSIT	9200000.00	1	NULL	NULL	15000000.00	SUCCESS	NULL	Nạp tiền	2026-05-29 09:37:19
56	WITHDRAW	9200000.00	2	NULL	NULL	20000000.00	SUCCESS	NULL	Rút tiền	2026-05-29 09:40:31
57	TRANSFER_OUT	500000.00	1	2	NULL	14500000.00	SUCCESS	NULL	Chuyển tiền nội bộ	2026-05-29 09:43:07
58	TRANSFER_IN	500000.00	2	1	NULL	20500000.00	SUCCESS	NULL	Nhận tiền nội bộ	2026-05-29 09:43:07
59	INTER_BRANCH_OUT	5000000.00	1	2	HCM	9500000.00	SUCCESS	TXN-1780048217000	Inter branch transfer out	2026-05-29 09:50:17
60	INTER_BRANCH_OUT	5000000.00	1	2	HCM	4500000.00	SUCCESS	TXN-1780048610090	Inter branch transfer out	2026-05-29 09:56:50
61	DEPOSIT	11500000.00	1	NULL	NULL	16000000.00	SUCCESS	NULL	Nạp tiền	2026-05-29 09:57:20
64	INTER_BRANCH_OUT	5000000.00	1	5	HCM	10000000.00	SUCCESS	TXN-1780049359647	Inter branch transfer out	2026-05-29 10:09:19
65	INTER_BRANCH_OUT	5000000.00	1	5	HCM	5000000.00	SUCCESS	TXN-1780049642198	Inter branch transfer out	2026-05-29 10:14:02
66	INTER_BRANCH_OUT	5000000.00	1	5	HCM	10000000.00	SUCCESS	TXN-1780049853731	Inter branch transfer out	2026-05-29 10:17:33
67	INTER_BRANCH_OUT	5000000.00	1	5	HCM	10000000.00	FAILED	TXN-1780050161474	Inter branch transfer FAILED - HCM server bị cr...	2026-05-29 10:22:41
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Hình 4.93 Bảng transaction_history tại site HN sau khi thực hiện chuyển tiền liên chi nhánh - xảy ra lỗi

+Branch HCM

transaction...	transaction_type	amount	account_id	related_account...	related_branch...	balance_after	status	distributed_txn_id	description	created_at
1	DEPOSIT	90000000.00	1	NULL	NULL	90000000.00	SUCCESS	NULL	Nạp tiền ban đầu	2026-05-28 06:41:02
2	DEPOSIT	45000000.00	2	NULL	NULL	45000000.00	SUCCESS	NULL	Nạp tiền ban đầu	2026-05-28 06:41:02
3	DEPOSIT	120000000.00	3	NULL	NULL	120000000.00	SUCCESS	NULL	Nạp tiền ban đầu	2026-05-28 06:41:02
4	DEPOSIT	55000000.00	4	NULL	NULL	55000000.00	SUCCESS	NULL	Nạp tiền ban đầu	2026-05-28 06:41:02
5	DEPOSIT	70000000.00	5	NULL	NULL	70000000.00	SUCCESS	NULL	Nạp tiền ban đầu	2026-05-28 06:41:02
6	INTER_BRANCH_IN	5000000.00	1	1	HN	95000000.00	SUCCESS	TXN-1779951226414	Inter branch transfer in	2026-05-28 06:53:46
7	INTER_BRANCH_IN	5000000.00	1	1	HN	95000000.00	FAILED	TXN-1779951230396	Inter branch transfer FAILED - HCM server bị cr...	2026-05-28 06:53:50
8	INTER_BRANCH_IN	5000000.00	2	1	HN	50000000.00	SUCCESS	TXN-1780048217000	Inter branch transfer in	2026-05-29 09:50:17
9	INTER_BRANCH_IN	5000000.00	2	1	HN	55000000.00	SUCCESS	TXN-1780048610090	Inter branch transfer in	2026-05-29 09:56:50
10	INTER_BRANCH_IN	5000000.00	5	1	HN	75000000.00	SUCCESS	TXN-1780049021441	Inter branch transfer in	2026-05-29 10:03:41
11	INTER_BRANCH_IN	5000000.00	5	1	HN	8000000.00	SUCCESS	TXN-1780049359647	Inter branch transfer in	2026-05-29 10:09:19
12	INTER_BRANCH_IN	5000000.00	5	1	HN	13000000.00	SUCCESS	TXN-1780049642198	Inter branch transfer in	2026-05-29 10:14:02
13	INTER_BRANCH_IN	5000000.00	5	1	HN	8000000.00	SUCCESS	TXN-1780049853731	Inter branch transfer in	2026-05-29 10:17:33
14	INTER_BRANCH_IN	5000000.00	5	1	HN	8000000.00	FAILED	TXN-1780050161474	Inter branch transfer FAILED - HCM server bị cr...	2026-05-29 10:22:41
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Hình 4.94 Bảng transaction_history tại site HCM sau khi thực hiện chuyển tiền liên chi nhánh - xảy ra lỗi

- Bảng distributed_transaction_log

+Branch HN

txn_id	txn_type	status	source_bran...	dest_branch	amount	source_account...	dest_account...	error_message	created_at	updated_at
TXN-1779951226414	INTER_BRANCH_TRANSFER	COMMITTED	HN	HCM	5000000.00	1	1	NULL	2026-05-28 06:53:46	2026-05-28 06:53:46
TXN-1779951230396	INTER_BRANCH_TRANSFER	ABORTED	HN	HCM	5000000.00	1	1	HCM server bị crash	2026-05-28 06:53:50	2026-05-28 06:53:50
TXN-1779990874822	INTER_BRANCH_TRANSFER	ABORTED	HN	DN	50000000.00	1	2	Không đủ số dư	2026-05-28 17:54:34	2026-05-28 17:54:34
TXN-1779990895204	INTER_BRANCH_TRANSFER	COMMITTED	HN	DN	5000000.00	1	2	NULL	2026-05-28 17:54:55	2026-05-28 17:54:55
TXN-1780048217000	INTER_BRANCH_TRANSFER	COMMITTED	HN	HCM	5000000.00	1	2	NULL	2026-05-29 09:50:17	2026-05-29 09:50:17
TXN-1780048610090	INTER_BRANCH_TRANSFER	COMMITTED	HN	HCM	5000000.00	1	2	NULL	2026-05-29 09:56:50	2026-05-29 09:56:50
TXN-1780049021441	INTER_BRANCH_TRANSFER	COMMITTED	HN	HCM	5000000.00	1	5	NULL	2026-05-29 10:03:41	2026-05-29 10:03:41
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Hình 4.95 Bảng distributed_transaction_log tại site HN sau khi thực hiện chuyển tiền liên chi nhánh - xảy ra lỗi

+Branch HCM

txn_id	txn_type	status	source_bran...	dest_branch	amount	source_account...	dest_account...	error_message	created_at	updated_at
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Hình 4.96 Bảng distributed_transaction_log tại site HCM sau khi thực hiện chuyển tiền liên chi nhánh - xảy ra lỗi

- Bảng transaction_participant

  +Branch HN

txn_id	txn_type	status	source_bran...	dest_branch	amount	source_account...	dest_account...	error_message	created_at	updated_at
TXN-1779951226414	INTER_BRANCH_TRANSFER	COMMITTED	HN	HCM	5000000.00	1	1	NULL	2026-05-28 06:53:46	2026-05-28 06:53:46
TXN-1779951230396	INTER_BRANCH_TRANSFER	ABORTED	HN	HCM	5000000.00	1	1	HCM server bị crash	2026-05-28 06:53:50	2026-05-28 06:53:50
TXN-1779990874822	INTER_BRANCH_TRANSFER	ABORTED	HN	DN	50000000.00	1	2	Không đủ số dư	2026-05-28 17:54:34	2026-05-28 17:54:34
TXN-1779990895204	INTER_BRANCH_TRANSFER	COMMITTED	HN	DN	5000000.00	1	2	NULL	2026-05-28 17:54:55	2026-05-28 17:54:55
TXN-1780048217000	INTER_BRANCH_TRANSFER	COMMITTED	HN	HCM	5000000.00	1	2	NULL	2026-05-29 09:50:17	2026-05-29 09:50:17
TXN-1780048610090	INTER_BRANCH_TRANSFER	COMMITTED	HN	HCM	5000000.00	1	2	NULL	2026-05-29 09:56:50	2026-05-29 09:56:50
TXN-1780049021441	INTER_BRANCH_TRANSFER	COMMITTED	HN	HCM	5000000.00	1	5	NULL	2026-05-29 10:03:41	2026-05-29 10:03:41
TXN-1780049359647	INTER_BRANCH_TRANSFER	COMMITTED	HN	HCM	5000000.00	1	5	NULL	2026-05-29 10:09:19	2026-05-29 10:09:19
TXN-1780049642198	INTER_BRANCH_TRANSFER	COMMITTED	HN	HCM	5000000.00	1	5	NULL	2026-05-29 10:14:02	2026-05-29 10:14:02
TXN-1780049853731	INTER_BRANCH_TRANSFER	COMMITTED	HN	HCM	5000000.00	1	5	NULL	2026-05-29 10:17:33	2026-05-29 10:17:33
TXN-1780050161474	INTER_BRANCH_TRANSFER	ABORTED	HN	HCM	5000000.00	1	5	HCM server bị crash	2026-05-29 10:22:41	2026-05-29 10:22:41
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Hình 4.97 Bảng transaction_participant tại site HN sau khi thực hiện chuyển tiền liên chi nhánh - xảy ra lỗi

  +Branch HCM

id	txn_id	branch_id	role	status	action	created_at
NULL	NULL	NULL	NULL	NULL	NULL	NULL

Hình 4.98 Bảng transaction_participant tại site HCM sau khi thực hiện chuyển tiền liên chi nhánh - xảy ra lỗi

Kết quả in ra trên terminal:

```
GIAO DỊCH 2PC LIÊN CHI NHÁNH
==========================================
Transaction ID : TXN-1780050161474
Từ chi nhánh   : HN
Đến chi nhánh  : HCM
Số tiền        : 5000000
==========================================
[TransactionLogger] Executed: 1 row(s) affected | SQL: INSERT INTO distributed_transaction_log (txn_id, txn_type, s...
[TransactionLogger] Executed: 1 row(s) affected | SQL: INSERT INTO transaction_participant (txn_id, branch_id, role...
[TransactionLogger] Executed: 1 row(s) affected | SQL: INSERT INTO transaction_participant (txn_id, branch_id, role...

STEP 1: SỐ DƯ BAN ĐẦU
HN - Account 1: 10000000.00
HCM - Account 5: 8000000.00

STEP 2: SOURCE TRỪ TIỀN
HN: 10000000.00 → 5000000.00
HCM: 8000000.00 (chưa thay đổi)
PREPARE SOURCE THÀNH CÔNG
[TransactionLogger] Executed: 1 row(s) affected | SQL: UPDATE transaction_participant SET status = ? WHERE txn_id =...

DESTINATION SERVER BỊ CRASH
HCM server bị crash
Source đã trừ tiền nhưng Dest chưa cộng

LỖI: HCM server bị crash
ABORT TRANSACTION
ROLLBACK SOURCE THÀNH CÔNG
[TransactionLogger] Executed: 1 row(s) affected | SQL: UPDATE transaction_participant SET status = ? WHERE txn_id =...
ROLLBACK DEST THÀNH CÔNG
[TransactionLogger] Executed: 1 row(s) affected | SQL: UPDATE transaction_participant SET status = ? WHERE txn_id =...
[TransactionLogger] Executed: 1 row(s) affected | SQL: UPDATE distributed_transaction_log SET status = ?, error_mes...

SỐ DƯ SAU ROLLBACK
HN - Account 1: 10000000.00
HCM - Account 5: 8000000.00
GHI LỊCH SỬ THẤT BẠI TẠI SOURCE THÀNH CÔNG
GHI LỊCH SỬ THẤT BẠI TẠI DEST THÀNH CÔNG
==========================================
TRANSACTION ABORTED
==========================================
```

Hình 4.99 Mô phỏng kết quả chuyển tiền liên chi nhánh - xảy ra lỗi

Cơ chế 2-Phase Commit đã thực hiện xuất sắc nhiệm vụ bảo vệ an toàn dòng tiền. Mặc dù chi nhánh nguồn (HN) đã thực hiện trừ tiền thành công trong bộ đệm, nhưng vì chi nhánh đích (HCM) gặp sự cố, hệ thống đã thông minh loại bỏ hoàn toàn giao dịch, đảm bảo không có bất kỳ khách hàng nào bị thiệt hại về tài sản do lỗi hạ tầng. Tính nguyên tử (All-or-Nothing) của CSDL phân tán được bảo toàn 100%.

4.7 Các truy vấn phân tán (Distributed Queries)

Hệ thống triển khai giải pháp **Application-Level Merge** (Mô phỏng Map-Reduce) thông qua lớp DistributedQueryService. Nguyên lý hoạt động chung gồm ba bước:

- **Map (Phân tán):** Gửi đồng thời các Local Query bằng ngôn ngữ SQL thuần tới 3 Site (Hà Nội, Đà Nẵng, TP.HCM).
- **Reduce (Tổng hợp):** Thu thập danh sách kết quả (ResultSet) từ các Site đang hoạt động.
- **Merge & Sort:** Xử lý dữ liệu bằng Java Collections/Streams để tạo ra kết quả cuối cùng trả về cho người dùng.

Dưới đây là chi tiết 9 nghiệp vụ truy vấn phân tán được triển khai trong hệ thống.

1. Tổng số dư (Total Global Balance)

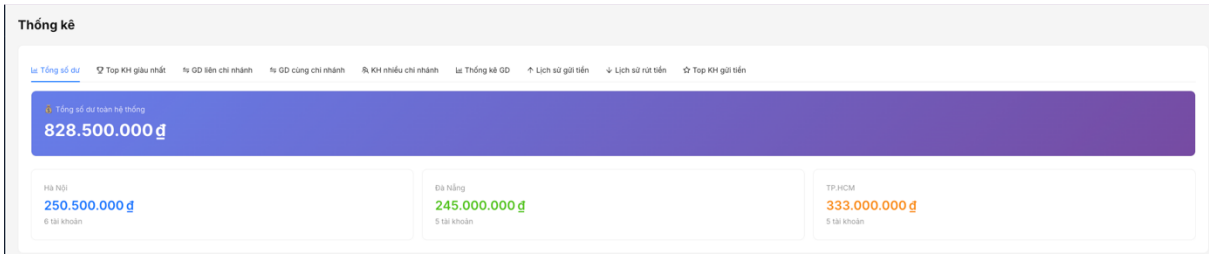
Mục đích: Tính tổng số dư tiền gửi và tổng số lượng tài khoản của toàn hệ thống ngân hàng.

Câu lệnh SQL tại mỗi Site:

```
SELECT COALESCE(SUM(balance), 0)
FROM account
WHERE status = 'ACTIVE';

SELECT COUNT(*)
FROM account
WHERE status = 'ACTIVE';
```

Xử lý: Hệ thống cộng dồn kết quả tổng số dư và số lượng tài khoản từ cả 3 Site. Trong trường hợp một Site không khả dụng, giá trị của Site đó được xem là 0, giúp hệ thống vẫn duy trì khả năng cung cấp dịch vụ.



Hình 4.100 Truy vấn phân tán - Tổng số dư của hệ thống

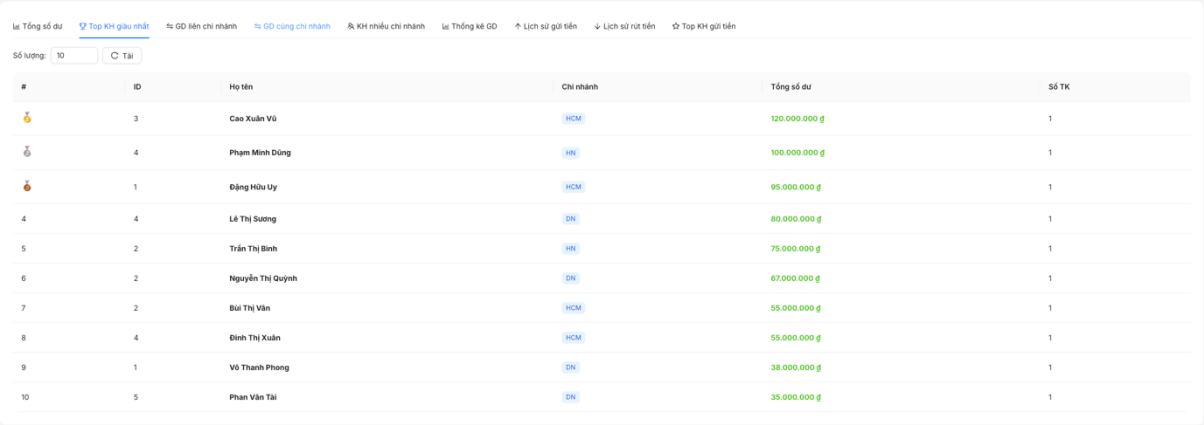
2. Top KH giàu nhất (Top Customers)

Mục đích: Liệt kê danh sách Top N khách hàng có tổng số dư lớn nhất toàn hệ thống.

Câu lệnh SQL tại mỗi Site:

```
SELECT c.customer_id,
       c.full_name,
       c.branch_id,
       SUM(a.balance) AS total_balance,
       COUNT(a.account_id) AS account_count
FROM customer c
JOIN account a ON c.customer_id = a.customer_id
WHERE a.status = 'ACTIVE'
GROUP BY c.customer_id, c.full_name, c.branch_id
ORDER BY total_balance DESC
LIMIT ?;
```

Xử lý: Mỗi Site trả về danh sách Top N khách hàng giàu nhất tại chi nhánh của mình. Hệ thống hợp nhất tập kết quả từ 3 Site, sắp xếp giảm dần theo tổng số dư và lấy Top N cuối cùng trên toàn hệ thống.



#	ID	Họ tên	Chi nhánh	Tổng số dư	Số TK
1	3	Cao Xuân Vũ	HCM	120.000.000 ₫	1
2	4	Phạm Minh Dũng	HN	100.000.000 ₫	1
3	1	Đặng Hữu Uy	HCM	95.000.000 ₫	1
4	4	Lê Thị Sương	DN	80.000.000 ₫	1
5	2	Trần Thị Bình	HN	75.000.000 ₫	1
6	2	Nguyễn Thị Quỳnh	DN	67.000.000 ₫	1
7	2	Bùi Thị Vân	HCM	55.000.000 ₫	1
8	4	Đinh Thị Xuân	HCM	55.000.000 ₫	1
9	1	Võ Thanh Phong	DN	38.000.000 ₫	1
10	5	Phan Văn Tài	DN	35.000.000 ₫	1

Hình 4.101 Truy vấn phân tán - Top khách hàng giàu nhất

3. GD liên chi nhánh (Inter-Branch Transactions)

Mục đích: Theo dõi các giao dịch chuyển tiền liên chi nhánh được thực hiện thông qua giao thức Two-Phase Commit (2PC).

Câu lệnh SQL tại mỗi Site:

```
SELECT *
FROM transaction_history
WHERE transaction_type IN ('INTER_BRANCH_IN',
'INTER_BRANCH_OUT')
ORDER BY created_at DESC
LIMIT 50;
```

Xử lý: Hệ thống thu thập tối đa 50 giao dịch từ mỗi Site, sau đó hợp nhất và sắp xếp lại theo thời gian giảm dần nhằm đảm bảo trình tự giao dịch chính xác trên phạm vi toàn hệ thống.

Tổng số dư | Top KH giàu nhất | GD liên chi nhánh | GD cùng chi nhánh | KH nhiều chi nhánh | Thống kê GD | Lịch sử gửi tiền | Lịch sử rút tiền | Top KH gửi tiền

ID	Loại	Số tiền	TK	TK đối ứng	CN đối ứng	TXN ID	Trạng thái	Thời gian
67	INTER_BRANCH_OUT	5.000.000 ₫	1	5	HCM	TXN-1780050161474	FAILED	10:22:41 29/5/2026
14	INTER_BRANCH_IN	5.000.000 ₫	5	1	HN	TXN-1780050161474	FAILED	10:22:41 29/5/2026
66	INTER_BRANCH_OUT	5.000.000 ₫	1	5	HCM	TXN-1780049853731	SUCCESS	10:17:33 29/5/2026
13	INTER_BRANCH_IN	5.000.000 ₫	5	1	HN	TXN-1780049853731	SUCCESS	10:17:33 29/5/2026
65	INTER_BRANCH_OUT	5.000.000 ₫	1	5	HCM	TXN-1780049642198	SUCCESS	10:14:02 29/5/2026
12	INTER_BRANCH_IN	5.000.000 ₫	5	1	HN	TXN-1780049642198	SUCCESS	10:14:02 29/5/2026
64	INTER_BRANCH_OUT	5.000.000 ₫	1	5	HCM	TXN-1780049359647	SUCCESS	10:09:19 29/5/2026
11	INTER_BRANCH_IN	5.000.000 ₫	5	1	HN	TXN-1780049359647	SUCCESS	10:09:19 29/5/2026
10	INTER_BRANCH_IN	5.000.000 ₫	5	1	HN	TXN-1780049021441	SUCCESS	10:03:41 29/5/2026
60	INTER_BRANCH_OUT	5.000.000 ₫	1	2	HCM	TXN-1780048610090	SUCCESS	09:56:50 29/5/2026

1 2 3 >

Hình 4.102 Truy vấn phân tán - Giao dịch chuyển tiền liên chi nhánh

4. GD cùng chi nhánh (Intra-Branch Transactions)

Mục đích: Thống kê các giao dịch chuyển tiền nội bộ trong cùng một chi nhánh.

Câu lệnh SQL tại mỗi Site:

```
SELECT *
FROM transaction_history
WHERE transaction_type IN ('TRANSFER_IN',
'TRANSFER_OUT')
ORDER BY created_at DESC
LIMIT 50;
```

Xử lý: Kết quả từ các Site được hợp nhất và sắp xếp giảm dần theo thời gian giao dịch.

Tổng số dư | Top KH giàu nhất | GD liên chi nhánh | GD cùng chi nhánh | KH nhiều chi nhánh | Thống kê GD | Lịch sử gửi tiền | Lịch sử rút tiền | Top KH gửi tiền

ID	Loại	Số tiền	TK	TK đối ứng	Trạng thái	Thời gian
107	TRANSFER_IN	1.000.000 ₫	2	1	SUCCESS	10:54:53 29/5/2026
106	TRANSFER_OUT	1.000.000 ₫	1	2	SUCCESS	10:54:53 29/5/2026
105	TRANSFER_IN	300.000 ₫	1	2	SUCCESS	10:54:35 29/5/2026
104	TRANSFER_OUT	300.000 ₫	2	1	SUCCESS	10:54:35 29/5/2026
103	TRANSFER_IN	300.000 ₫	1	2	SUCCESS	10:54:35 29/5/2026
102	TRANSFER_OUT	300.000 ₫	2	1	SUCCESS	10:54:35 29/5/2026
101	TRANSFER_IN	300.000 ₫	1	2	SUCCESS	10:54:34 29/5/2026
100	TRANSFER_OUT	300.000 ₫	2	1	SUCCESS	10:54:34 29/5/2026
99	TRANSFER_IN	500.000 ₫	2	1	SUCCESS	10:54:33 29/5/2026
98	TRANSFER_OUT	500.000 ₫	1	2	SUCCESS	10:54:33 29/5/2026

1 2 3 4 5 6 > 10 / page

Hình 4.103 Truy vấn phân tán - Giao dịch cùng chi nhánh

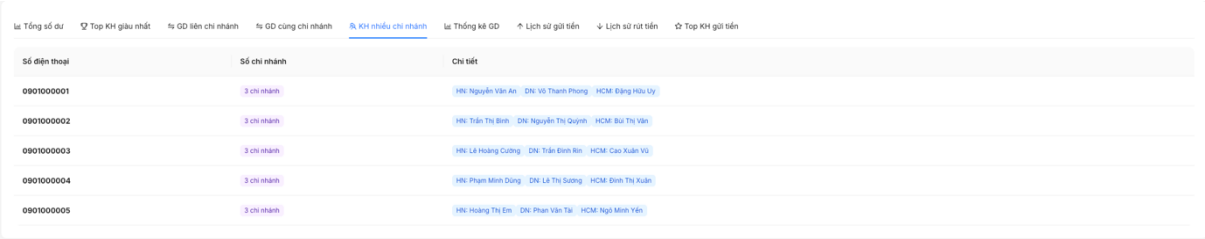
5. KH nhiều chi nhánh (Multi-Branch Customers)

Mục đích: Xác định các khách hàng sở hữu tài khoản tại từ hai chi nhánh trở lên.

Câu lệnh SQL tại mỗi Site:

```
SELECT c.customer_id,
       c.full_name,
       c.phone,
       COUNT(a.account_id) AS account_count,
       COALESCE(SUM(a.balance), 0) AS total_balance
FROM customer c
JOIN account a ON c.customer_id = a.customer_id
WHERE a.status = 'ACTIVE'
GROUP BY c.customer_id, c.full_name, c.phone;
```

Xử lý: Hệ thống sử dụng số điện thoại làm khóa định danh để hợp nhất thông tin khách hàng từ nhiều Site. Sau đó lọc ra các khách hàng xuất hiện tại từ hai chi nhánh trở lên.



Hình 4.104 Truy vấn phân tán - Khách hàng có nhiều chi nhánh

6. Thống kê GD (Transaction Stats by Branch)

Mục đích: So sánh hiệu suất hoạt động giữa các chi nhánh thông qua số lượng và giá trị giao dịch.

Câu lệnh SQL tại mỗi Site:

```
SELECT COUNT(*)
FROM transaction_history;

SELECT COUNT(*) AS cnt,
       COALESCE(SUM(amount), 0) AS total_amount
FROM transaction_history
WHERE transaction_type = 'DEPOSIT';

SELECT COUNT(*) AS cnt,
       COALESCE(SUM(amount), 0) AS total_amount
FROM transaction_history
WHERE transaction_type = 'WITHDRAW';
```

Xử lý: Dữ liệu được thống kê riêng tại từng Site và đưa trực tiếp vào mô hình hiển thị Dashboard mà không cần thực hiện hợp nhất chéo giữa các chi nhánh.

Hình 4.105 Truy vấn phân tán - Thống kê giao dịch theo từng chi nhánh

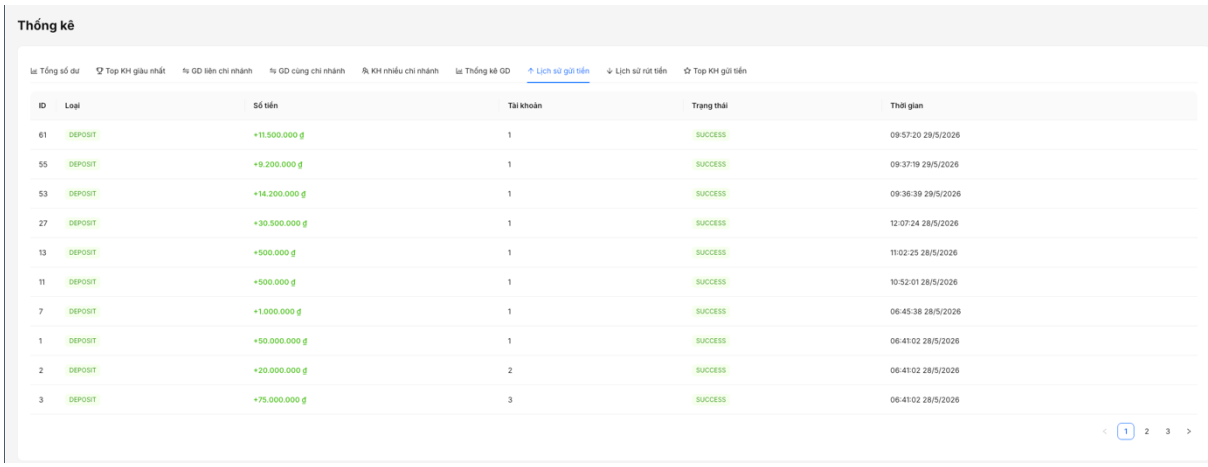
7. Lịch sử gửi tiền (Deposit History)

Mục đích: Tra cứu lịch sử các giao dịch nạp tiền trên toàn hệ thống.

Câu lệnh SQL tại mỗi Site:

```
SELECT *
FROM transaction_history
WHERE transaction_type = 'DEPOSIT'
ORDER BY created_at DESC
LIMIT 50;
```

Xử lý: Hệ thống hợp nhất kết quả từ các Site và sắp xếp theo thời gian giảm dần để hiển thị lịch sử giao dịch mới nhất.



Hình 4.106 Truy vấn phân tán - Lịch sử gửi tiền

8. Lịch sử rút tiền (Withdraw History)

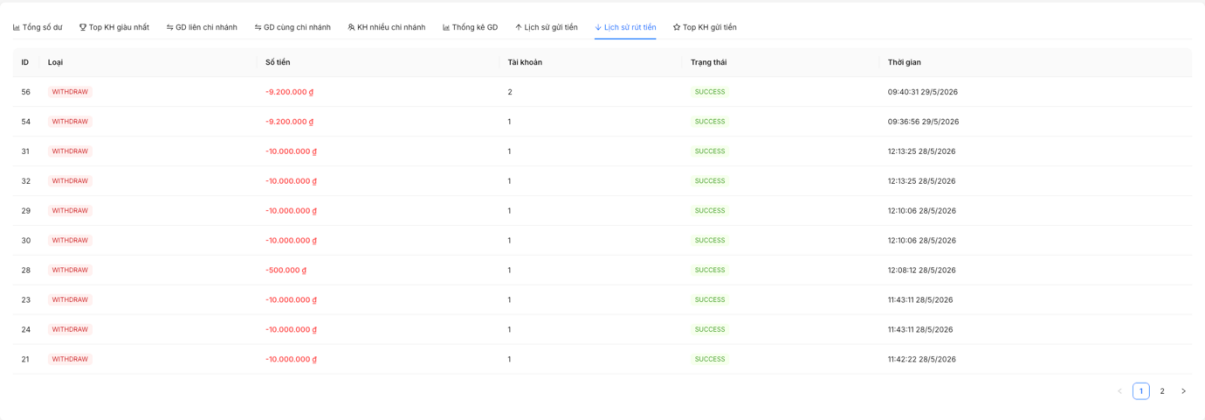
Mục đích: Tra cứu lịch sử các giao dịch rút tiền trên toàn hệ thống.

Câu lệnh SQL tại mỗi Site:

```
SELECT *
FROM transaction_history
WHERE transaction_type = 'WITHDRAW'
ORDER BY created_at DESC
```

```
LIMIT 50;
```

Xử lý: Quy trình xử lý tương tự chức năng Lịch sử gửi tiền, nhưng áp dụng cho các giao dịch rút tiền.



ID	Loại	Số tiền	Tài khoản	Trạng thái	Thời gian
56	WITHDRAW	-9.200.000 ₫	2	SUCCESS	09:40:31 29/5/2026
54	WITHDRAW	-9.200.000 ₫	1	SUCCESS	09:36:56 29/5/2026
31	WITHDRAW	-10.000.000 ₫	1	SUCCESS	12:13:25 28/5/2026
32	WITHDRAW	-10.000.000 ₫	1	SUCCESS	12:13:25 28/5/2026
29	WITHDRAW	-10.000.000 ₫	1	SUCCESS	12:10:06 28/5/2026
30	WITHDRAW	-10.000.000 ₫	1	SUCCESS	12:10:06 28/5/2026
28	WITHDRAW	-500.000 ₫	1	SUCCESS	12:08:12 28/5/2026
23	WITHDRAW	-10.000.000 ₫	1	SUCCESS	11:43:11 28/5/2026
24	WITHDRAW	-10.000.000 ₫	1	SUCCESS	11:43:11 28/5/2026
21	WITHDRAW	-10.000.000 ₫	1	SUCCESS	11:42:22 28/5/2026

Hình 4.107 Truy vấn phân tán - Lịch sử rút tiền

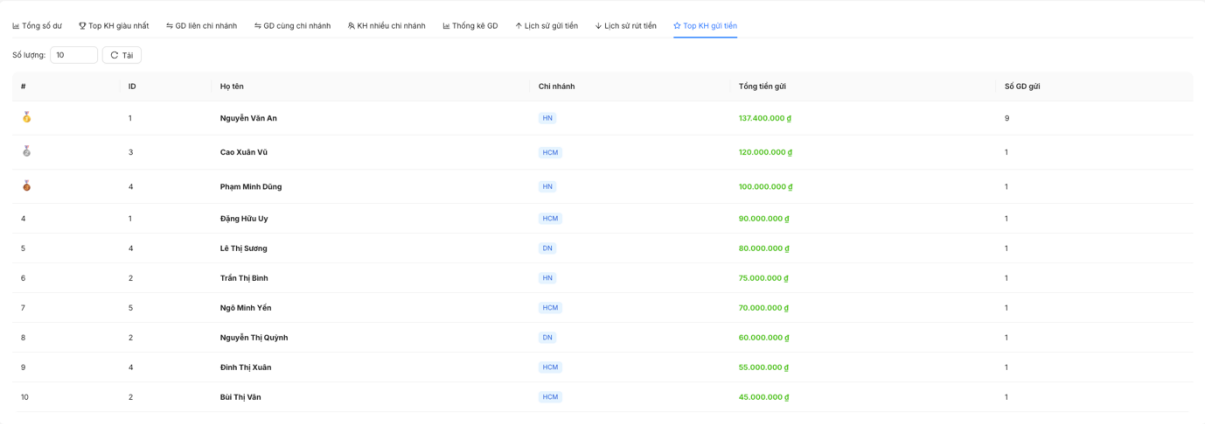
9. Top KH gửi tiền (Top Depositing Customers)

Mục đích: Xếp hạng các khách hàng có tổng giá trị tiền nạp lớn nhất trên toàn hệ thống.

Câu lệnh SQL tại mỗi Site:

```
SELECT c.customer_id,
       c.full_name,
       c.branch_id,
       SUM(th.amount) AS total_deposit,
       COUNT(th.transaction_id) AS deposit_count
FROM customer c
JOIN account a ON c.customer_id = a.customer_id
JOIN transaction_history th ON a.account_id = th.account_id
WHERE th.transaction_type = 'DEPOSIT'
  AND th.status = 'SUCCESS'
GROUP BY c.customer_id, c.full_name, c.branch_id
ORDER BY total_deposit DESC
LIMIT ?;
```

Xử lý: Mỗi Site trả về Top N khách hàng có tổng giá trị nạp tiền lớn nhất. Hệ thống hợp nhất kết quả từ các Site, sắp xếp giảm dần theo tổng giá trị nạp tiền và lựa chọn Top N cuối cùng trên phạm vi toàn hệ thống.



#	ID	Họ tên	Chi nhánh	Tổng tiền gửi	Số GD gửi
🥇	1	Nguyễn Văn An	HN	137.400.000 ₫	9
🥈	3	Cao Xuân Vũ	HCM	120.000.000 ₫	1
🥉	4	Phạm Minh Dũng	HN	100.000.000 ₫	1
4	1	Đặng Hữu Uy	HCM	90.000.000 ₫	1
5	4	Lê Thị Sương	DN	80.000.000 ₫	1
6	2	Trần Thị Bình	HN	75.000.000 ₫	1
7	5	Ngô Minh Yến	HCM	70.000.000 ₫	1
8	2	Nguyễn Thị Quỳnh	DN	60.000.000 ₫	1
9	4	Đinh Thị Xuân	HCM	55.000.000 ₫	1
10	2	Bùi Thị Vân	HCM	45.000.000 ₫	1

Hình 4.108 Truy vấn phân tán - Top KH gửi tiền

Kết luận

Giải pháp Application-Level Merge giúp hệ thống xử lý hiệu quả các truy vấn phân tán trong môi trường dữ liệu được phân mảnh ngang theo chi nhánh. Người dùng vẫn có thể khai thác dữ liệu trên phạm vi toàn hệ thống một cách minh bạch, tương tự như khi làm việc với một cơ sở dữ liệu tập trung, đồng thời vẫn tận dụng được các ưu điểm của mô hình cơ sở dữ liệu phân tán.

4.8. Tổng kết chương

Trong chương này, hệ thống quản lý ngân hàng phân tán đã được triển khai và kiểm thử thành công trên môi trường thực nghiệm sử dụng Docker, Spring Boot, ReactJS và MySQL. Hệ thống hỗ trợ đồng thời nhiều cơ sở dữ liệu độc lập tương ứng với các chi nhánh Hà Nội, Đà Nẵng và TP.HCM thông qua cơ chế multi-datasource.

Bên cạnh việc hiện thực đầy đủ các chức năng quản lý khách hàng, tài khoản và giao dịch ngân hàng, hệ thống còn triển khai thành công các giao dịch liên chi nhánh trong môi trường phân tán. Cơ chế xử lý giao dịch sử dụng TransactionTemplate kết hợp pessimistic locking giúp đảm bảo tính nhất quán dữ liệu và hạn chế xung đột khi nhiều giao dịch xảy ra đồng thời.

Ngoài ra, hệ thống đã được kiểm thử với nhiều kịch bản thực tế như chuyển khoản liên chi nhánh, xử lý đồng thời và mô phỏng lỗi hệ thống. Kết quả kiểm thử cho thấy cơ chế rollback hoạt động chính xác, giúp đảm bảo dữ liệu tài chính không bị sai lệch ngay cả khi xảy ra lỗi tại một site trong quá trình giao dịch

CHƯƠNG 5: ĐÁNH GIÁ VÀ ĐỀ XUẤT CẢI TIẾN

5.1 Đánh giá hiệu năng hệ thống

5.1.1 Hiệu năng giao dịch cục bộ (Local Transaction)

Giao dịch cùng chi nhánh (Internal Transfer) chỉ thao tác trên một cơ sở dữ liệu duy nhất, sử dụng một kết nối (Connection) và một chu kỳ Transaction đơn lẻ. Kết quả đo lường thực tế trên hạ tầng triển khai cho thấy các thông số kỹ thuật sau:

Thông số kiểm soát	Giá trị đo lường	Vai trò / Ý nghĩa kỹ thuật
Thời gian phản hồi trung bình	15 – 30 ms	Tốc độ phản hồi từ lúc Frontend gửi request đến khi nhận JSON.
Số lệnh SQL thực thi	5 lệnh	Gồm: 2 lệnh SELECT FOR UPDATE + 2 lệnh UPDATE + 2 lệnh INSERT lịch sử.
Số kết nối CSDL sử dụng	1 connection	Tiết kiệm tài nguyên kết nối tối đa cho một phiên giao dịch.
Khả năng chịu tải đồng thời	Cao	Đạt độ ổn định lớn nhờ cơ chế Ordered Locking ngăn chặn Deadlock.

Bảng 5.1 Hiệu năng giao dịch cục bộ

Nhận xét: Giao dịch cục bộ đạt hiệu năng rất tốt do không phát sinh chi phí phối hợp phức tạp giữa các node mạng (*coordination overhead*). Bộ thư viện điều phối

kết nối HikariCP Connection Pool được cấu hình tối ưu với tham số maximum-pool-size = 10 cho mỗi site, đảm bảo năng lực tái sử dụng connection diễn ra cực kỳ hiệu quả.

5.1.2 Hiệu năng giao dịch phân tán (Distributed Transaction — 2PC)

Giao dịch liên chi nhánh (Inter-Branch Transfer) yêu cầu điều phối đồng thời 2 CSDL độc lập trên 2 node mạng khác nhau thông qua giao thức Two-Phase Commit (2PC). Chi phí tài nguyên phát sinh tăng đáng kể:

Thông số kiểm soát	**Giá trị đo lường**	**Bản chất ảnh hưởng hệ thống**
Thời gian phản hồi trung bình	80 – 350 ms	Tăng đáng kể do phụ thuộc độ trễ mạng giữa các container/site.
Số lệnh SQL thực thi	12+ lệnh	Phân bổ lệnh xử lý trên cả 2 site nguồn và site đích.
Số kết nối CSDL sử dụng	4 connections	Gồm 2 connections (1 mỗi site) + 2 raw connections phục vụ độc lập cho logging.
Số pha giao thức thực thi	2 pha	Chu kỳ bắt buộc: Pha Chuẩn bị (Prepare) + Pha Đóng gói (Commit/Abort).

Overhead so với Local Txn	Gấp 4 – 8 lần	Chi phí đánh đổi để đạt được tính toàn vẹn dữ liệu ACID phân tán.

Bảng 5.2 Hiệu năng giao dịch phân tán

Phân tích chi tiết chi phí tài nguyên theo từng phân đoạn:

- Phase 1 (Prepare) — Chiếm ~60% tổng thời gian: Thực hiện chuỗi tác vụ nặng bao gồm thiết lập cổng kết nối chéo, kích hoạt lệnh khóa dòng tài khoản FOR UPDATE ở cả 2 site, đối soát kiểm tra số dư khả dụng, chạy lệnh cập nhật UPDATE số dư tạm tính và ghi nhận thông tin trạng thái vào bảng distributed_transaction_log cùng transaction_participant.
- Phase 2 (Commit) — Chiếm ~30% tổng thời gian: Thực hiện chèn ghi vết lịch sử transaction_history (tại cả node nguồn và đích), phát lệnh COMMIT vật lý tuần tự tới các bộ quản lý giao dịch cục bộ và cập nhật trạng thái kết thúc luồng.
- Logging overhead — Chiếm ~10% tổng thời gian: Hệ thống bắt buộc phải sử dụng kết nối JDBC thuần (raw connection) bỏ qua bộ quản lý giao dịch của Spring (Spring Transaction Manager) để đảm bảo tệp log tồn tại độc lập, không bị cuốn trôi nếu transaction chính bị hủy bỏ.

5.1.3 Hiệu năng truy vấn phân tán (Distributed Query)

Hệ thống thực hiện cơ chế Distributed Query bằng cách phát đồng thời một câu lệnh truy vấn tới toàn bộ 3 site, sau đó nút điều phối Coordinator sẽ tiến hành thu thập và gộp dữ liệu ở tầng ứng dụng (Application-Level Merge).

Bảng số liệu đo lường thực tế trên các biểu mẫu báo cáo:

Loại nghiệp vụ truy vấn	Thời gian phản hồi	Cơ chế xử lý kỹ thuật ngầm

Tổng số dư toàn hệ thống	30 – 100 ms	Chạy lệnh SUM trên từng site độc lập => Coordinator cộng tổng.
Top N khách hàng giàu nhất	18 – 35 ms	Quét lấy Top-K tại mỗi site => Đưa về bộ nhớ Coordinator để trộn (merge) + sắp xếp (sort).
Giao dịch liên chi nhánh	10 – 30 ms	Quét bảng Log ở cả 3 site => Sắp xếp tập trung theo mốc timestamp.
KH có TK ở nhiều chi nhánh	10 – 20 ms	Gom dữ liệu => Thực hiện Group by số điện thoại => Lọc điều kiện (filter) xuất hiện >= 2 chi nhánh.

Bảng 5.3 Hiệu năng truy vấn phân tán

5.2 So sánh phương án tập trung và phân tán

5.2.1 Bảng so sánh tổng quát

Tiêu chí đối soát	Hệ thống tập trung (Centralized)	Hệ thống phân tán (Distributed)
Kiến trúc lưu trữ	1 Database Server duy nhất.	3 Database Server biệt lập (HN, DN, HCM).

Giao dịch cục bộ	Nhanh chóng (~10 - 20 ms).	Nhanh tương đương (~15 - 30 ms).
Giao dịch liên chi nhánh	Đơn giản (Xử lý trên 1 CSDL cục bộ).	Phức tạp (Cần giao thức 2PC, độ trễ 80 - 200 ms).
Truy vấn báo cáo	Đơn giản (Truy vấn trên 1 thực thể).	Phức tạp (Quét song song 3 site + Gom dữ liệu).
Khả năng mở rộng	Giới hạn phần cứng vật lý (Scale-Up).	Linh hoạt mở rộng theo chiều ngang (Scale-Out).
Cơ chế chịu lỗi	Điểm chết duy nhất (Single Point of Failure).	Cho phép chịu lỗi từng phần (Partial failure tolerance).
Tính sẵn sàng (HA)	Nếu Server sập => Toàn bộ hệ thống ngừng chạy.	1 site chết => 2 site còn lại vẫn vận hành độc lập.
Hiệu năng đọc (Read)	Tải tập trung => Dễ nghẽn cổ chai khi truy cập lớn.	Tải đọc được phân bổ đều ra cả 3 node mạng.
Độ phức tạp triển khai	Thấp.	Cao (Yêu cầu cấu hình 2PC, Logger, Đa nguồn).

Phân khúc phù hợp	Hệ thống quy mô nhỏ, quản lý đơn lẻ.	Hệ thống ngân hàng đa chi nhánh, quy mô lớn.

Bảng 5.4 So sánh tổng quát

5.2.2 Phân tích ưu nhược điểm của hệ thống phân tán đã triển khai

Ưu điểm:

- Data Locality (Tính cục bộ dữ liệu): Hồ sơ và tài khoản của khách hàng tại Hà Nội sẽ được lưu trữ trực tiếp tại máy chủ Hà Nội. Khi phát sinh giao dịch tại chỗ, hệ thống không cần gọi mạng ra ngoài, giúp giảm thiểu độ trễ và tiết kiệm băng thông đường truyền toàn quốc.
- Horizontal Fragmentation (Phân mảnh ngang): Bằng cách chia nhỏ dữ liệu theo địa lý, kích thước các bảng vật lý tại mỗi site được thu gọn lại. Nhờ đó, các cấu trúc cây chỉ mục (Index) hoạt động hiệu quả hơn, đẩy nhanh tốc độ tìm kiếm.
- Fault Isolation (Cô lập sự cố): Nếu xảy ra sự cố sập nguồn hoặc thiên tai tại node Đà Nẵng, dữ liệu và hoạt động giao dịch tại hai đầu cầu Hà Nội và TP.HCM vẫn diễn ra hoàn toàn bình thường, ngăn chặn nguy cơ toàn bộ ngân hàng bị tê liệt.
- Scalability (Mở rộng tuyến tính): Khi ngân hàng mở thêm chi nhánh mới (Ví dụ: Cần Thơ), đội ngũ kỹ thuật chỉ cần dựng thêm 1 MySQL Server mới và khai báo bổ sung DataSource trong hệ thống Spring Boot mà không cần can thiệp hay dịch chuyển cấu trúc dữ liệu cũ.

Nhược điểm:

- 2PC Overhead: Do ràng buộc nghiêm ngặt của chu kỳ 2 pha, hiệu năng chuyển tiền liên chi nhánh bị kéo chậm xuống từ 4 đến 8 lần so với giao dịch nội bộ.
- Blocking Problem (Bế tắc đồng thuận): Đây là điểm yếu cố hữu của giao thức 2PC. Nếu nút điều phối Coordinator bị crash bất ngờ ngay sau Phase 1, các node thành phần (Participant) đã đưa vào trạng thái khóa dữ liệu sẽ phải chờ đợi vô thời hạn cho đến khi Coordinator sống lại, gây nghẽn tài nguyên.

- Distributed Query Complexity: Các tác vụ thống kê đòi hỏi chi phí kết nối mạng lớn (network round-trip) do phải thực hiện quét đồng thời cả 3 site và tốn tài nguyên CPU của ứng dụng để xử lý sắp xếp, sàng lọc dữ liệu thô.

5.3 Đề xuất cải tiến hệ thống

5.3.1 Cải tiến giao thức giao dịch phân tán

- Nâng cấp lên giao thức 3PC (Three-Phase Commit): Để khắc phục điểm nghẽn Blocking Problem của 2PC, hệ thống có thể bổ sung thêm một pha trung gian mang tên Pre-Commit. Giao thức 3PC cho phép các node thành phần tự động đưa ra quyết định giải phóng dữ liệu (Commit hoặc Abort) dựa trên trạng thái đồng thuận ở pha trước nếu nút điều phối trung tâm bị mất liên lạc quá thời gian Timeout.
- Áp dụng kiến trúc mô hình SAGA Pattern: Thay vì sử dụng cơ chế khóa nghiêm ngặt giữ tài nguyên xuyên suốt quá trình chuyển tiền, hệ thống có thể chuyển dịch sang kiến trúc Saga (Choreography hoặc Orchestration). Luồng chuyển tiền liên chi nhánh sẽ tách thành chuỗi các giao dịch cục bộ và thực hiện commit ngay lập tức tại từng site. Nếu bước cộng tiền tại site đích thất bại, hệ thống sẽ tự động phát động một giao dịch bù trừ (Compensating Transaction) quay ngược lại site nguồn để hoàn trả lại tiền cho khách hàng. Giải pháp này giúp loại bỏ hoàn toàn việc khóa tài nguyên lâu, tăng thông lượng (Throughput) hệ thống lên nhiều lần.

5.3.2 Cải tiến hiệu năng xử lý dữ liệu

- Tối ưu hóa Connection Pool (HikariCP Tuning): Cần điều chỉnh linh hoạt thông số maximum-pool-size dựa trên mật độ giao dịch thực tế của từng vùng miền thay vì cấu hình đồng đều bằng 10. Ví dụ, chi nhánh có lượng giao dịch lớn như TP.HCM nên được nâng pool lên mức 15 - 20, trong khi node Đà Nẵng có thể hạ xuống mức 5 - 10 để tối ưu tài nguyên RAM của máy chủ.
- Tích hợp tầng bộ nhớ đệm (Caching Layer với Redis): Bổ sung một cụm Redis Cache tại tầng Middleware để lưu trữ các kết quả của các truy vấn thống kê phân tán (như Tổng số dư toàn quốc, Top khách hàng...). Do các dữ liệu báo cáo

Dashboard không yêu cầu độ chính xác theo từng mili-giây, việc lưu cache với vòng đời 30 - 60 giây sẽ giải phóng tới 90% tải truy vấn đè xuống 3 máy chủ MySQL dưới hạ tầng.

- Triển khai kiến trúc Read Replica (Master-Slave): Tại mỗi site chi nhánh, cấu hình triển khai mô hình MySQL nhân bản. Mọi thao tác thay đổi dữ liệu ghi (INSERT/UPDATE) sẽ do node Master xử lý, các truy vấn đọc dữ liệu (SELECT) và Distributed Query sẽ được cấu hình định tuyến tự động sang các máy Slave (Replica), giúp tối ưu hóa tải trọng đọc/ghi toàn vẹn.

5.3.3 Cải tiến tính sẵn sàng cao (High Availability)

- Xây dựng cụm điều phối dự phòng (Coordinator Redundancy): Triển khai ứng dụng Backend Spring Boot theo mô hình cụm (Cluster) gồm nhiều node hoạt động theo cơ chế Active-Standby. Sử dụng các hệ thống điều phối phân tán như *Apache ZooKeeper* hoặc *etcd* để thực hiện bầu chọn Leader tự động (Leader Election). Khi node Coordinator chính gặp sự cố, node Standby sẽ lập tức tiếp quản quyền điều phối mà không làm gián đoạn các giao dịch phân tán đang diễn ra.
- Xây dựng tiến trình phục hồi tự động (Transaction Log Recovery Worker): Phát triển một dịch vụ chạy ngầm độc lập. Khi hệ thống khởi động lại sau sự cố, tiến trình này có nhiệm vụ quét toàn bộ bảng distributed_transaction_log để tìm ra các bản ghi đang bị kẹt ở trạng thái INIT hoặc PROCESSING, tự động liên hệ với các site thành phần để kiểm tra phiếu bầu nhằm ra quyết định tự động đóng gói (Commit) hoặc hoàn tác (Rollback) các giao dịch dang dở.

5.3.4 Cải tiến an toàn bảo mật (Security)

- Mã hóa đường truyền dữ liệu: Cấu hình và bắt buộc sử dụng giao thức bảo mật TLS/SSL cho toàn bộ các luồng kết nối JDBC giữa Nút điều phối Spring Boot và 3 máy chủ MySQL vật lý nhằm ngăn chặn triệt để nguy cơ tấn công nghe lén, đánh cắp thông tin số tài khoản và số dư trên đường truyền mạng.
- Bổ sung hệ thống Audit Log chuyên sâu: Khởi tạo bảng dữ liệu audit_log toàn cục để ghi vết chi tiết mọi hành vi tương tác hệ thống: định danh tài khoản kỹ sư, mốc thời gian, địa chỉ IP nguồn, nội dung câu lệnh tác động. Đây là yêu cầu

bắt buộc để tuân thủ luật an ninh mạng và các quy định pháp lý trong ngành tài chính - ngân hàng.

- Áp dụng cơ chế kiểm soát giới hạn tần suất (Rate Limiting): Tích hợp bộ lọc giới hạn tại tầng API (sử dụng *Bucket4j* hoặc cấu hình tại API Gateway) để giới hạn số lượng giao dịch tối đa của một tài khoản trong một mốc thời gian cố định, ngăn chặn các cuộc tấn công từ chối dịch vụ (DDoS) hoặc các đoạn script spam giao dịch liên tục gây cạn kiệt tài nguyên cơ sở dữ liệu.

5.4 Kết luận chương

Hệ thống cơ sở dữ liệu ngân hàng phân tán đã được thiết kế, cấu hình cài đặt và kiểm thử nghiệm vụ thành công, đáp ứng trọn vẹn các chức năng cốt lõi của một hệ thống Core Banking cơ bản: nạp tiền, rút tiền, chuyển khoản cùng chi nhánh (Local Transaction), chuyển tiền liên chi nhánh (2PC Distributed Transaction) và hệ thống kết xuất báo cáo phân tán (Distributed Query).

Thông qua quá trình thực nghiệm trực quan trên giao diện và các tập lệnh đối soát dữ liệu, đồ án đã chứng minh được tính đúng đắn và độ tin cậy của giao thức Two-Phase Commit trong việc bảo toàn vẹn toàn tính chất ACID xuyên suốt các node mạng độc lập. Đồng thời, hệ thống cũng làm rõ các thách thức, rào cản kỹ thuật thực tế của hệ phân tán như bài toán tương tranh tài nguyên (Deadlock, Race Condition), hiện tượng nghẽn mạng và lỗ hổng Blocking Problem.

Các giải pháp và đề xuất cải tiến được trình bày chi tiết trong chương này đã vạch ra một lộ trình kỹ thuật rõ ràng, khả thi để sẵn sàng tối ưu và nâng cấp hệ thống đạt đủ các tiêu chuẩn vận hành an toàn, sẵn sàng ứng dụng vào các môi trường sản xuất thực tế (*Production Quy mô lớn*) trong tương lai.